

从 WaveNet 到 Omni：语音合成教科书

从零开始用 PyTorch 构建猫娘语音助手

Neko Speech Team

2026-06-29

Contents

1	Ch01: Audio Fundamentals — Neko 的第一课：声音是什么？	6
1.1	本章导学	6
1.2	1.1 波形与采样：声音的数字化身	6
1.3	1.2 FFT：从时域到频域	8
1.4	1.3 STFT：短时傅里叶变换	8
1.5	1.4 Mel 频谱：模拟人耳的感知	10
1.6	1.5 Griffin-Lim：从频谱重建波形	13
1.7	1.6 本章小结	15
1.8	习题	16
1.9	目录结构	16
2	Ch02: Tacotron2 — Neko 学会说话	17
2.1	本章导学	17
2.2	2.1 模型架构总览	17
2.3	2.2 Encoder：把文本变成“声音的特征”	18
2.4	2.3 Location-Sensitive Attention：让对齐不再乱跳	19
2.5	2.4 Decoder：自回归生成 Mel	19
2.6	2.5 PostNet：最后的打磨	20
2.7	2.6 训练	20
2.8	2.7 推理与声码器	21
2.9	2.8 本章小结	22
2.10	习题	22
2.11	目录结构	23
3	Ch03: WaveNet — Neko 的声音变好听了	24
3.1	本章导学	24
3.2	3.1 为什么需要声码器？	24
3.3	3.2 因果卷积：只看过去，不看未来	25
3.4	3.3 扩张卷积与感受野	26
3.5	3.4 门控激活单元	27
3.6	3.5 WaveNet 完整架构	28
3.7	3.6 训练	29
3.8	3.7 推理：Mel → 波形	30

3.9	3.8 本章小结	30
3.10	习题	31
3.11	目录结构	32
4	Ch04: FastSpeech2 – Neko 学会了快速说话	33
4.1	本章导学	33
4.2	4.1 架构总览	33
4.3	4.2 Duration Predictor: 每个音素说多久?	34
4.4	4.3 Length Regulator: 把文本拉伸到语音长度	35
4.5	4.4 Pitch & Energy: 控制韵律	35
4.6	4.5 FFT Block: 前馈 Transformer	36
4.7	4.6 训练	37
4.8	4.7 推理	38
4.9	4.8 本章小结	39
4.10	习题	39
4.11	目录结构	39
5	Ch05: VITS — 端到端语音合成	41
5.1	本章导学	41
5.2	5.1 VITS 架构总览	42
5.3	5.2 VAE: 变分自编码器	42
5.4	5.3 归一化流 (Normalizing Flow)	44
5.5	5.4 GAN 对抗训练	45
5.6	5.5 核心模块详解	46
5.7	5.6 Monotonic Alignment Search (MAS)	48
5.8	5.7 训练	48
5.9	5.8 推理	49
5.10	5.9 端到端 vs 两阶段对比	50
5.11	5.10 本章小结	51
5.12	习题	51
5.13	目录结构	52
6	Ch06: Neural Audio Codec — Neko 的音频压缩术	53
6.1	本章导学	53
6.2	6.1 VQ-VAE 基础: 用离散码本表示连续向量	54
6.3	6.2 RVQ 残差向量量化: 从粗到细的多层逼近	55
6.4	6.3 EnCodec 架构: Encoder-Decoder 设计	56
6.5	6.4 训练与损失函数	57
6.6	6.5 实验	58
6.7	6.6 与 EnCodec / SoundStream 的关系	59
6.8	6.7 Codec 作为 Audio Tokenizer 的意义	60
6.9	6.8 遗留问题: 如何用这些 Token 做 TTS?	60
6.10	参考文献	61
6.11	代码文件	61
7	Ch07: VALL-E — Neko 学会了克隆声音	62
7.1	本章导学	62
7.2	7.1 音频 Codec: 从连续到离散	63

7.3	7.2 AR 模型：用 GPT 的方式预测音频 Token	64
7.4	7.3 NAR 模型：并行填充细节	65
7.5	7.4 零样本声音克隆：为什么 3 秒就够？	66
7.6	7.5 从零实现 VALL-E：代码走读	67
7.7	7.6 训练与推理	68
7.8	7.7 VALL-E 与工业界的演进	70
7.9	7.8 本章小结	71
7.10	习题	71
7.11	参考文献	72
7.12	目录结构	72
8	Ch08: Modern TTS Models — Flow Matching, Zero-Shot, and Controllability	73
8.1	0. 本章导学	73
8.2	1. F5-TTS: Flow Matching + DiT	74
8.3	2. CosyVoice: Zero-Shot Voice Cloning	75
8.4	3. IndexTTS: Controllable TTS	76
8.5	4. 模型对比	77
8.6	5. 工业界应用现状 (2024-2026)	78
8.7	6. 运行代码	78
8.8	7. 延伸阅读	79
8.9	8. 思考题	80
8.10	9. 文件清单	80
9	Ch09: GPT-SoVITS – Few-Shot 声音克隆	82
9.1	本章导学	82
9.2	9.1 系统架构总览	83
9.3	9.2 语义 Token：为什么 $n_q=1$ 就够了？	84
9.4	9.3 AR 模型：GPT-style Transformer	85
9.5	9.4 SoVITS 声码器：VITS 变体	88
9.6	9.5 两阶段训练	90
9.7	9.6 推理流程	92
9.8	9.7 代码走读	93
9.9	9.8 与 VALL-E 的对比	95
9.10	9.9 ONNX 导出	95
9.11	9.10 本章小结	97
9.12	习题	97
9.13	目录结构	98
10	Ch10: VoxCPM — 猫娘不再需要 Tokenizer	99
10.1	本章导学	99
10.2	10.1 AudioVAE：把声音变成连续潜空间	100
10.3	10.2 为什么不要 Tokenizer？	101
10.4	10.3 TSLM + FSQ：语义规划	102
10.5	10.4 RALM：声学细化	104
10.6	10.5 CFM：条件流匹配	105
10.7	10.6 完整代码走读	106
10.8	10.7 训练与推理	108
10.9	10.8 与工业界的关系	110

10.10练习	110
10.11参考资料	111
11 Ch11: MiniMind-O — Neko Learns to Listen, See, and Speak	112
11.1 本章导学	112
11.2 11.1 从 TTS 到 Omni	113
11.3 11.2 Thinker-Talker 架构	113
11.4 11.3 音频输入：冻结编码器	113
11.5 11.4 桥接层：为什么不用最后一层？	113
11.6 11.5 音频输出：Mimi Codec	113
11.7 11.6 多 Token 预测 (MTP)	113
11.8 11.7 序列格式：9 条流的故事	113
11.9 11.8 流式生成	113
11.10 11.9 声音克隆	113
11.11 11.10 训练流水线	113
11.12 11.11 VAD 与打断 (Barge-In)	113
11.13 11.12 从零实现 SimpleOmni	113
11.14 11.13 实验与对比	113
11.15 代码	113
11.16 参考资料	114
11.17 前置知识	114

从 WaveNet 到 Omni

语音合成教科书

从零开始用 PyTorch 构建猫娘语音助手

Neko Speech Team

2026-06-29

喵 ~ 和猫娘一起学语音合成吧!

Contents

1 Ch01: Audio Fundamentals — Neko 的第一课：声音是什么？

在让 Neko 学会说话之前，她需要先理解“声音”本身。

这一章建立所有现代 Audio AI 模型共同依赖的基础。

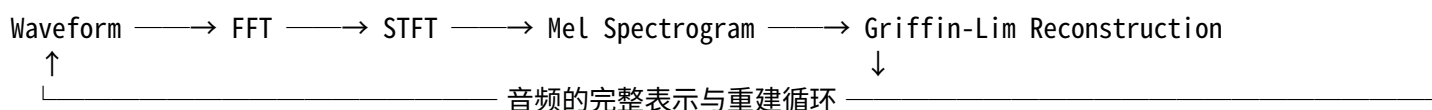
1.1 本章导学

1.1.1 为什么学这一章?

Tacotron、VITS、GPT-SoVITS、VALL-E、Omni.....这些模型的输入或输出，几乎都离不开同一个东西：**Mel Spectrogram (梅尔频谱)**。

如果你跳过这一章直接学 Tacotron，你会遇到：- 为什么模型输出的是 80 维向量而不是波形？- Griffin-Lim 是什么？为什么音质这么差？- VITS 为什么说“不需要声码器”？

这些问题都会迫使你**不断回头补基础**。所以，我们先一次性把音频表示的完整链条走完：



1.1.2 学习路线

节	内容	目标
1.1	波形与采样	理解数字音频的时域表示
1.2	FFT: 从时域到频域	理解频率分解
1.3	STFT: 时频分析	理解语音是非稳态信号
1.4	Mel 频谱	理解人耳感知与非线性频率刻度
1.5	Griffin-Lim 重建	理解“从频谱回到波形”的问题
1.6	动手实验	运行全部 Demo, 眼见为实

1.2 1.1 波形与采样：声音的数字化身

1.2.1 1.1.1 什么是声音?

声音是**空气压力的振动**。当声带振动、琴弦颤动、或扬声器膜片推动空气时，周围的空气分子就会形成疏密相间的波动。

人耳听到的是**压力随时间的变化**。如果我们把压力变化记录下来，就得到了**波形 (Waveform)**。

1.2.2 1.1.2 采样与数字化

真实世界的声音是**连续**的，但计算机只能处理**离散**的数字。所以需要两步：

采样 (Sampling): 每隔固定时间测一次压力值。- 采样率 (Sample Rate): 每秒采多少个点。语音常用 16kHz (每秒 16000 个样本)。- 奈奎斯特定理: 采样率必须 $> 2 \times$ 最高频率, 才能完整还原信号。人耳上限约 20kHz, 所以 CD 用 44.1kHz。

量化 (Quantization): 把每个采样值映射到有限精度的整数。- 16-bit PCM: 每个样本用 16 位表示, 范围 $[-32768, 32767]$ 。

Neko 笔记: 你可以把采样想象成给波形 “拍照” —— 拍得越密 (采样率越高)、像素越好 (量化位数越高), 还原得越真。



Figure 1: Neko 讲解采样与数字化

1.2.3 1.1.3 动手: 生成并观察波形

运行代码, 生成一个 440Hz 的正弦波 (A4 音, 钢琴中央 A):

```
cd chapters/ch01_audio_fundamentals/code
python 01_waveform.py --generate
```

输出到 ../outputs/: - waveform.wav — 可播放的音频 - waveform.png — 时域波形图

观察重点： - 上图是完整波形，下图放大 20ms。你会发现波形是**周期性的**——这就是“音高”的来源。

1.3 1.2 FFT：从时域到频域

1.3.1 1.2.1 为什么需要 FFT？

波形告诉我们“什么时候声音大”，但没告诉我们“声音由哪些频率组成”。

傅里叶变换的核心思想：任何信号都可以表示为不同频率正弦波的叠加。

$$X(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

1.3.2 1.2.2 快速傅里叶变换 (FFT)

FFT 是傅里叶变换的高效算法，把 $O(N^2)$ 降到 $O(N \log N)$ 。

输入：时域波形 (N 个点) 输出：频域幅度谱 (N/2+1 个频率 bins)

1.3.3 1.2.3 动手：观察频谱

```
python 02_fft.py --generate
```

输出：../outputs/fft.png

观察重点： - 上图是线性幅度谱，你会看到 440Hz 和 1320Hz (3 次谐波) 两个尖峰。 - 下图是对数刻度 (dB)，更适合观察动态范围大的信号。

Neko 笔记： FFT 假设信号是**稳态**的——频率成分不随时间变化。但语音不是稳态的！这就需要 STFT。

1.4 1.3 STFT：短时傅里叶变换

1.4.1 1.3.1 语音是非稳态信号

说“你好”时，“n”、“i”、“h”、“a”、“o”的发音频率完全不同。FFT 只能告诉你整段音频里有哪些频率，但不知道这些频率什么时候出现。

从时域到频域：FFT快速傅里叶变换

—— 把信号从“时间”搬到“频率”的魔法桥梁



Figure 2: Neko 讲解 FFT

1.4.2 1.3.2 STFT 的核心思想

加窗 → 逐帧 FFT → 拼接

1. 用一个窗函数（通常是汉明窗）截取一小段音频（20-50ms）
2. 对这一小段做 FFT
3. 窗向右滑动（hop length，通常 10ms）
4. 重复，把所有帧的频谱竖着拼起来

$$X[m, \omega] = \sum_n x[n] w[n - m] e^{-j\omega n}$$

其中 $w[n - m]$ 就是滑动的窗。

1.4.3 1.3.3 动手：观察时频图

```
python 03_stft.py --generate
```

输出：../outputs/spectrogram.png

观察重点：- 生成的是一个 chirp 信号（频率从 200Hz 线性增加到 2000Hz）。- 频谱图上应该看到一条斜线——这是时频分析的意义：你能“看到”频率随时间的变化。

1.5 1.4 Mel 频谱：模拟人耳的感知

1.5.1 1.4.1 人耳的非线性听觉

人耳对低频敏感、对高频不敏感。- 100Hz → 200Hz，你明显感到音高翻倍了。- 8000Hz → 8100Hz，你几乎听不出区别。

1.5.2 1.4.2 Mel 刻度

Mel 刻度模拟了人耳的这种非线性感知：

$$\text{mel} = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

- 低频区：Mel 值增长快（人耳敏感）
- 高频区：Mel 值增长慢（人耳不敏感）

1.5.3 1.4.3 Mel 滤波器组

STFT 给出的是线性频率轴的频谱。Mel 频谱把它转换为 Mel 刻度：

1. 在 Mel 刻度上均匀放置若干三角滤波器（通常 80 个）
2. 每个滤波器覆盖一段频率范围，中心在 Mel 刻度上等距
3. 用这些滤波器对线性频谱做加权求和



Figure 3: Neko 讲解 STFT



Figure 4: Neko 讲解 Mel 频谱

1.5.4 1.4.4 Log-Mel Spectrogram

最后取对数：

$$\text{Log-Mel} = \log(\text{MelFilter} \cdot |\text{STFT}| + \epsilon)$$

为什么取对数？ - 人耳对响度的感知也是对数的（分贝刻度） - 压缩动态范围，让数值更稳定

这就是 Tacotron/VITS 等模型的标准输入！ 80-bin Log-Mel，帧移 256 样本（约 16ms @ 16kHz）。

1.5.5 1.4.5 动手：观察 Mel 频谱

```
python 04_mel.py --generate
```

输出：../outputs/mel.png、../outputs/mel_filters.png

观察重点： - mel_filters.png：三角滤波器在低频密集、高频稀疏。 - mel.png：模拟语音的 Mel 频谱，可以看到共振峰（formants）——这些是区分元音的关键。

1.6 1.5 Griffin-Lim：从频谱重建波形

1.6.1 1.5.1 问题：相位丢失

STFT 把时域信号变成复数谱： $X = |X| \cdot e^{j\phi}$ 。

我们保存/使用的通常是幅度谱 $|X|$ ，相位 ϕ 被丢掉了。

从幅度谱逆变换回时域，如果直接用随机相位，声音会像“金属噪音”。

1.6.2 1.5.2 Griffin-Lim 算法

Griffin-Lim 是一个迭代算法，通过约束“逆变换后的信号再正变换，其幅度应该接近原幅度”来估计相位：

1. 随机初始化相位
2. 逆 STFT $\rightarrow$ 时域信号
3. 正 STFT $\rightarrow$ 新复数谱
4. 保留新相位，替换回原始幅度
5. 重复 30-100 次

1.6.3 1.5.3 局限性

Griffin-Lim 能重建出可听的语音，但： - 有“金属感”或“嗡嗡声” - 高频细节丢失 - 音质远不如原始录音

这就是为什么需要神经声码器（WaveNet / HiFi-GAN）的原因。



Figure 5: Neko 讲解 Griffin-Lim 重建

1.6.4 1.5.4 动手：对比原始与重建

```
python 05_reconstruct.py --input ../outputs/waveform.wav --n-iter 30
```

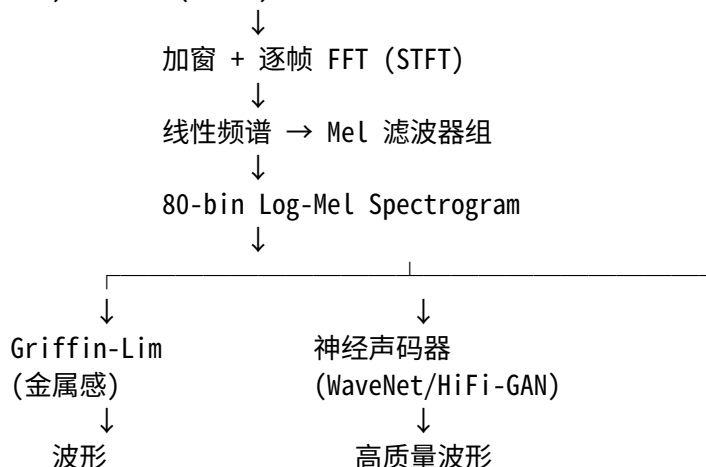
输出：../outputs/reconstructed.wav、../outputs/original_vs_reconstructed.png

观察重点：- 听 waveform.wav 和 reconstructed.wav，对比差异。- 看对比图：差异（最下图）不是零——这就是相位信息丢失造成的损失。

1.7 1.6 本章小结

1.7.1 核心链条

模拟声波 → 采样(16kHz) → 量化(16bit) → 数字波形



1.7.2 关键概念

概念	作用	本章代码
采样率	决定可还原的最高频率	01_waveform.py
FFT	时域 → 频域	02_fft.py
STFT	时域 → 时频表示	03_stft.py
Mel Filter	模拟人耳感知	04_mel.py
Griffin-Lim	幅度谱 → 波形（估计相位）	05_reconstruct.py

1.7.3 遗留问题

Griffin-Lim 重建音质差。Tacotron 论文最初就是用 Griffin-Lim 作为声码器，音质自然不够好。下一章，Neko 将学会用 WaveNet 生成更自然的波形。

1.7.4 参考文献

- [1] 语音合成基础 (3)——关于梅尔频谱你想知道的都在这里. 知乎.

- [2] Wang et al., 2018. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions.
 - [3] Griffin & Lim, 1984. Signal Estimation from Modified Short-Time Fourier Transform.
-

1.8 习题

1. 如果把采样率从 16kHz 降到 8kHz, FFT 的最高频率会发生什么变化? 对语音质量有什么影响?
 2. 为什么 STFT 的窗长通常是 20-50ms? 窗太长或太短分别有什么问题?
 3. Mel 刻度公式中, 为什么是 $f/700$ 而不是 $f/1000$? 这个 700 有什么物理意义?
 4. 运行 04_mel.py, 观察 mel_filters.png。为什么低频区的三角滤波器更密集?
 5. 把 05_reconstruct.py 的 --n-iter 从 30 改为 5 和 100, 对比重建质量。迭代次数越多越好吗?
-

1.9 目录结构

```
ch01_audio_fundamentals/
├── README.md           # 本章教程 (你正在看的文件)
├── code/               # 代码
│   ├── 01_waveform.py
│   ├── 02_fft.py
│   ├── 03_stft.py
│   ├── 04_mel.py
│   └── 05_reconstruct.py
└── outputs/           # 运行输出 (.gitignore)
```


2 Ch02: Tacotron2 — Neko 学会说话

有了音频基础，Neko 终于可以尝试把文字变成声音了。

这一章，我们实现第一个端到端神经 TTS 模型。

2.1 本章导学

2.1.1 为什么学 Tacotron2?

在 Tacotron 之前，语音合成的路线是：

文本分析 → 音素序列 → HMM/GMM 声学模型 → 参数声码器 → 波形

这条路线的问题：1. **文本分析依赖人工规则**（分词、注音、韵律标注）2. **HMM 假设每个音素的状态转移是马尔可夫的**，无法捕捉长程依赖 3. **参数声码器（如 MLSA）过度平滑**，音质像“机器人”

Tacotron2 的革命性在于：**端到端**——直接从字符/音素输入，输出 Mel Spectrogram，无需人工特征工程。

2.1.2 Tacotron2 之前的演进

模型	年份	核心改进	问题
Tacotron	2017	首个端到端 Seq2Seq TTS	对齐不稳定（跳字、重复）
Tacotron2	2018	Location-Sensitive Attention + WaveNet 声码器	自回归慢，需预训练声码器

2.1.3 学习路线

节	内容	目标
2.1	模型架构总览	理解 Encoder-Attention-Decoder-PostNet 流程
2.2	Encoder	文本 → 时序特征
2.3	Location-Sensitive Attention	解决对齐问题
2.4	Decoder	自回归生成 Mel
2.5	PostNet	细化频谱细节
2.6	训练	用真实数据训练
2.7	推理	生成 Mel + Griffin-Lim 声码器

2.2 2.1 模型架构总览

Tacotron2 的核心流程：

Text (字符序列)

↓

Encoder (Embedding + Conv + BiLSTM)

↓ (T_text, encoder_dim)

Location-Sensitive Attention

↓ (context向量, 每步一个)
 Decoder (PreNet + LSTM, 自回归)
 ↓ (每步预测1帧Mel)
 Mel Spectrogram (T_{mel} , 80)
 ↓
 PostNet (Conv, 残差修正)
 ↓
 Refined Mel Spectrogram
 ↓
 Vocoder (WaveNet / Griffin-Lim) → Waveform

2.2.1 为什么这个结构?

Encoder: 文本是离散的, 需要先变成连续的向量序列。

Attention: 文本长度和语音长度不对等 (“你好” 2 个字符 → 约 100 帧 Mel)。Attention 自动学习这种对齐。

Decoder: 语音是时序信号, 天然适合自回归生成。

PostNet: Decoder 输出的 Mel 有 “毛刺”, PostNet 用卷积平滑它。

2.3 2.2 Encoder: 把文本变成 “声音的特征”

2.3.1 2.2.1 结构

字符 Embedding (512维)
 ↓
 3层 Conv1d (kernel=5) + ReLU + BN + Dropout
 ↓
 BiLSTM ($256 \times 2 = 512$ 维)
 ↓
 Encoder Output: (B, T_{text} , 512)

2.3.2 2.2.2 为什么用 Conv1d + BiLSTM?

- **Conv1d:** 提取局部上下文 (类似 n-gram), 比纯 RNN 更稳定
- **BiLSTM:** 捕捉长程依赖, 且双向编码让每个位置都能看到全文

2.3.3 2.2.3 代码位置

code/model.py 中的 Encoder 类。

2.4 2.3 Location-Sensitive Attention: 让对齐不再乱跳

2.4.1 2.3.1 Tacotron1 的对齐问题

Tacotron1 使用普通 Attention，经常出现：- **跳字**：注意力权重突然跳到很远的位置，漏掉中间的字 - **重复**：注意力在某个位置来回震荡，导致同一个字读多遍 - **漏字**：注意力从来没访问到某个位置

2.4.2 2.3.2 Location-Sensitive Attention 的解决思路

核心思想：让 Attention 知道“上一秒我看了哪里”。

普通 Attention：

$$\text{score} = v^T \tanh(W_{enc} \cdot enc + W_{dec} \cdot dec)$$

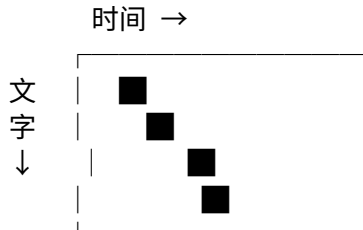
Location-Sensitive Attention：

$$\text{score} = v^T \tanh(W_{enc} \cdot enc + W_{dec} \cdot dec + W_{loc} \cdot \text{conv}(\alpha_{prev}))$$

其中 α_{prev} 是上一时刻的注意力权重，先做一个 Conv1d 提取“位置特征”，再加入到 score 中。
这样 Attention 就被约束了：只能缓慢向前移动，不能跳跃。

2.4.3 2.3.3 可视化

训练良好的 Tacotron2，Attention 权重应该呈现一条**清晰的对角线**：



如果 Attention 是混乱的斑块，说明训练出了问题。

2.5 2.4 Decoder: 自回归生成 Mel

2.5.1 2.4.1 Teacher Forcing

训练时，Decoder 的输入不是上一帧的**预测**，而是**真实的**上一帧 Mel。这称为 **Teacher Forcing**。
好处：训练稳定，梯度传播直接。坏处：训练和推理不一致（推理时没有 ground truth）。

2.5.2 2.4.2 PreNet: 信息瓶颈

Decoder 输入先过 PreNet（2 层 FC + Dropout），把 80 维 Mel 压到 256 维。

Neko 笔记：PreNet 强制模型压缩信息，防止 Decoder 直接“抄”上一帧。类似正则化效果。

2.5.3 2.4.3 双输出头

Decoder 每一步输出两个东西：1. **Mel 帧**：80 维向量 2. **Stop Token**：二分类（是否结束生成）
Stop Token 让模型自己决定 “这段话说完了”。

2.6 2.5 PostNet: 最后的打磨

2.6.1 2.5.1 为什么需要 PostNet?

Decoder 是 LSTM，擅长时序建模，但不擅长局部细节修正。

PostNet 是 5 层 Conv1d，专门做 “局部平滑”：- 修正 Mel 频谱的小波动 - 增强谐波结构 - 残差连接：输出 = 输入 + 卷积修正

2.6.2 2.5.2 残差连接的意义

$$\text{output} = \text{input} + \text{conv}(\text{input})$$

模型只需要学习 “差值”（残差），比从零学习完整映射更容易。

2.7 2.6 训练

2.7.1 2.6.1 损失函数

Tacotron2 使用三个损失：

$$\mathcal{L} = \mathcal{L}_{mel}^{before} + \mathcal{L}_{mel}^{after} + \mathcal{L}_{stop}$$

- $\mathcal{L}_{mel}^{before}$ ：Decoder 直接输出的 Mel 与 GT 的 MSE
- $\mathcal{L}_{mel}^{after}$ ：PostNet 修正后的 Mel 与 GT 的 MSE
- $\mathcal{L}_{stop}$ ：Stop Token 的 BCE

2.7.2 2.6.2 数据准备

下载猫娘数据集 (ModelScope)

```
cd data
```

```
python download_neko_1k.py --output-dir processed --num-samples 1000
```

输出格式：

```
processed/
```

```
|—— wav/000001.wav
```

```
|—— train.list          # wav_path|speaker|language|text
```

```
|—— metadata.csv
```

注意：原始数据是 24kHz，训练脚本会自动重采样到 16kHz。超过 25 秒的音频会被过滤以避免 OOM。

2.7.3 2.6.3 启动训练

```
cd chapters/ch02_tacotron/code
python train.py \
  --data-dir ../../data/processed \
  --epochs 50 \
  --batch-size 4
```

预期现象： - 前 10 个 epoch loss 快速下降 - 20-50 epoch Mel MSE 进入平台期 - 如果 loss 不下降，检查数据是否正确加载 - 每 10 epoch 自动保存 checkpoint 到 ../checkpoints/

2.7.4 2.6.4 训练技巧

1. **Gradient Clipping:** max norm = 1.0, 防止 RNN 梯度爆炸
 2. **Teacher Forcing Ratio:** 本实现使用 100% teacher forcing (简化版)
 3. **Batch Size:** 如果显存不够, 用 4 甚至 2
-

2.8 2.7 推理与声码器

2.8.1 2.7.1 自回归推理

训练时用的是 Teacher Forcing。推理时, Decoder 的输入是**上一帧的预测**:

第 1 帧: 输入 = 零向量 → 预测第 1 帧 Mel
第 2 帧: 输入 = 第 1 帧预测 → 预测第 2 帧 Mel
第 3 帧: 输入 = 第 2 帧预测 → ...

直到 Stop Token > 0.5 或达到最大长度。

2.8.2 2.7.2 Griffin-Lim 声码器

Tacotron2 论文用的是 WaveNet 声码器, 但 WaveNet 很慢。

本章先用 **Griffin-Lim** (Ch01 已实现) 作为占位声码器, 让整条链路能跑通。

基础推理

```
python inference.py \
  --checkpoint ../checkpoints/tacotron_final.pt \
  --text " 你好, 我是猫娘。" \
  --output ../outputs/neko_output.wav
```

端到端测试 (带计时和 RTF 统计)

```
python test_tts.py \
  --checkpoint ../checkpoints/tacotron_epoch_50.pt \
  --text " 你好, 我是猫娘。" \
  --output ../outputs/test_output.wav
```

2.8.3 2.7.3 预期效果

- v0.1 (早期训练): 可能输出噪音或单个音的重复
- v0.3 (训练充分): 能听出语调轮廓, 但吐字不清
- v0.6 (加上 WaveNet 声码器): 音质大幅提升

这是正常的! Tacotron 训练需要大量数据和计算。本章的目标是理解结构和跑通流程。

2.9 2.8 本章小结

2.9.1 Tacotron2 的核心贡献

问题	解决方案
人工特征工程	端到端: 字符 → Mel
对齐不稳定	Location-Sensitive Attention
Mel 细节差	PostNet 残差修正
不知道何时停	Stop Token

2.9.2 遗留问题

1. **自回归慢**: 生成 1 秒语音需要 1000 步前向传播 → **FastSpeech** (Ch04)
2. **Griffin-Lim 音质差** → **WaveNet** (Ch03)
3. **需要大量 paired 数据** → **GPT-SoVITS** (Ch06)

2.9.3 参考文献

- [1] Shen et al., 2018. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions.
- [2] Tachibana et al., 2018. Efficiently Trainable Text-to-Speech System Based on Deep Convolutional Networks with Guided Attention.

2.10 习题

1. 为什么 Encoder 用 3 层 Conv1d 而不是直接用 LSTM? Conv 相比 RNN 有什么优势?
2. Location-Sensitive Attention 中, 如果对上一时刻的注意力做卷积 (kernel=31), 这个感受野覆盖了多少个字符位置?
3. 为什么训练时用 Teacher Forcing, 但推理时不用? 这种不一致会带来什么问题?
4. PostNet 的输入和输出维度都是 80, 中间层是 512。这种 “瓶颈-扩张-收缩” 结构和 AutoEncoder 有什么相似之处?
5. 如果 Stop Token 一直预测为 “不停止”, 模型会输出什么? 这在实际系统中怎么解决?

2.11 目录结构

```
ch02_tacotron/
├── README.md      # 本章教程
├── code/          # 代码
│   ├── model.py   # Tacotron2 模型 (~2600万参数)
│   ├── train.py    # 训练脚本 (含数据重采样、时长过滤)
│   ├── inference.py # 推理 + Griffin-Lim 声码器
│   └── test_tts.py # 端到端 TTS 测试 (带 RTF 统计)
├── checkpoints/   # 模型保存 (.gitignore)
└── outputs/       # 生成音频
```


3 Ch03: WaveNet — Neko 的声音变好听了

Ch02 中, Neko 学会了说话, 但声音像机器人。

罪魁祸首是 Griffin-Lim 声码器——它只能从 Mel 频谱恢复“差不多”的波形，丢失了大量细节。

这一章，我们实现 WaveNet 声码器，让 Neko 的声音变得自然。

3.1 本章导学

3.1.1 为什么学 WaveNet?

Ch02 的 Tacotron2 输出了 Mel 频谱，但 Mel 不是波形。我们需要一个**声码器（Vocoder）**把 Mel 变成波形。

Ch01 介绍了 Griffin-Lim, 但它的缺陷很明显: - 只能恢复幅度谱, 相位信息完全丢失 - 输出声音有“金属感”, 不自然 - 无法建模音频的时序依赖性

WaveNet (van den Oord et al., 2016) 是第一个直接从波形学习的声码器：- **自回归**：逐采样点生成，建模 $P(y_t | y_{<t}, \text{mel})$ - **因果卷积**：保证时序因果性 - **门控激活**：比 ReLU 更强的表达能力

3.1.2 在整条链路中的位置

Text → Tacotron2 (Ch02) → Mel Spectrogram → Vocoder → Waveform
 ↑
 本章: WaveNet

3.1.3 学习路线

节	内容	目标
3.1	为什么需要声码器	理解 Griffin-Lim 的缺陷
3.2	因果卷积	理解自回归序列建模的时序约束
3.3	扩张卷积与感受野	理解如何用小卷积核覆盖大时间范围
3.4	门控激活单元	理解 $\tanh \times \text{sigmoid}$ 的设计动机
3.5	WaveNet 完整架构	理解 Mel 条件注入和残差/跳跃连接
3.6	训练	用猫娘数据集训练
3.7	推理	Mel $\rightarrow$ 波形, 对比 Griffin-Lim
3.8	遗留问题	自回归太慢 $\rightarrow$ 引出 Ch04

3.2 3.1 为什么需要声码器?

3.2.1 3.1.1 Griffin-Lim 的缺陷

在 Ch01 中，我们实现了 Griffin-Lim 相位恢复。它的核心问题：

1. 只优化幅度谱一致性，相位靠随机初始化迭代逼近
2. 没有学习过程——不知道“语音应该听起来像什么”

3. 输出过度平滑——谐波结构模糊，高频细节丢失

听感上，Griffin-Lim 生成的语音有明显的“嗡嗡”声，像隔着玻璃说话。

3.2.2 3.1.2 WaveNet 的思路

WaveNet 不尝试恢复相位，而是**直接学习波形的条件分布**：

$$P(y|\text{mel}) = \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, \text{mel})$$

每个采样点的概率由**所有过去的采样点**和 **Mel 条件**共同决定。

这和语言模型一样：P(下一个 token | 之前所有 token)。

3.2.3 3.1.3 代价：自回归慢

WaveNet 的代价是**生成速度**。16kHz 音频每秒 16000 个采样点，每个点都需要一次完整前向传播。即使前向传播只要 1ms，1 秒音频也需要 16 秒生成。

这就是为什么 Ch04 要学 Parallel WaveNet / HiFi-GAN。

3.3 3.2 因果卷积：只看过去，不看未来

3.3.1 3.2.1 为什么需要因果性？

WaveNet 是自回归模型：生成 y_t 时，只能看到 $y_1, \dots, y_{t-1}$ 。

如果卷积“看到”了 y_{t+1} ，就是信息泄漏——训练时模型偷看了答案。

3.3.2 3.2.2 因果卷积的实现

标准 Conv1d (kernel_size=K, dilation=d) 对每个位置 t 看到：

$$[x_{t-d(K-1)/2}, \dots, x_t, \dots, x_{t+d(K-1)/2}]$$

因果卷积通过**只在左侧 padding** 实现：

左侧 padding = $(K-1) \times d$

右侧 padding = 0

标准卷积 (K=3, d=1):

$[x_{t-1}, x_t, x_{t+1}] \leftarrow$ 看到未来!

因果卷积 (K=3, d=1):

$[x_{t-2}, x_{t-1}, x_t] \leftarrow$ 只看过去 ✓

3.3.3 3.2.3 代码

```
class CausalConv1d(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=3, dilation=1):
        super().__init__()
        self.pad = (kernel_size - 1) * dilation
        self.conv = nn.Conv1d(in_ch, out_ch, kernel_size, dilation=dilation)

    def forward(self, x):
        if self.pad > 0:
            x = F.pad(x, (self.pad, 0)) # 左侧补零
        return self.conv(x)           # 输出长度 = 输入长度
```

验证方法：改变 $x[t+1:]$ 的值， $y[t]$ 不应该变化。model.py 的测试中包含了因果性验证。

3.4 3.3 扩张卷积与感受野

3.4.1 3.3.1 问题：普通卷积感受野太小

kernel_size=3 的普通卷积，每层感受野只增加 2。要看 1000 个采样点，需要 500 层！

3.4.2 3.3.2 扩张卷积的解决方案

扩张卷积 (Dilated Convolution) 在卷积核元素之间插入 “空洞”：

dilation=1: $[x_t, x_{t+1}, x_{t+2}]$	跨度 = 3
dilation=2: $[x_t, x_{t+2}, x_{t+4}]$	跨度 = 5
dilation=4: $[x_t, x_{t+4}, x_{t+8}]$	跨度 = 9
dilation=8: $[x_t, x_{t+8}, x_{t+16}]$	跨度 = 17

3.4.3 3.3.3 感受野计算

每一层 dilation=d, kernel_size=K 的因果卷积，感受野增加 $d \times (K-1)$ 。

单层感受野：

$$RF_{\text{single}} = \sum_{i=0}^{L-1} d_i \times (K-1) + 1$$

指数增长 (dilation = 1, 2, 4, 8, ...):

$$RF = (K-1) \times \sum_{i=0}^{L-1} 2^i + 1 = (K-1) \times (2^L - 1) + 1$$

多个 cycle 重复 (每个 cycle L 层):

$$RF_{\text{total}} = C \times (K - 1) \times (2^L - 1) + 1$$

默认参数: K=3, L=10, C=3

$$RF = 3 \times 2 \times (1024 - 1) + 1 = 6139 \text{ samples}$$

16kHz 下约 0.38 秒。对语音来说，这足以覆盖一个音节。

直觉: dilation 翻倍就像把“放大镜”拉远一倍，每层看到的范围加倍。

3.5 3.4 门控激活单元

3.5.1 3.4.1 为什么不用 ReLU?

ReLU 只能“截断负值”，不能动态调节信号强度。

3.5.2 3.4.2 门控激活公式

WaveNet 使用门控激活 (Gated Activation Unit):

$$z = \tanh(W_f * x + V_f * c) \odot \sigma(W_g * x + V_g * c)$$

其中: - $\tanh(\cdot)$ 是信息通道 (filter), 值域 $[-1, 1]$ - $\sigma(\cdot)$ 是门 (gate), 值域 $[0, 1]$ - x 是波形特征, c 是 mel 条件 - $\odot$ 是逐元素乘法

门决定“让多少信息通过”: gate=0 时完全阻断, gate=1 时全部通过。

3.5.3 3.4.3 和 LSTM 门控的关系

模型	门控公式	共同点
LSTM	$f \odot c + i \odot \tilde{c}$	sigmoid gate $\times$ tanh candidate
GLU	$x \odot \sigma(Wx)$	自门控
WaveNet	$\tanh(Wx + Vc) \odot \sigma(Wx + Vc)$	条件门控

3.5.4 3.4.4 代码

```
h = self.dil_conv(x) + self.mel_proj(mel_cond) # (B, 2*res, T)
h_filter, h_gate = h.chunk(2, dim=1) # 各 (B, res, T)
h = torch.tanh(h_filter) * torch.sigmoid(h_gate) # 门控激活
```

3.6 3.5 WaveNet 完整架构

3.6.1 3.5.1 整体结构

Mel Spectrogram (B, 80, T_mel)
↓ TransposedConv (上采样 hop_length 倍)
Mel Condition (B, 80, T_wav)

Waveform (mu-law 编码, B, T_wav)
↓ One-Hot + Linear Projection
Features (B, res_channels, T_wav)
↓
┌—— ResidualBlock (dilation=1) —— skip_1
├—— ResidualBlock (dilation=2) —— skip_2
├—— ResidualBlock (dilation=4) —— skip_3
├—— ...
└—— ResidualBlock (dilation=512) —— skip_10
↓ (Cycle 重复 3 次 = 30 个 block)
Sum of All Skips (B, skip_channels, T_wav)
↓ ReLU + 1×1 Conv + ReLU + 1×1 Conv
Logits (B, 256, T_wav) ← 256 类 = mu-law 量化

3.6.2 3.5.2 残差连接与跳跃连接

每个残差块产生两个输出：

1. **残差 (Residual)**：传回主路径 $x = x + \text{residual}$ ，维持信息流
2. **跳跃 (Skip)**：累加到全局 skip_sum，最终用于预测

这和 ResNet 的思路一致：- 残差连接让梯度直接流过，深层网络可训练 - 跳跃连接让输出层可以“访问”每一层的特征

3.6.3 3.5.3 Mel 条件注入

Mel 在每个残差块中通过 1×1 卷积注入：

```
h = dilated_causal_conv(x) + conv1x1(mel_condition)
```

Mel 先被 TransposedConv 上采样到波形分辨率（每个 mel 帧扩展为 hop_length 个采样点），确保波形每个时间步都有对应的 mel 条件。

3.6.4 3.5.4 mu-law 编码

波形是连续的浮点值，直接回归困难。WaveNet 用 **mu-law 压缩** 将波形量化为 256 个类别：

$$f(x) = \text{sgn}(x) \frac{\ln(1 + \mu|x|)}{\ln(1 + \mu)}, \quad \mu = 255$$

然后均匀量化到 [0, 255] 的整数。

训练目标变为 256 分类交叉熵，而不是 MSE：

$$\mathcal{L} = - \sum_t \sum_{k=0}^{255} y_{t,k} \log \hat{y}_{t,k}$$

比 MSE 更好：可以建模多模态分布（同一个 mel 可能对应多种合理的波形）。

3.7 3.6 训练

3.7.1 3.6.1 数据准备

使用猫娘数据集（Ch02 相同）：

如果还没有下载数据

```
cd data && python download_neko_1k.py --output-dir processed --num-samples 1000
```

3.7.2 3.6.2 启动训练

```
cd chapters/ch03_wavenet/code
```

基础训练（约 30 分钟 on GPU）

```
python train.py \
  --data-dir ../../../../data/processed \
  --epochs 20 \
  --batch-size 4
```

更大模型（需要更多显存）

```
python train.py \
  --data-dir ../../../../data/processed \
  --epochs 50 \
  --batch-size 4 \
  --res-channels 128 \
  --skip-channels 256 \
  --n-blocks 10 \
  --n-cycles 3
```

3.7.3 3.6.3 预期现象

- 前 5 epoch: loss 快速下降（模型学习基本分布）
- 10-20 epoch: loss 缓慢下降（细化波形建模）
- 50+ epoch: 平台期，音质缓慢提升
- 每 5 epoch 自动保存 checkpoint 到 ../checkpoints/

3.7.4 3.6.4 训练技巧

1. **Gradient Clipping**: max norm = 1.0, 防止深层网络梯度爆炸
2. **AMP**: GPU 上自动启用混合精度 (FP16), 加速 ~1.5x
3. **Segment Length**: 默认 8192 采样点 (0.5 秒), 增大可提升质量但消耗更多显存

3.8 3.7 推理: Mel → 波形

3.8.1 3.7.1 从 Ground Truth Mel 推理

```
python inference.py \
  --checkpoint ../checkpoints/wavenet_final.pt \
  --source ../../data/processed/wavs/000001.wav \
  --max-samples 8000 \
  --output-dir ../outputs
```

这会: 1. 从源音频计算 Mel 频谱 2. 用 WaveNet 自回归生成波形 3. 用 Griffin-Lim 生成基线 4. 保存两个音频供对比

3.8.2 3.7.2 与 Tacotron2 联动

```
python inference.py \
  --checkpoint ../checkpoints/wavenet_final.pt \
  --tacotron-checkpoint ../../ch02_tacotron/checkpoints/tacotron_final.pt \
  --text " 你好, 我是猫娘。" \
  --output-dir ../outputs
```

3.8.3 3.7.3 预期结果

方法	音质特点	生成速度
Griffin-Lim	金属感, 模糊, 高频伪影	快 (迭代收敛)
WaveNet (训练充分)	更自然, 清晰的高频细节	非常慢 (逐采样点)
WaveNet (训练不足)	可能是噪声	-

注意: 自回归生成非常慢! 8000 采样点 (0.5 秒) 可能需要几分钟。这是因为每一步都要过一遍完整的 30 层网络。

3.9 3.8 本章小结

3.9.1 WaveNet 的核心贡献

问题	解决方案
Griffin-Lim 相位丢失	直接建模波形分布，无需相位恢复
普通卷积感受野小	扩张卷积：指数增长的感受野
自回归时序约束	因果卷积：左侧 padding
ReLU 表达力不足	门控激活： $\tanh \times \text{sigmoid}$
深层网络难训练	残差连接 + 跳跃连接
连续波形难建模	mu-law 压缩 $\rightarrow$ 256 分类
声码器如何条件化	Mel 上采样 + 逐层 1×1 注入

3.9.2 遗留问题：自回归太慢

WaveNet 最大的问题是**推理速度**。生成 1 秒 16kHz 音频需要 16000 步前向传播，每步约 1ms，总计 16 秒。

RTF (Real-Time Factor) = 16x，完全无法实时使用。

后续解决方案：

方法	思路	加速比
Parallel WaveNet (2018)	知识蒸馏，并行生成	~1000x
WaveRNN (2019)	次采样 + 轻量 RNN	~100x
WaveGlow (2019)	Flow-based，非自回归	~50x
HiFi-GAN (2020)	GAN 训练，一次生成整段	~100x

下一章 (Ch04) 我们实现 HiFi-GAN——用 GAN 的方式一次性生成整段波形。

3.9.3 参考文献

- [1] van den Oord et al., 2016. WaveNet: A Generative Model for Raw Audio.
- [2] Mehri et al., 2017. SampleRNN: An Unconditional End-to-End Neural Audio Generation Model (mu-law encoding).
- [3] Shen et al., 2018. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions (Tacotron2 + WaveNet).
- [4] van den Oord et al., 2018. Parallel WaveNet: Fast High-Fidelity Speech Synthesis.

3.10 习题

1. **因果卷积验证**：如果去掉因果 padding (用对称 padding)，训练 loss 会怎样变化？为什么这反而可能导致训练 loss 更低但推理效果更差？
2. **感受野计算**：
 - 当前默认配置 (K=3, L=10, C=3)，感受野是多少？
 - 如果把 dilation 序列改为 [1, 2, 4, 8, 16, 32] (L=6)，感受野是多少？
 - 感受野小于 hop_length (256) 会有什么问题？

3. **扩张卷积的效率**: dilation=512, kernel_size=3 的因果卷积, 每个输出位置实际需要读取多少个输入值? 这和普通 kernel_size=1025 的卷积有什么区别?
 4. **门控激活 vs ReLU**:
 - 画出 $\tanh(x) \times \text{sigmoid}(x)$ 的函数图像
 - 当 sigmoid 输出接近 0 时, 梯度会怎样? 这是否会导致 “门控死区”?
 5. **自回归速度**:
 - WaveNet 生成 1 秒 16kHz 音频需要多少步前向传播?
 - WaveRNN 通过 4x 次采样减少了多少步?
 - HiFi-GAN 为什么可以一次性生成整段? (提示: GAN 的生成器是非自回归的)
 6. **mu-law 编码**:
 - mu-law 编码对大振幅和小振幅的量化精度有何不同? 为什么这对语音有利?
 - 如果改用线性量化 256 级, $[-1, 1]$ 的步长是多少? 小振幅信号会怎样?
-

3.11 目录结构

```
ch03_wavenet/
├── README.md          # 本章教程
├── code/
│   ├── model.py       # WaveNet 架构 + mu-law + 形状验证 (~260 行)
│   ├── train.py       # 训练脚本 (~190 行)
│   └── inference.py    # 推理脚本 + Griffin-Lim 对比 (~170 行)
├── checkpoints/       # 模型权重 (.gitignore)
└── outputs/           # 生成的音频
```

4 Ch04: FastSpeech2 – Neko 学会了快速说话

Ch02 的 Tacotron2 能用，但太慢了。

这一章，我们用**并行生成**取代自回归，让 TTS 快 10-100 倍。

4.1 本章导学

4.1.1 为什么需要 FastSpeech2?

Ch02 的 Tacotron2 是**自回归**模型：生成 Mel 频谱时，必须一帧一帧地预测。

Tacotron2 推理：

第1帧 -> 第2帧 -> 第3帧 -> ... -> 第N帧

每帧需要一次 LSTM 前向传播

1秒语音 $\approx$ 87帧 (hop=256, sr=22050)

→ 1秒语音需要 87 次串行计算

这带来两个问题：

1. **推理慢**：生成 3 秒语音需要 200+ 次前向传播，RTF (实时率) > 1 ，不能实时
2. **鲁棒性差**：自回归的误差会累积——某一帧预测偏了，后面全跟着偏（跳字、重复）

FastSpeech2 的核心思想：**不要一帧一帧生成，而是一次性生成所有帧。**

4.1.2 学习路线

节	内容	目标
4.1	架构总览	理解 Encoder -> Length Regulator -> Decoder
4.2	Duration Predictor	时长建模：每个音素持续多少帧
4.3	Length Regulator	根据时长拉伸序列
4.4	Pitch & Energy	方差适配器：控制韵律
4.5	FFT Blocks	前馈 Transformer 模块
4.6	训练	时长标签 + 多任务损失
4.7	推理	并行生成 + 速度对比

4.2 4.1 架构总览

Text (字符序列)

|
Embedding + Positional Encoding

|
FFT Encoder (Self-Attention + FFN, x4)
| (B, T_text, d_model)

+-- Duration Predictor -> durations (每音素帧数)

+-- Pitch Predictor -> pitch (音高)

+-- Energy Predictor -> energy (响度)

```

|
Length Regulator (按 durations 拉伸)
|   (B, T_mel, d_model)
|
FFT Decoder (Self-Attention + FFN, x4)
|
Linear -> Mel Spectrogram (B, 80, T_mel)
|
Vocoder (HiFi-GAN / Griffin-Lim) -> Waveform

```

4.2.1 与 Tacotron2 的关键区别

	Tacotron2 (Ch02)	FastSpeech2 (本章)
解码方式	自回归 (逐帧)	并行 (一次性)
对齐机制	Attention (隐式)	Duration Predictor (显式)
推理速度	慢 (RTF > 1)	快 (RTF < 0.01)
鲁棒性	可能跳字/重复	不会 (确定性映射)
可控性	难以控制语速/音高	可直接调节

4.3 4.2 Duration Predictor: 每个音素说多久?

4.3.1 4.2.1 为什么需要时长信息?

Tacotron2 用 Attention 自动学习 “文本 -> 语音” 的对齐。但 Attention 有两个问题：- 训练不稳定（需要大量数据才能学到好的对角线注意力）- 推理时可能出错（跳字、重复）

FastSpeech2 的方案：**显式预测每个音素的持续帧数**。

输入: "你好世界" (4 个字符)

Duration Predictor 预测: [12, 8, 15, 10] (帧数)

总和 = 45 帧 = Mel 频谱长度

4.3.2 4.2.2 预测器结构

```

# 2 层 Conv1d + LayerNorm + ReLU
Encoder Output (B, T_text, d)
-> Conv1d -> LN -> ReLU -> Dropout
-> Conv1d -> LN -> ReLU -> Dropout
-> Linear -> (B, T_text) # 每个音素一个标量

```

输出在 **log 域**: 因为时长分布高度偏斜 (有的音素 1 帧, 有的 50 帧), 取 log 后更接近正态分布, MSE 损失更好优化。

4.3.3 4.2.3 时长标签从哪来?

这是 FastSpeech2 的核心难题。三种常见方案:

1. **Teacher Model 对齐**: 先训练一个 Tacotron2, 提取其 Attention 矩阵, 每列的 argmax 就是时长。这是原论文的方法。
2. **Montreal Forced Aligner (MFA)**: 独立的语音对齐工具, 基于 HMM, 不需要训练 TTS 模型。
3. **均匀分配** (本章简化方案): 假设每个字符持续相同帧数 $\text{mel_len} / \text{text_len}$ 。

均匀分配 (本教程使用)

```
def estimate_uniform_durations(text_len, mel_len):
    base = mel_len // text_len
    rem = mel_len - base * text_len
    durs = [base] * text_len
    for i in range(rem):
        durs[i] += 1
    return durs
```

Neko 笔记: 均匀分配是简化方案, 会导致语速不自然。生产中应使用 MFA 或 Tacotron2 对齐。但模型结构是完全一样的, 只是训练数据质量不同。

4.4 4.3 Length Regulator: 把文本拉伸到语音长度

4.4.1 4.3.1 问题

Encoder 输出 T_{text} 个向量, 但 Mel 频谱有 T_{mel} 帧。需要把 T_{text} 扩展到 T_{mel} 。

4.4.2 4.3.2 方案: 按持续时间重复

```
# 核心操作: repeat_interleave
encoder_out = [h1, h2, h3]          # 3 个音素
durations   = [2, 3, 1]             # 每个音素的帧数
expanded    = [h1, h1, h2, h2, h2, h3] # 6 帧
```

就是这么简单! 每个音素的向量重复其预测的时长次。

4.4.3 4.3.3 为什么有效?

一个音素在时间上持续多帧, 但其“语义内容”不变。重复同一个向量相当于说: **这几帧都属于同一个音素, 内容相同。**

Decoder 再通过 Self-Attention 看到“相邻帧来自不同音素”, 自动学到过渡和韵律。

4.5 4.4 Pitch & Energy: 控制韵律

4.5.1 4.4.1 为什么需要?

Duration 只决定了“每个音素说多久”, 但没说“以什么音调说”、“以多大声音说”。

- **Pitch (音高)**: 疑问句的末尾音高上扬, 陈述句平稳

- **Energy (能量/响度)**: 强调某个词时响度变大

4.5.2 4.4.2 Variance Adapter

每个都有一个 Predictor (结构和 Duration Predictor 相同) + 一个 Embedding 层:

```
# Pitch Predictor: encoder_out -> (B, T_text) pitch values
# Pitch Embedding: pitch -> (B, T_text, d_model)
# Add to encoder output
x = x + pitch_embed(pitch.unsqueeze(-1))
x = x + energy_embed(energy.unsqueeze(-1))
```

4.5.3 4.4.3 推理时的控制

因为 pitch 和 energy 是独立预测的, 我们可以在推理时直接缩放:

```
# 高音版
mel = model.inference(text, pitch_scale=1.2)
# 轻声版
mel = model.inference(text, energy_scale=0.8)
```

Neko 笔记: 本教程使用 mel 频谱的统计量 (频谱质心、均值幅度) 作为 pitch/energy 的代理训练目标。生产系统会用 pyworld 提取真实 F0 (基频)。

4.6 4.5 FFT Block: 前馈 Transformer

4.6.1 4.5.1 结构

每个 FFT Block 就是一个标准的 Pre-LN Transformer 层:

```
Input (B, T, d)
|
+-- LayerNorm -> Multi-Head Self-Attention -> Residual
|
+-- LayerNorm -> FFN (Linear->ReLU->Linear) -> Residual
|
Output (B, T, d)
```

Encoder 和 Decoder 都是 4 层 FFT Block 的堆叠。

4.6.2 4.5.2 Encoder vs Decoder

- **Encoder**: 处理文本序列, Self-Attention 让每个字符看到全文上下文
- **Decoder**: 处理展开后的帧序列, Self-Attention 让每帧看到前后帧的过渡

两者结构完全相同, 只是输入不同。

4.6.3 4.5.3 为什么不用 LSTM?

Tacotron2 用 LSTM, 因为当时 (2018) Transformer 还没在 TTS 中广泛使用。

FFT 的优势: - **并行计算**: Self-Attention 不需要逐步处理序列 - **长程依赖**: 任意两个位置都直接交互 - **训练稳定**: 不存在 RNN 的梯度消失/爆炸

4.7 4.6 训练

4.7.1 4.6.1 多任务损失

FastSpeech2 同时优化 4 个目标:

$$\mathcal{L} = \mathcal{L}_{mel} + \mathcal{L}_{dur} + \mathcal{L}_{pitch} + \mathcal{L}_{energy}$$

- **Mel Loss**: 预测 Mel 与 GT 的 MSE (核心目标)
- **Duration Loss**: 预测 log-duration 与 GT 的 MSE (辅助目标)
- **Pitch Loss**: 预测 pitch 与 GT 的 MSE
- **Energy Loss**: 预测 energy 与 GT 的 MSE

4.7.2 4.6.2 数据准备

使用和 Ch02 相同的数据集:

```
# 下载猫娘数据集
cd data
python download_neko_1k.py --output-dir processed --num-samples 1000
```

4.7.3 4.6.3 启动训练

```
cd chapters/ch04_fastspeech/code
python train.py \
  --data-dir ../../../../data/processed \
  --epochs 50 \
  --batch-size 8
```

预期现象: - 前 5 epoch loss 快速下降 (duration/pitch/energy 开始学习) - 10-20 epoch mel loss 进入平台期 - 多任务损失中 dur_loss 下降最快 (均匀时长容易学) - 每 10 epoch 自动保存 checkpoint

4.7.4 4.6.4 与 Ch02 训练对比

	Tacotron2 (Ch02)	FastSpeech2 (本章)
每 batch 计算量	大 (T_mel 次 LSTM 步进)	小 (全并行)
训练速度	慢	快 2-3x
需要的额外数据	无	时长标签 (可自动估计)
收敛难度	中 (Attention 需要对齐)	低 (确定性映射)

4.8 4.7 推理

4.8.1 4.7.1 并行推理

```
python inference.py \  
  --checkpoint ../checkpoints/fs2_final.pt \  
  --text " 你好，我是猫娘。" \  
  --output ../outputs/fs2_output.wav
```

FastSpeech2 推理只需 **1 次前向传播** 就生成全部 Mel 帧：

```
# 全部 mel 帧一次性生成  
mel = model.inference(text) # (B, 80, T_mel) 一次调用
```

4.8.2 4.7.2 速度对比

假设生成 100 帧 Mel (约 1.2 秒语音)：

模型	前向传播次数	典型耗时 (GPU)	RTF
Tacotron2	100 (串行)	~500ms	~0.4
FastSpeech2	1 (并行)	~10ms	~0.008
加速比	100x	50x	50x

RTF (Real-Time Factor) = 生成时间 / 音频时长。RTF < 1 表示可以实时。

4.8.3 4.7.3 可控生成

```
# 正常  
mel = model.inference(text)  
# 高音 (pitch_scale > 1)  
mel_high = model.inference(text, pitch_scale=1.3)  
# 慢速 (duration 自动变长)  
# 注：需要修改 durations 的缩放因子
```

4.8.4 4.7.4 预期效果

- 早期训练 (< 10 epoch)：输出可能是平坦的噪声
- 中期 (10-30 epoch)：开始出现语调轮廓
- 充分训练后：能生成可辨识的语音（配合好的声码器）

声码器仍然重要：FastSpeech2 只生成 Mel 频谱，还需要声码器转为波形。本章用 Griffin-Lim (音质差)，Ch05 VITS 会学到端到端方案。

4.9 4.8 本章小结

4.9.1 FastSpeech2 的核心贡献

问题	解决方案
自回归慢	并行生成（1 次前向传播）
Attention 不稳定	显式 Duration Predictor
误差累积	确定性映射，无累积
无法控制韵律	Pitch/Energy Variance Adapters

4.9.2 遗留问题

1. **仍需独立声码器**: FastSpeech2 只输出 Mel, 还需要 HiFi-GAN 等声码器 -> Ch05 VITS 端到端
2. **时长标签依赖**: 均匀估计不够好, 需要 MFA 或 teacher model
3. **音质上限**: 非自回归模型的 mel 预测精度略低于 Tacotron2 (但推理快得多)

4.9.3 参考文献

- [1] Ren et al., 2021. FastSpeech 2: Fast and High-Quality End-to-End Text to Speech. ICLR 2021.
- [2] Ren et al., 2019. FastSpeech: Fast, Robust and Controllable Text to Speech. NeurIPS 2019.
- [3] Vaswani et al., 2017. Attention Is All You Need. NeurIPS 2017.

4.10 习题

1. **时长建模**: 为什么 Duration Predictor 在 log 域预测, 而不是直接预测帧数? 提示: 想想时长值的分布特点。
2. **Length Regulator**: 如果 Duration Predictor 预测的所有时长之和与实际 Mel 长度不匹配, 会发生什么? 如何缓解?
3. **并行 vs 自回归**: Tacotron2 的 Decoder 每步需要上一帧的输出, 而 FastSpeech2 的 Decoder 不需要。这意味着 FastSpeech2 的 Decoder 无法建模什么信息? 这对音质有什么影响?
4. **Variance Adapter**: 如果去掉 Pitch Predictor 和 Energy Predictor, 只保留 Duration Predictor, 模型还能正常训练吗? 音质会受到什么影响?
5. **进阶思考**: VITS (Ch05) 把 FastSpeech2 的结构和 VAE + Normalizing Flow 结合, 实现了端到端 TTS。它解决了 FastSpeech2 的什么问题? 又引入了什么新的复杂度?

4.11 目录结构

```
ch04_fastspeech/  
├── README.md      # 本章教程（你正在读的）  
└── code/
```



```
|   |—— model.py      # FastSpeech2 模型 (~280 行)
|   |—— train.py     # 训练脚本 (含均匀时长估计)
|   |—— inference.py  # 并行推理 + Griffin-Lim
|—— checkpoints/      # 模型权重
|—— outputs/          # 生成的音频
```

5 Ch05: VITS — 端到端语音合成

前面几章我们学了“文本 → Mel → 波形”的两阶段方法。

这一章，Neko 要学一个更优雅的方案：**直接从文本生成波形**。

5.1 本章导学

5.1.1 为什么需要 VITS?

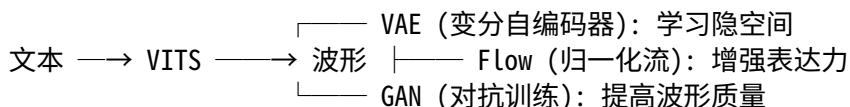
Ch02 的 Tacotron2 实现了“文本 → Mel Spectrogram”，但要得到波形，还需要一个独立的声码器（如 Griffin-Lim 或 WaveNet）。这条两阶段路线有几个问题：

问题	影响
级联误差	Mel 预测的小误差会在声码器中被放大
训练割裂	声码器和声学模型分开训练，无法联合优化
Mel 信息损失	Mel 频谱丢弃了相位信息，声码器只能猜测

VITS (Kim et al., 2021) 的解决方案：**端到端** — 文本直接出波形，无需 Mel 中间表示。

5.1.2 VITS 的核心创新

VITS 把三个强大的工具组合在一起：



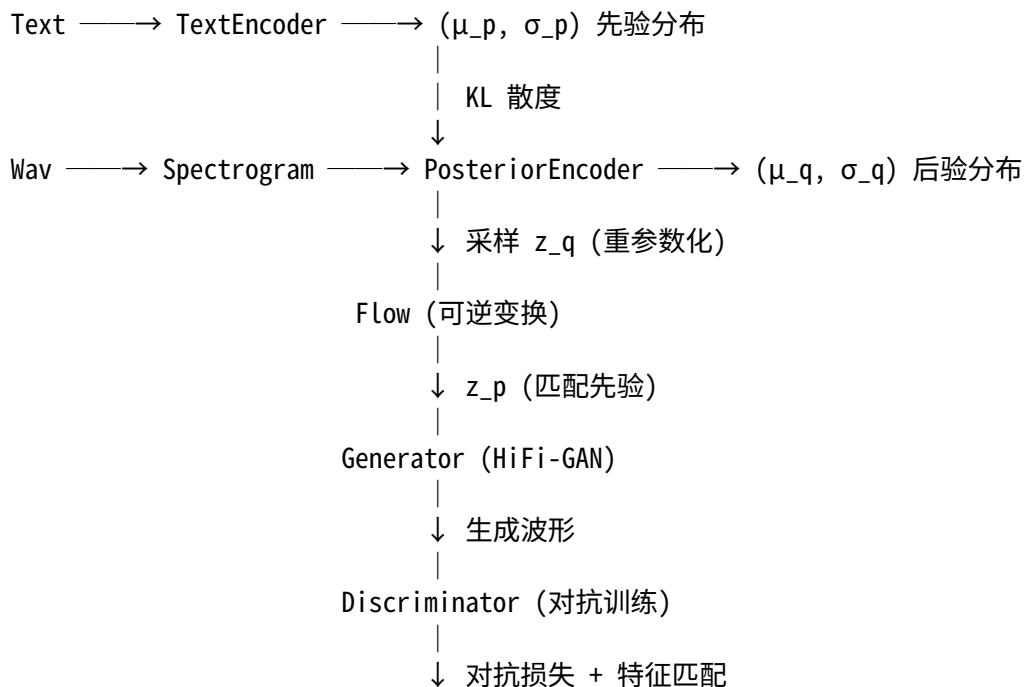
1. **VAE**: 编码器把文本/波形编码到隐空间，解码器从隐空间生成波形
2. **Flow**: 可逆变换，让简单的高斯先验能表达复杂的后验分布
3. **GAN**: 判别器迫使生成器产出更逼真的波形

5.1.3 学习路线

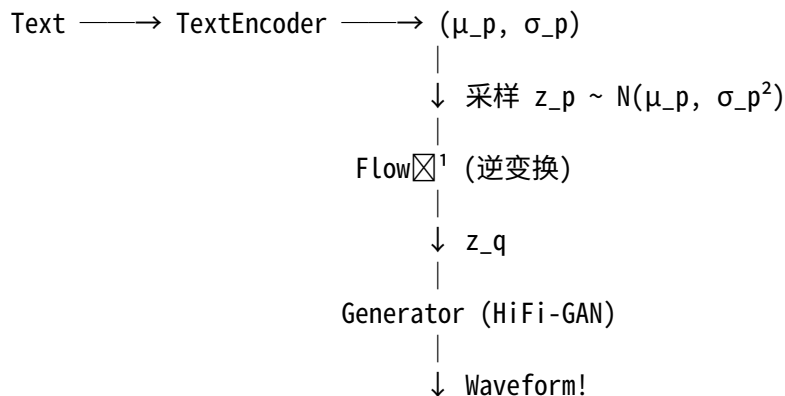
节	内容	目标
5.1	VITS 架构总览	理解训练和推理的数据流
5.2	VAE 原理	理解变分推断和 ELBO
5.3	归一化流	理解可逆变换如何提高表达力
5.4	GAN 对抗训练	理解判别器如何提升质量
5.5	核心模块	TextEncoder, PosteriorEncoder, Flow, Generator
5.6	Monotonic Alignment Search	理解无监督对齐
5.7	训练	多损失函数的平衡
5.8	推理	端到端文本到波形
5.9	端到端 vs 两阶段	定量对比

5.2 5.1 VITS 架构总览

5.2.1 训练时的数据流



5.2.2 推理时的数据流



关键区别：**推理时不需要 PosteriorEncoder 和 Flow 的前向变换**。Flow 的逆变换用于把先验采样映射到解码器能理解的隐空间。

5.3 5.2 VAE：变分自编码器

5.3.1 5.2.1 直觉

VAE 的核心思想：**学习一个压缩的隐空间，能捕捉语音的本质特征。**

想象把一段语音压缩成几个数字（隐变量 z ），再从这几个数字还原出整段语音。如果压缩得好， z 就包含了“说了什么”的所有信息。

编码器: $\text{Wav} \rightarrow z$ (压缩)

解码器: $z \rightarrow \text{Wav}$ (解压)

5.3.2 5.2.2 变分推断

普通 AutoEncoder 的问题是：隐空间 z 可以是任何形状，难以采样。

VAE 的解决：**强制 z 服从高斯分布。**

编码器不再输出单个 z ，而是输出分布参数：

$$q_\phi(z|x) = \mathcal{N}(z; \mu_q, \sigma_q^2)$$

- μ_q : 后验均值 (波形的“中心”表示)
- σ_q : 后验标准差 (表示的不确定性)

5.3.3 5.2.3 重参数化技巧

问题：从分布中采样是随机操作，无法反向传播。

解决：**重参数化**

$$z = \mu_q + \sigma_q \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

把随机性转移到 ϵ 上， μ_q 和 σ_q 是可微的。

5.3.4 5.2.4 KL 散度

VAE 要求后验分布 $q(z|x)$ 接近先验分布 $p(z)$ ：

$$\text{KL}(q\|p) = \mathbb{E}_q \left[\log \frac{q(z|x)}{p(z)} \right]$$

对两个对角高斯分布，KL 有解析解：

$$\text{KL} = \sum_i \left[\log \sigma_{p,i} - \log \sigma_{q,i} + \frac{\sigma_{q,i}^2 + (\mu_{q,i} - \mu_{p,i})^2}{2\sigma_{p,i}^2} - \frac{1}{2} \right]$$

直觉：KL 惩罚后验偏离先验。如果先验是 $\mathcal{N}(0, I)$ ，KL 鼓励后验的均值接近 0、方差接近 1。

5.3.5 5.2.5 ELBO

VAE 的目标函数是最大化 ELBO (Evidence Lower Bound)：

$$\text{ELBO} = \mathbb{E}_q[\log p(x|z)] - \text{KL}(q(z|x)\|p(z))$$

- 第一项：**重建损失** — 从 z 还原的 x 应该接近原始 x

- 第二项：**KL 正则** — z 的分布应该接近先验

Neko 笔记：ELBO 是一个下界——真实的对数似然 $\geq$ ELBO。最大化 ELBO 就是在逼近真实似然。

5.4 5.3 归一化流 (Normalizing Flow)

5.4.1 5.3.1 问题

VAE 假设先验 $p(z)$ 是简单高斯。但真实语音的隐空间可能非常复杂——多峰、弯曲、不规则。简单高斯无法表达这种复杂性 $\rightarrow$ 生成质量受限。

5.4.2 5.3.2 解决思路

归一化流：**通过一系列可逆变换，把简单分布变成复杂分布。**

简单高斯 z_0

$\downarrow f_1$ (可逆)

z_1

$\downarrow f_2$ (可逆)

z_2

$\downarrow \dots$

$\downarrow f_K$ (可逆)

z_K (复杂分布)

每个 f_i 都是可逆的，所以：- 正向 (训练)： $z_0 \rightarrow z_K$ ，把后验变换到先验空间 - 逆向 (推理)： $z_K \rightarrow z_0$ ，从先验采样得到解码器输入

5.4.3 5.3.3 仿射耦合层

VITS 使用 **仿射耦合层** (Affine Coupling Layer)：

输入 $x = [x_1, x_2]$ (按通道分成两半)

x_1 保持不变

$z_2 = (x_2 - t(x_1)) \times \exp(-s(x_1)) \quad \leftarrow$ 由 x_1 决定如何变换 x_2

输出 $z = [x_1, z_2]$

其中 $s(x_1)$ 和 $t(x_1)$ 是神经网络输出的 scale 和 shift。

为什么这很巧妙？

1. **可逆**：逆变换只需 $x_2 = z_2 \times \exp(s(x_1)) + t(x_1)$
2. **雅可比行列式高效**：三角矩阵 $\rightarrow$ 行列式 = 对角元素之积
3. **交替反转**：堆叠多层时交替反转通道，让所有维度都被变换

5.4.4 5.3.4 对数行列式

Flow 变换会改变概率密度。为了正确计算 KL 散度，需要修正：

$$\log p(z_K) = \log p(z_0) + \sum_{i=1}^K \log \left| \det \frac{\partial f_i}{\partial z_{i-1}} \right|$$

仿射耦合层的行列式： $\log |\det J| = \sum s(x_1)$

5.4.5 5.3.5 VITS 中 Flow 的作用

训练：PosteriorEncoder $\rightarrow$ z_q $\rightarrow$ Flow $\rightarrow$ z_p (应该匹配先验)

推理：先验采样 $z_p \rightarrow$ Flow⁻¹ $\rightarrow$ $z_q \rightarrow$ Generator $\rightarrow$ 波形

Flow 让简单的高斯先验能够表达复杂的后验分布，提升生成质量。

5.5 5.4 GAN 对抗训练

5.5.1 5.4.1 为什么需要 GAN?

VAE 的重建损失 (L1/L2) 倾向于产出平均化的结果：- 波形在高频段被平滑 - 听起来“闷”、缺乏清晰度

GAN 的判别器提供另一种信号：不比较像素级的差异，而是判断“听起来像不像真的”。

5.5.2 5.4.2 多周期判别器 (MPD)

VITS 使用 HiFi-GAN 的多周期判别器：

波形 (1D) $\longrightarrow$ 重塑为 2D (period $\times$ subsequence) $\longrightarrow$ Conv2d $\longrightarrow$ 真/假

使用质数周期 [2, 3, 5, 7, 11]：- period=2：捕获半周期模式 - period=3：捕获三周期模式 - 不同周期看到不同尺度的结构

为什么用质数？避免周期性重复，让每个判别器关注不同的频率范围。

5.5.3 5.4.3 对抗损失

使用 Least Squares GAN：

判别器损失：

$$L_D = \mathbb{E}[(D(x) - 1)^2] + \mathbb{E}[D(G(z))^2]$$

生成器损失：

$$L_G = \mathbb{E}[(D(G(z)) - 1)^2]$$

5.5.4 5.4.4 特征匹配

仅靠对抗损失训练不稳定。VITS 还加入**特征匹配损失**：

$$L_{feat} = \sum_l \|f_l^{real} - f_l^{fake}\|_1$$

要求生成样本在判别器中间层的特征接近真实样本。这比纯对抗损失更稳定。

5.6 5.5 核心模块详解

5.6.1 5.5.1 TextEncoder

```
class TextEncoder:
    """ 文本 → 先验分布参数 ( $\mu_p$ ,  $\log \sigma_p$ ) """

    架构:
    Token Embedding
    ↓
    相对位置编码 + N × Transformer Encoder Block
    ↓
    线性投影 → ( $\mu_p$ ,  $\log \sigma_p$ )
```

设计选择：- **Transformer 而非 BiLSTM**：并行计算，训练更快 - **相对位置编码**：对不同长度文本泛化更好 - **输出高斯参数**：为 VAE 提供先验分布

5.6.2 5.5.2 PosteriorEncoder

```
class PosteriorEncoder:
    """ 线性频谱 → 后验分布参数 ( $\mu_q$ ,  $\log \sigma_q$ ) + 采样  $z_q$  """

    架构:
    Linear Spectrogram (B, 513, T_spec)
    ↓
    Conv1d 投影
    ↓
    N × WaveNet 门控卷积块 (膨胀卷积)
    ↓
    Conv1d → ( $\mu_q$ ,  $\log \sigma_q$ )
    ↓
    重参数化采样 →  $z_q$ 
```

关键点：- **仅在训练时使用**，推理时不需要 - **输入线性频谱**（非 Mel），保留更多频率细节 - **膨胀卷积**：dilation 按 2^k 递增，感受野指数增长 - **跳跃连接**：汇总所有层输出，避免信息丢失

5.6.3 5.5.3 Flow

```
class Flow:
    """ 归一化流：可逆变换  $z_q \leftrightarrow z_p$  """
```

架构：

$K \times (\text{AffineCouplingLayer} + \text{Flip})$

每个 AffineCouplingLayer: 1. 按通道分成两半 ($x_{\text{left}}, x_{\text{right}}$) 2. 神经网络预测 scale s 和 shift t 3. $z_{\text{right}} = (x_{\text{right}} - t) \times \exp(-s)$ 4. 最后一层初始化为 0 $\rightarrow$ 初始变换接近恒等映射

5.6.4 5.5.4 Generator (HiFi-GAN)

```
class Generator:
    """  $z \rightarrow$  波形 (HiFi-GAN Decoder) """

    架构:
    z (B, 192, T_z)
      ↓ Conv1d 预处理
      ↓  $4 \times [\text{ConvTranspose1d}(\text{上采样}) + \text{ResBlock}(\text{MRF})]$ 
      ↓ Conv1d  $\rightarrow \tanh$ 
    waveform (B, 1, T_wav)
```

上采样倍率: $[8, 8, 2, 2] \rightarrow$ 总倍率 256

每 1 帧隐变量 $\rightarrow$ 256 个音频采样点 ($16\text{kHz} / 256 = 62.5 \text{ Hz}$ 帧率)

5.6.5 5.5.5 Discriminator (MPD)

```
class Discriminator:
    """ 多周期判别器 """

     $5 \times \text{PeriodDiscriminator}(\text{period} \in [2, 3, 5, 7, 11])$ 
```

5.6.6 5.5.6 DurationPredictor

```
class DurationPredictor:
    """ 预测每个文本 token 的帧数 """

     $3 \times \text{Conv1d} + \text{LayerNorm} + \text{ReLU} \rightarrow \text{投影} \rightarrow \log(\text{duration})$ 
```

推理时用于把文本编码序列展开到音频时间轴。

5.7 5.6 Monotonic Alignment Search (MAS)

5.7.1 5.6.1 问题

训练时，文本有 T_{text} 个 token，频谱有 T_{spec} 帧。我们不知道哪个 token 对应哪些帧。

5.7.2 5.6.2 解决

MAS 是一种动态规划算法，找到**使总概率最大的单调对齐**：

文本： [你] [好] [猫] [娘]
 ↓ ↓ ↓ ↓
频谱： [1 2 3 4 5 6 7 8 9 10]

最优对齐：[你]→[1,2,3]，[好]→[4,5]，[猫]→[6,7,8]，[娘]→[9,10]

约束：1. **单调性**：对齐不能倒退 2. **完整性**：每个 token 至少对应 1 帧 3. **最优性**：总 log 似然最大

5.7.3 5.6.3 动态规划

$dp[i][j]$ = 对齐到文本第 i 个 token、频谱第 j 帧的最大概率

转移：

$dp[i][j] = \max(dp[i-1][j-1], dp[i][j-1]) + \log_p[i][j]$
 (消耗一个 token) (不消耗)

回溯路径得到硬对齐矩阵（每列只有一个 1）

从对齐矩阵可以提取每个 token 的时长： $duration[i] = \text{sum}(\text{attn}[i, :])$

5.8 5.7 训练

5.8.1 5.7.1 多损失函数

VITS 的总损失是多个损失的加权和：

$L_{\text{gen}} = c_{\text{mel}} \times L_{\text{mel}} + c_{\text{kl}} \times L_{\text{KL}} + c_{\text{dur}} \times L_{\text{dur}} + c_{\text{adv}} \times L_{\text{adv_G}} + c_{\text{feat}} \times L_{\text{feat}}$
 $L_{\text{disc}} = L_{\text{adv_D}}$

损失	含义	默认权重
L_{mel}	Mel 频谱重建 (L1)	45
L_{KL}	KL 散度 (后验 → 先验)	1
L_{dur}	时长预测 MSE	1
$L_{\text{adv_G}}$	生成器对抗	1
L_{feat}	特征匹配	2

5.8.2 5.7.2 训练流程

每个 batch:

1. Generator forward:
 - TextEncoder → 先验参数
 - PosteriorEncoder → 后验参数 + z_q
 - Flow(z_q) → z_p
 - MAS → 对齐 + 时长
 - Generator(z_q) → 波形
2. Discriminator:
 - 判断真假波形
3. Generator backward:
 - $L_{gen} = L_{mel} + L_{KL} + L_{dur} + L_{adv_G} + L_{feat}$
4. Discriminator backward:
 - $L_{disc} = L_{adv_D}$

5.8.3 5.7.3 启动训练

```
cd chapters/ch05_vits/code

# 下载数据 (如果还没有)
cd ../../../../data
python download_neko_1k.py --output-dir processed --num-samples 1000

# 训练
cd ../chapters/ch05_vits/code
python train.py \
    --data-dir ../../../../data/processed \
    --epochs 50 \
    --batch-size 2 \
    --max-spec-len 400 \
    --max-wav-sec 10
```

显存需求: batch_size=2 + max_spec_len=400 约需 12GB 显存。如果显存不足, 减小 --max-spec-len 和 --batch-size。

预期训练曲线: - Epoch 1-5: loss_g 从 ~500 快速下降到 ~200 - Epoch 5-20: loss_mel 稳步下降, KL 趋于稳定 - Epoch 20+: 对抗损失开始起作用, 波形质量提升

5.9 5.8 推理

5.9.1 5.8.1 端到端推理

```

# 使用训练好的模型
python inference.py \
    --checkpoint ../checkpoints/vits_final.pt \
    --text " 你好，我是猫娘。" \
    --output ../outputs/vits_neko.wav

# 调整语速 (1.2 = 慢 20%)
python inference.py \
    --checkpoint ../checkpoints/vits_final.pt \
    --text " 你好，我是猫娘。" \
    --length-scale 1.2 \
    --output ../outputs/vits_slow.wav

# 快速测试 (随机模型，输出噪声)
python inference.py \
    --text " 测试" \
    --output ../outputs/vits_test.wav

```

5.9.2 5.8.2 推理参数

- noise_scale: 采样噪声缩放。越小 → 越确定性（但缺乏变化）。推荐 0.667。
- length_scale: 语速控制。1.0 = 正常，>1.0 = 慢，<1.0 = 快。

5.9.3 5.8.3 推理速度

VITS 是非自回归的（不像 Tacotron2 逐帧生成），所以推理很快：

模型	生成方式	典型 RTF
Tacotron2 + WaveNet	自回归	0.01-0.1x
FastSpeech2 + HiFi-GAN	非自回归（两阶段）	0.001-0.01x
VITS	非自回归（端到端）	0.001-0.01x

5.10 5.9 端到端 vs 两阶段对比

5.10.1 5.9.1 定量对比

特性	Tacotron2 (两阶段)	VITS (端到端)
输出	Mel Spectrogram	Waveform
声码器	需要独立声码器	内置 HiFi-GAN
训练方式	分阶段训练	联合训练
推理步骤	Text→Mel + Mel→Wav	Text→Wav
自回归	是（慢）	否（快）
信息损失	Mel 丢弃相位	无

特性	Tacotron2 (两阶段)	VITS (端到端)
训练复杂度	低	高

5.10.2 5.9.2 质量对比

端到端方法的优势：1. **无级联误差**：不会在 Mel→Wav 阶段引入额外失真 2. **联合优化**：所有组件共同优化同一目标 3. **相位恢复**：Generator 直接输出波形，无需猜测相位

5.10.3 5.9.3 代价

端到端的代价：1. **训练更复杂**：需要平衡多个损失函数 2. **显存需求大**：同时加载 Generator + Discriminator 3. **调试困难**：出问题时难以定位是哪个组件的问题

5.11 5.10 本章小结

5.11.1 VITS 的核心贡献

问题	VITS 的解决方案
两阶段级联误差	端到端：Text → Waveform
隐空间过于简单	归一化流增强表达力
VAE 波形质量差	GAN 对抗训练提高质量
不知道对齐	MAS 无监督对齐
说话速度固定	SDP 随机时长预测

5.11.2 遗留问题

1. **音色固定**：VITS 只能合成训练时的音色，无法克隆 → GPT-SoVITS (Ch06)
2. **需要大量数据**：单说话人 VITS 需要数小时数据 → Few-shot TTS (Ch07)
3. **可控性有限**：难以精确控制音调、情感 → Controllable TTS (Ch08)

5.11.3 参考文献

- [1] Kim et al., 2021. Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech.
- [2] Dinh et al., 2017. Density estimation using Real-NVP.
- [3] Kong et al., 2020. HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis.
- [4] Kingma & Welling, 2014. Auto-Encoding Variational Bayes.

5.12 习题

1. **VAE vs AE**：普通 AutoEncoder 和 VAE 的编码器有什么区别？为什么 VAE 可以采样新数据而 AE 不行？

2. **Flow 的可逆性**：仿射耦合层为什么是可逆的？如果去掉 scale（只用 shift），它还是可逆的吗？行列式是什么？
 3. **MAS 的必要性**：如果没有 MAS，直接用 DurationPredictor 的预测时长来训练，会出什么问题？
 4. **GAN 训练稳定性**：如果判别器太强（loss_adv_d 趋近 0），生成器会怎样？实践中如何解决？
 5. **端到端的代价**：VITS 相比 Tacotron2，训练时需要多加载哪些模块？显存占用大约是多少？
 6. **对比实验**：用同样的文本分别用 Ch02 (Tacotron2) 和 Ch05 (VITS) 合成，对比音质和速度。
-

5.13 目录结构

```
ch05_vits/
├── README.md          # 本章教程（你在这里）
├── code/
│   ├── modules.py     # 子模块（TextEncoder, PosteriorEncoder, Flow, Generator, Discriminator）
│   ├── model.py       # VITS 主架构 + 损失函数 + MAS
│   ├── train.py       # 训练脚本（多损失函数，G/D 交替训练）
│   └── inference.py    # 端到端推理（Text → Waveform）
├── checkpoints/       # 模型保存
└── outputs/           # 生成音频
```

6 Ch06: Neural Audio Codec — Neko 的音频压缩术

在让 Neko 用 Token 说话之前，她需要先学会“压缩”声音。

这一章，我们实现简化的 EnCodec，理解 RVQ-VAE 如何把波形变成离散 Token。

6.1 本章导学

6.1.1 为什么需要 Neural Codec?

在前面的章节中，我们已经见过两种音频表示：

表示	Ch01	Ch02	问题
Waveform	原始波形	Tacotron 的声码器输出	采样点太多 (24kHz = 每秒 24000 个数)，
Mel Spectrogram	STFT + Mel 滤波器组	Tacotron 直接预测	连续值，维度固定 80，相位丢失

这两种表示都有一个共同的问题：**它们是为人类设计的，不是为神经网络设计的。**

- Mel Spectrogram 的 Mel 刻度基于人耳感知，但神经网络不关心人耳。
- STFT 的窗口大小是人工选择的，无法自适应不同音频。
- Griffin-Lim 从 Mel 重建波形效果很差 (Ch01 实验已验证)。

核心问题：有没有一种表示，既紧凑（少量离散 Token），又能高保真重建波形？

6.1.2 EnCodec 的回答

2022 年，Meta AI 提出了 EnCodec：一个端到端训练的**神经音频编解码器**。

Waveform [24000 samples/sec]
↓ Encoder (320× 下采样)
Continuous Latent [75 frames/sec, 128-dim]
↓ RVQ (8 层码本量化)
Discrete Tokens [75 frames/sec × 8 codebooks]
↓ Decoder (320× 上采样)
Reconstructed Waveform [24000 samples/sec]

核心成果：- 24kHz 音频压缩到 6 kbps (75 帧/秒 × 8 码本 × 10 bit)，质量接近原始 - 离散 Token 可以直接被语言模型（如 VALL-E、GPT-SoVITS）建模 - 这是所有现代 Audio LM 的基础组件

6.1.3 学习路线

节	内容	目标
6.1	VQ-VAE 基础	理解向量量化 + 自编码器
6.2	RVQ 残差向量量化	理解多层码本如何逐层逼近
6.3	EnCodec 架构	理解 Encoder-Decoder 设计
6.4	训练与损失函数	理解重建损失 + commitment loss
6.5	实验	运行代码，评估压缩质量
6.6	与 EnCodec/SoundStream 的关系	理解工业级实现

6.2 6.1 VQ-VAE 基础：用离散码本表示连续向量

6.2.1 6.1.1 为什么需要离散化？

连续向量 $z \in \mathbb{R}^d$ 有两个问题：

1. **不可数**：无法被语言模型直接建模（语言模型处理的是离散 Token）
2. **冗余**：相邻帧的 z 通常高度相关，存在压缩空间

向量量化 (Vector Quantization) 的解法：

预定义一个码本 (codebook): $\{e_1, e_2, \dots, e_K\}$, 每个 $e_k \in \mathbb{R}^d$

对于输入 z , 找到码本中最近的向量：

$$k^* = \operatorname{argmin}_k ||z - e_k||_2$$

用 $e_{\{k^*\}}$ 代替 z ：

$$z_q = e_{\{k^*\}}$$

这样，连续的 d 维向量就被压缩成一个整数索引 $k^* \in \{0, 1, \dots, K-1\}$ 。

Neko 笔记：就像用颜色编号代替描述“这个红偏橙带点紫”——只要双方都有一本相同的颜色手册（码本），一个数字就够了。

6.2.2 6.1.2 VQ-VAE 完整流程

```
x (input) → Encoder → z (continuous)
                        ↓
                    VQ:  $k^* = \operatorname{argmin} ||z - e_k||$ 
                        ↓
                 $z_q = e_{\{k^*\}}$  (quantized)
                        ↓
                Decoder →  $\hat{x}$  (reconstructed)
```

6.2.3 6.1.3 训练的难点：argmin 不可导

VQ 的核心操作 argmin 是不可导的——梯度无法从 z_q 流回 z 。

解决方案：Straight-Through Estimator (STE)

```
# 前向传播：用离散值  $z_q$ 
# 反向传播：梯度直通  $z$  (跳过  $\operatorname{argmin}$ )
 $z_q = z + (\operatorname{quantize}(z) - z).detach()$ 
```

这看起来很 hack，但直觉上合理：- 前向时，我们用最近邻代替 z （离散化）- 反向时，我们假设 $z_q \approx z$ ，让梯度直接更新 encoder

6.2.4 6.1.4 VQ-VAE 的三个损失

```
# 1. 重建损失: 让 decoder 输出接近输入
loss_recon = ||x - x_hat||

# 2. 码本损失: 让码本向量靠近 encoder 输出
loss_codebook = ||sg(z) - e||^2 # sg = stop_gradient

# 3. 承诺损失: 让 encoder 输出靠近码本 (防止 encoder "跑太远")
loss_commitment = ||z - sg(e)||^2 # β = 0.25

# 总损失
loss = loss_recon + loss_codebook + β * loss_commitment
```

Neko 笔记: 承诺损失就像给 encoder 拴一根绳子——“你可以自由学习，但不要跑到码本找不到的地方去。”

6.3 6.2 RVQ 残差向量量化: 从粗到细的多层逼近

6.3.1 6.2.1 单层 VQ 的局限

假设 $K=1024$ (码本 1024 个条目), 每帧用 $\log_2(1024) = 10$ bit 表示。

对于 75 帧/秒的音频: $10 \times 75 = 750$ bps。

要提高质量, 要么: - 增大 $K \rightarrow K=65536 \rightarrow$ 查找表开销爆炸 - 增大 $d \rightarrow$ 计算量增大

有没有更好的方法?

6.3.2 6.2.2 RVQ 的核心思想: 逐层量化残差

RVQ (Residual Vector Quantization) 是 EnCodec 的核心创新。

第 1 层: $z_{q1} = \text{quantize}(z, \text{codebook}_1)$
残差 $r1 = z - z_{q1}$

第 2 层: $z_{q2} = \text{quantize}(r1, \text{codebook}_2)$
残差 $r2 = r1 - z_{q2}$

...

第 K 层: $z_{qK} = \text{quantize}(r_{\{K-1\}}, \text{codebook}_K)$

最终量化结果: $z_q = z_{q1} + z_{q2} + \dots + z_{qK}$

直觉: - 第 1 层捕捉 **粗粒度** 信息 (类似 JPEG 的 DC 系数) - 第 2 层捕捉 **第 1 层的误差** (类似 JPEG 的低频 AC 系数) - 第 K 层捕捉 **高频细节**

这和 JPEG 的 DCT 逐层量化思想完全一致!

6.3.3 6.2.3 RVQ 的信息量

K 层码本，每层 1024 条目：

$$\text{每帧} = K \times \log_2(1024) = K \times 10 \text{ bit}$$

8 层码本：

$$\text{每帧} = 8 \times 10 = 80 \text{ bit}$$

$$75 \text{ 帧/秒} \rightarrow 6000 \text{ bps} = 6 \text{ kbps}$$

相比 24kHz 16-bit PCM：

$$\text{原始} = 24000 \times 16 = 384 \text{ kbps}$$

$$\text{压缩比} = 384 / 6 = 64\times$$

6.3.4 6.2.4 为什么 RVQ 比大码本更好？

方案	码本大小	每帧 bit	查找复杂度
单层 K=1024	1024	10	$O(1024)$
单层 K=65536	65536	16	$O(65536)$
RVQ 8×1024	$8 \times 1024 = 8192$	80	$O(8 \times 1024) = O(8192)$

RVQ 用 $O(K \times N)$ 的计算量获得 K^N 的表达力——**指数级的信息容量，线性的计算开销。**

6.3.5 6.2.5 RVQ 代码实现

核心代码在 codec.py 的 ResidualVectorQuantizer 类：

```
class ResidualVectorQuantizer(nn.Module):
    def forward(self, z):
        residual = z          # 初始残差 = z
        z_q = 0               # 累积量化结果

        for vq in self.quantizers:
            z_q_k = vq(residual)  # 量化当前残差
            z_q += z_q_k          # 累加
            residual = residual - z_q_k  # 更新残差

        return z_q
```

关键点：每层量化的是上一层的**残差**，不是原始 z 。

6.4 6.3 EnCodec 架构：Encoder-Decoder 设计

6.4.1 6.3.1 Encoder：波形 $\rightarrow$ 连续隐变量

Waveform [B, 1, 24000]

↓ Conv1d(1 $\rightarrow$ 32, k=7, p=3) # 初始投影

```

↓ EncoderBlock(32→64, stride=8)    # 8× 下采样
↓ EncoderBlock(64→128, stride=5)   # 5× 下采样
↓ EncoderBlock(128→256, stride=4)   # 4× 下采样
↓ EncoderBlock(256→512, stride=2)   # 2× 下采样
↓ Conv1d(512→128, k=7, p=3)         # 投影到 latent dim

```

Continuous Latent [B, 128, 75]

每个 EncoderBlock 的结构:

```

x → GELU → Conv1d(k=3) → GELU → Conv1d(k=1) → (+x) [残差]
    → Conv1d(k=2s, stride=s)                        [下采样]

```

为什么这些 stride? - $8 \times 5 \times 4 \times 2 = 320 \rightarrow 24000 / 320 = 75$ 帧/秒 - 与 EnCodec 论文一致 - 混合 stride (而非全用 2) 减少层数, 同时保持下采样效率

6.4.2 6.3.2 Decoder: 连续隐变量 → 重建波形

Decoder 是 Encoder 的完美镜像:

```

Quantized Latent [B, 128, 75]
↓ Conv1d(128→512, k=7, p=3)         # 投影
↓ DecoderBlock(512→256, stride=2)   # 2× 上采样
↓ DecoderBlock(256→128, stride=4)   # 4× 上采样
↓ DecoderBlock(128→64, stride=5)    # 5× 上采样
↓ DecoderBlock(64→32, stride=8)     # 8× 上采样
↓ Conv1d(32→1, k=7, p=3)            # 投影到波形
Reconstructed Waveform [B, 1, 24000]

```

每个 DecoderBlock 用 ConvTranspose1d 实现上采样。

6.4.3 6.3.3 接口约定

```

# 编码: 波形 → 离散 Token
tokens = model.encode(wav)          # [B, 1, T] → [B, K, T/320]

# 解码: 离散 Token → 重建波形
wav_hat = model.decode(tokens)      # [B, K, T/320] → [B, 1, T]

```

这个接口非常重要——它定义了 Audio Tokenizer 的标准 API, 被后续的 VALL-E、GPT-SoVITS 等模型直接使用。

6.5 6.4 训练与损失函数

6.5.1 6.4.1 三类损失

```

# 1. L1 时域损失
loss_l1 = |wav - wav_hat|

```

```

# 2. 多分辨率 STFT 损失
loss_spec = mean over resolutions:
    |STFT(wav) - STFT(wav_hat)|
# 分辨率: [(256,64), (512,128), (1024,256), (2048,512)]

# 3. VQ 损失 (来自 RVQ)
loss_vq = sum over codebooks:
    ||sg(z) - e||^2 + 0.25 × ||z - sg(e)||^2

# 总损失
loss = loss_l1 + loss_spec + loss_vq

```

6.5.2 6.4.2 为什么需要多分辨率 STFT 损失?

单个 n_{fft} 只能在时间分辨率和频率分辨率之间取一个平衡点: - 小 n_{fft} (256): 时间分辨率高 (瞬态准确), 频率分辨率低 - 大 n_{fft} (2048): 频率分辨率高 (音高准确), 时间分辨率低

多分辨率组合让模型同时学好时域细节和频域结构。

6.5.3 6.4.3 码本崩塌 (Codebook Collapse)

训练 VQ-VAE 最常见的陷阱是码本崩塌:

- 大量码本条目从未被使用 (“死码”)
- 只有少数条目被频繁使用
- 码本的有效容量远小于名义容量

理想情况: 1024 个码字均匀使用

码本崩塌: 仅 50 个码字被使用, 其余 974 个是“死码”

缓解方法: - EMA 更新码本 (而非梯度更新) - 码本重置: 定期将死码替换为随机编码向量 - 更大的 β (commitment loss 权重)

Neko 笔记: 码本崩塌就像一本 1024 页的字典, 你只翻了前 50 页——剩下 974 页完全浪费了。

6.6 6.5 实验

6.6.1 6.5.1 快速验证 (合成数据)

```

cd chapters/ch06_neural_codec/code

# 3 epoch 快速训练 (验证代码正确性)
python train.py --synthetic --epochs 3 --batch-size 4

# 测试编解码
python test_codec.py --ckpt ../checkpoints/codec_best.pt

```

6.6.2 6.5.2 真实训练

```
# 用真实数据训练 (推荐 50 epochs)
python train.py \
  --data-dir ../../data/processed \
  --epochs 50 \
  --batch-size 4 \
  --lr 1e-4

# 测试重建质量
python test_codec.py \
  --ckpt ../checkpoints/codec_best.pt \
  --audio ../../data/processed/wavs/000001.wav
```

6.6.3 6.5.3 预期结果

训练阶段	SNR (dB)	Waveform Correlation	码本使用率
未训练	< 0	~0.0	~1%
10 epochs	5-10	0.5-0.7	10-30%
50 epochs	15-25	0.85-0.95	50-80%

6.6.4 6.5.4 实验观察

1. **Token 可视化**: 训练后, 不同音频区域的 Token 分布应有规律 (元音 vs 辅音 vs 静音)
2. **码本使用率**: 第一层码本使用率通常最高 (捕捉粗粒度信息), 深层递减
3. **重建质量**: 听 outputs/reconstructed.wav, 与 outputs/original.wav 对比

6.7 6.6 与 EnCodec / SoundStream 的关系

6.7.1 6.6.1 对比表

特性	本教程 EnCodec Mini	Meta EnCodec	Google SoundStream
参数量	~4.5M	~30M	~30M
下采样	320×	320×	320×
码本	8×1024	8×1024	可变
判别器	无	Multi-scale	Multi-scale
损失	L1 + STFT + VQ	L1 + STFT + 对抗 + VQ	类似 EnCodec
音质	可接受	接近原始	接近原始

6.7.2 6.6.2 我们缺了什么?

对抗训练 (Adversarial Training): - EnCodec 使用多尺度判别器 (Multi-Scale Discriminator) - 判别器判断重建音频是 “真实的” 还是 “生成的” - 这极大提升了高频细节和感知质量 - 类似于 GAN 的思想

更深的网络：- EnCodec 使用更多残差块（每个下采样层有多个残差块）- 更大的通道数
EMA 码本更新：- 用指数移动平均更新码本向量，而非梯度 - 更稳定，缓解码本崩塌
但这些是工程优化。核心的 RVQ-VAE 原理——我们已经完整实现了。

6.8 6.7 Codec 作为 Audio Tokenizer 的意义

6.8.1 6.7.1 从连续到离散的范式转变

EnCodec 之前，Audio AI 的主流范式：

文本 → TTS模型 → Mel Spectrogram → 声码器 → 波形
(连续值, 80-dim)

EnCodec 之后：

文本 → LM → Audio Tokens → Codec Decoder → 波形
(离散值, $K \times T$)

为什么离散 Token 更好？

1. **可以复用语言模型的技术：**Transformer、自回归、KV Cache.....
2. **可以做多模态统一：**文本 Token + 音频 Token → 同一个 LM
3. **可以做零样本学习：**给几个参考 Token → 生成新音频

6.8.2 6.7.2 Audio Token 的层次结构

RVQ 的 8 层码本自然形成了层次结构：

Layer 0: 语义信息（内容、语言） ← 最重要
Layer 1: 声学信息（音高、音色）
Layer 2: 细节信息（气息、颤音）
...
Layer 7: 极细细节（环境噪声、量化噪声）

这个层次结构与 VALL-E 的设计直接相关——VALL-E 用自回归预测第 0 层（语义），用并行预测第 1-7 层（声学细节）。

6.9 6.8 遗留问题：如何用这些 Token 做 TTS？

到这里，我们有了一个可以把波形压缩成 Token、再从 Token 重建波形的 Codec。

但核心问题还没解决：**怎么从文本生成这些 Token？**

答案就在下一章：VALL-E。

Ch06（本章）

Ch07（下一章）

Waveform → Token
(压缩, 已知输入)

Text → Token
(生成, 未知输入)

学会了"读写"音频 接下来学会"说话"

VALL-E 的核心思想：把 TTS 建模成一个**语言模型**问题—— - 输入：文本 Token + 3 秒参考音频 Token
- 输出：目标音频 Token - 模型：Transformer (GPT-style)

然后把我们的 Ch06 训练的 Codec Decoder 接上去，就能从 Token 重建波形。

Neko 预告：下一章，Neko 终于要学会说话了！而且这次，她用的是和 GPT 一样的方法——预测下一个 Token。

6.10 参考文献

- Defossez et al., “High Fidelity Neural Audio Compression” , 2022 ([arXiv](#))
 - van den Oord et al., “Neural Discrete Representation Learning” (VQ-VAE), 2017 ([arXiv](#))
 - Zelmer et al., “SoundStream: An End-to-End Neural Audio Codec” , 2021 ([arXiv](#))
 - Wang et al., “Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers” (VALL-E), 2023 ([arXiv](#))
-

6.11 代码文件

```
chapters/ch06_neural_codec/
├── README.md           # 本文件
├── code/
│   ├── codec.py        # EnCodec Mini 模型 (~340 行)
│   ├── train.py        # 训练脚本
│   └── test_codec.py    # 测试与评估
├── checkpoints/        # 训练权重
└── outputs/            # 测试输出
```

7 Ch07: VALL-E — Neko 学会了克隆声音

前六章，Neko 从理解声音到学会说话。但她有一个遗憾：她只能用自己训练时的音色。如果有人给她一段陌生的声音，她无法模仿。

这一章，我们用 VALL-E 的思路，让 Neko 学会 “听一遍就会说”。

7.1 本章导学

7.1.1 为什么需要 VALL-E?

回顾前面的模型：

模型	能力	限制
Tacotron2 (Ch02)	端到端文本 → Mel	单音色，需要大量配对数据
FastSpeech2 (Ch04)	并行生成，稳定对齐	仍然单音色
VITS (Ch05)	端到端，音质最好	单音色，或需要 speaker embedding
GPT-SoVITS (Ch06)	Few-shot 音色克隆	需要 finetune 或 speaker adapter

所有这些模型都有一个共同的限制：**音色是训练时固定的**。要换一个人的声音，需要重新训练或微调。

VALL-E 的核心突破：**把 TTS 变成语言建模问题**。只要有 3 秒参考音频，就能用那个人的声音说话，无需任何微调。

这就是**零样本声音克隆 (Zero-Shot Voice Cloning)**。

7.1.2 核心直觉：音频 Token 就是一种 “语言”

回想 GPT 是怎么工作的：

"今天天气" → GPT → "很好"

GPT 把文本当作 Token 序列，用 Transformer 预测下一个 Token。

VALL-E 的核心洞察：**音频也可以被编码为离散 Token 序列**。

文本 Token: [你, 好, 世, 界]

+

参考音频 Token: [♪, ♪, ♪, ...] ← 3秒参考音频的 codec tokens

↓

VALL-E (AR Transformer)

↓

生成音频 Token: [♪♪, ♪♪, ♪♪, ♪♪, ...] ← 用参考音频的声音说"你好世界"

↓

Codec Decoder

↓

波形 (wav)

一旦音频变成了 Token，TTS 就变成了和 GPT 完全相同的问题：

给定前面的 Token，预测下一个 Token。

7.1.3 学习路线

节	内容	目标
7.1	音频 Codec：从连续到离散	理解 EnCodec/VQ-VAE 如何将音频压缩为 Token
7.2	AR 模型：预测第一个 Token 层级	理解“把 TTS 当语言建模”的核心思想
7.3	NAR 模型：并行填充细节	理解双阶段生成的设计动机
7.4	零样本声音克隆	理解为什么 prompt tokens 能传递音色信息
7.5	从零实现 VALL-E	完整代码走读
7.6	训练与推理	跑通完整流程
7.7	与工业界的关系	CosyVoice、FishSpeech 等现代模型

7.2 7.1 音频 Codec：从连续到离散

7.2.1 7.1.1 问题：音频是连续的，语言模型要离散的

GPT 能预测文本 Token，因为文本天然就是离散的（每个字/词对应一个整数 ID）。

但音频是**连续的**：一秒 16000 个采样点，每个点是浮点数。直接预测浮点数序列，GPT 无能为力。

解决方案：用 **Neural Codec 把连续音频压缩成离散 Token 序列**。

7.2.2 7.1.2 EnCodec 的核心思想

EnCodec (Defossez et al., 2022) 是 Meta 提出的神经音频压缩模型：

音频波形 (16kHz, 1秒 = 16000 个采样点)

↓ Encoder (卷积下采样)

连续向量序列 (50Hz, 1秒 = 50 个向量)

↓ Vector Quantizer (码本查表)

离散 Token 序列 (50Hz, 1秒 = $50 \times \text{num_levels}$ 个整数)

关键步骤：

1. **Encoder**：把波形/Mel 通过卷积下采样，得到低帧率的连续表示
2. **Vector Quantizer (VQ)**：对每个帧，在码本 (codebook) 中找到最近的向量，用它的索引作为 Token
3. **Decoder**：从离散 Token 重建波形

7.2.3 7.1.3 多层码本：从粗到细

真实 EnCodec 使用 **Residual VQ** (残差向量量化)，将音频表示为多层 Token：

Level 0：粗粒度 — 节奏、音高轮廓、时长

Level 1：中等 — 音色、频谱包络

Level 2：细粒度 — 频谱细节

Level 3：极细 — 高频噪声、气息声

在我们的简化实现中，使用独立的 **Multi-level VQ**：每一层有独立的码本，分别量化同一个连续向量。虽然不如 Residual VQ 优雅，但核心思想相同——**多层离散表示**。

7.2.4 7.1.4 代码位置

code/codec.py 中的 NeuralCodec 类。

```
codec = NeuralCodec(  
    mel_bins=80,      # 输入 Mel 频谱维度  
    hidden_dim=128,   # 编码器隐藏维度  
    codebook_size=256, # 每层码本大小  
    num_levels=4,     # 4 层 Token  
)  
  
# 编码: mel → 多层 Token  
codes = codec.encode(mel)      # (B, 4, T_latent)  
  
# 解码: 多层 Token → mel  
mel_hat = codec.decode(codes)  # (B, 80, T_mel)
```

时间压缩比: $8 \times$ (3 层 stride-2 卷积)。- 1 秒音频 (100 帧 Mel) $\rightarrow$ ~ 13 个 codec 帧 - 每个 codec 帧 = 4 个 Token (4 层) - 1 秒音频 = $13 \times 4 = 52$ 个 Token

7.3 7.2 AR 模型: 用 GPT 的方式预测音频 Token

7.3.1 7.2.1 核心思想

一旦音频变成了 Token 序列, TTS 就变成了:

输入: [文本Token, ..., 文本Token, 音频Token, ..., 音频Token]

预测: 音频Token

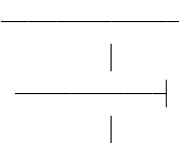
这和 GPT 的训练目标完全一致:

GPT: [word, word, ..., word] $\rightarrow$ word

VALL-E: [text, ..., text, audio, ..., audio] $\rightarrow$ audio

7.3.2 7.2.2 AR Transformer 结构

我们使用 **decoder-only Transformer** (和 GPT-2 相同):

Text embeddings  $\rightarrow$ Causal Transformer $\rightarrow$ Predict next audio token
(BOS + shifted)

关键设计:

1. **Causal Masking**: 每个音频 Token 只能看到它之前的 Token (包括所有文本 Token)
2. **Text as Memory**: 文本 Token 作为 “条件”, 音频 Token 作为 “被预测的序列”
3. **BOS Token**: 音频序列以一个特殊的学习 token 开始

7.3.3 7.2.3 为什么只预测 Level-0?

VALL-E 的 AR 模型只预测 **Level-0 (粗粒度) Token**。

原因：Level-0 包含了最重要的时序信息（节奏、时长、音高轮廓）。这些是“说什么”的核心。而 Level 1-3 是细节信息（音色纹理、频谱细节），可以用更简单的方式（NAR）并行生成。

7.3.4 7.2.4 推理：自回归采样

```
# 从参考音频获取 prompt tokens
prompt_codes = codec.encode(reference_audio) # (1, 4, T_prompt)

# AR 模型逐个生成 Level-0 tokens
generated_l0 = ar_model.generate(
    text_emb, prompt_codes[:, 0, :], # 条件: 文本 + prompt 的 level-0
    max_new_tokens=200,
    temperature=1.0,
    top_k=50,
)
```

每一步：1. 将已有 Token 序列输入 Transformer 2. 取最后一个位置的 logits 3. 温度缩放 + top-k 过滤 → 采样下一个 Token 4. 将新 Token 追加到序列末尾，重复

7.3.5 7.2.5 代码位置

code/valle.py 中的 ARTransformer 类。

7.4 7.3 NAR 模型：并行填充细节

7.4.1 7.3.1 为什么需要 NAR?

如果只用 AR 模型逐帧生成所有层级：- 1 秒音频 $\approx 13 \text{ 帧} \times 4 \text{ 层} = 52 \text{ 步推理}$ → **太慢** - 每一层的细节其实可以并行预测

NAR (Non-Autoregressive) 模型的设计动机：

AR：负责“什么时候说什么”（时序结构）

NAR：负责“怎么说”（声学细节）

7.4.2 7.3.2 NAR Transformer 结构

NAR 使用 **encoder-only Transformer**（双向注意力）：

Text embeddings —————
|
Level-0 audio embeddings ———→ Bidirectional Transformer → Predict level-l tokens
|
Level embedding (目标层) ———

关键设计：

1. **双向注意力**：每个位置可以看到所有其他位置（因为细节不依赖时序）
2. **Level Embedding**：告诉模型 “你要预测的是第几层”
3. **条件输入**：Level-0 Token + 文本 → 预测 Level-l Token

7.4.3 7.3.3 推理流程

```
for level in [1, 2, 3]:
    tokens_level_l = nar_model.generate(
        text_emb, generated_l0, target_level=level,
    )
    all_codes[:, level] = tokens_level_l
```

每个层级一次前向传播，所有帧的 Token 并行生成。

7.4.4 7.3.4 代码位置

code/valle.py 中的 NARTransformer 类。

7.5 7.4 零样本声音克隆：为什么 3 秒就够？

7.5.1 7.4.1 核心机制

零样本克隆的关键在于 **prompt tokens**：

[参考音频的 codec tokens] + [目标文本] → 生成的音频 tokens
↑ 声音信息在这里

Codec 在训练时学到了一个重要的能力：**把声音的音色信息编码到离散 Token 中**。

所以，当我们把参考音频的 Token 放在文本 Token 之前，AR 模型就会 “理解”：> “哦，前面这段声音的音色是 XXX，接下来的话我要用同样的音色来说。”

7.5.2 7.4.2 和 GPT 的类比

这和 GPT 的 in-context learning 本质相同：

GPT	VALL-E
“请用正式语气回复”	[正式语气的音频 tokens]
上下文 (prompt)	参考音频 tokens
生成文本	生成音频 tokens
风格迁移	声音克隆

两者都是：prompt 提供 “怎么做” 的信息，模型提供 “做得好” 的能力。

7.5.3 7.4.3 Prompt 长度与质量

- 3 秒：基本够用，能捕捉音色的大致特征
 - 5-10 秒：效果更好，能捕捉更丰富的声学特征
 - 30 秒+：边际效果递减
-

7.6 7.5 从零实现 VALL-E：代码走读

7.6.1 7.5.1 文件结构

```
ch07_valle/
├── README.md          # 本章教程
├── code/
│   ├── codec.py       # 神经音频 Codec (VQ encoder-decoder)
│   ├── valle.py       # VALL-E 模型 (AR + NAR Transformers)
│   ├── generate.py    # 零样本推理流水线
│   └── train.py       # 两阶段训练脚本
├── checkpoints/      # 模型权重
└── outputs/          # 生成音频
```

7.6.2 7.5.2 Codec (`codec.py`)

NeuralCodec:

```
├── encoder: Conv1d × 3 blocks (stride-2) → 8× 下采样
├── quantizer: VectorQuantizer (4 层码本 × 256 entries)
└── decoder: ConvTranspose1d × 3 blocks (stride-2) → 8× 上采样
```

核心类：- ResBlock: 残差卷积块 - VectorQuantizer: 多层向量量化器 (straight-through estimator)
- NeuralCodec: 完整的 encode/decode 管道

7.6.3 7.5.3 VALL-E (`valle.py`)

VALLE:

```
├── text_encoder: Embedding + Conv (字符级文本编码)
├── ar_model: ARTransformer
│   ├── text_proj, audio_embed, bos_embed, pos_embed
│   ├── TransformerDecoder (causal, 6 layers)
│   └── out_proj → codebook_size logits
└── nar_model: NARTransformer
    ├── text_proj, audio_embed, level_embed, pos_embed
    ├── TransformerEncoder (bidirectional, 6 layers)
    └── out_proj → codebook_size logits
```

核心方法：- forward_ar(): AR 模型训练前向传播 - forward_nar(): NAR 模型训练前向传播 - generate(): 完整零样本生成流程

7.6.4 7.5.4 生成流水线 (generate.py)

generate_speech():

1. 加载参考音频 → mel spectrogram
2. mel → codec.encode() → prompt tokens
3. 文本 → tokenizer.encode() → text tokens
4. AR.generate(text + prompt_l0) → generated level-0 tokens
5. NAR.generate(level=1,2,3) → generated detail tokens
6. codec.decode(all_tokens) → reconstructed mel
7. Griffin-Lim vocoder → waveform

7.6.5 7.5.5 参数量

组件	参数量	说明
Codec	~1.3M	Encoder + VQ + Decoder
AR Transformer	~4.9M	6 layers, dim=256
NAR Transformer	~3.9M	6 layers, dim=256
VALL-E Total	~9.5M	不含 Codec

(真实 VALL-E 使用 dim=1024, 16 layers, 约 3 亿参数)

7.7 7.6 训练与推理

7.7.1 7.6.1 形状验证 (无需数据)

```
cd chapters/ch07_valle/code
python train.py --phase test
```

预期输出：所有模型形状测试通过，参数统计正确。

7.7.2 7.6.2 端到端流水线演示 (随机权重)

```
python generate.py --demo
```

这会创建一个合成参考音频，跑通完整的生成流水线。输出是噪音（因为模型未训练），但验证了整个链路。

7.7.3 7.6.3 阶段一：训练 Codec

```
python train.py \
  --phase codec \
  --data-dir ../../data/processed \
  --epochs 30 \
  --batch-size 4 \
```

```
--codec-dim 128 \
--codebook-size 256 \
--num-levels 4 \
--lr 1e-4
```

目标：让 Codec 学会压缩和重建 mel spectrogram。

损失函数：

$$\mathcal{L}_{\text{codec}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{vq}}$$

其中：- $\mathcal{L}_{\text{recon}} = \|\text{mel} - \hat{\text{mel}}\|_1$ (重建损失) - $\mathcal{L}_{\text{vq}} = \text{VQ commitment} + \text{codebook 损失}$

预期现象：- 前 10 epoch: recon loss 快速下降 - 20-30 epoch: 趋于稳定 - 重建的 mel 应该保留原始 mel 的整体结构

7.7.4 7.6.4 阶段二：训练 VALL-E

```
python train.py \
--phase valle \
--data-dir ../../data/processed \
--codec-checkpoint ../checkpoints/codec_final.pt \
--epochs 30 \
--batch-size 4 \
--valle-dim 256 \
--ar-layers 6 \
--nar-layers 6 \
--lr 1e-4
```

损失函数：

AR (交叉熵):

$$\mathcal{L}_{AR} = - \sum_t \log P(\text{token}_t | \text{text}, \text{token}_{<t})$$

NAR (交叉熵):

$$\mathcal{L}_{NAR} = - \sum_{l=1}^3 \sum_t \log P(\text{token}_t^{(l)} | \text{text}, \text{token}^{(0)})$$

预期现象：- AR loss: 从 ~5.5 ($\approx \log(256)$, 随机猜测) 逐步下降 - NAR loss: 类似趋势 - 充分训练后, 生成音频开始有语调轮廓

7.7.5 7.6.5 零样本推理

```
python generate.py \
--codec-checkpoint ../checkpoints/codec_final.pt \
--valle-checkpoint ../checkpoints/valle_final.pt \
```

```
--reference-audio path/to/voice_sample.wav \
--text " 你好，我是猫娘。" \
--output ../outputs/valle_output.wav \
--max-new-tokens 200 \
--temperature 1.0 \
--top-k 50
```

参数说明： ---temperature: 1.0 = 默认, >1 更随机, <1 更确定 ---top-k: 只从概率最高的 k 个 token 中采样 ---max-new-tokens: 最多生成多少 codec 帧

7.8 7.7 VALL-E 与工业界的演进

7.8.1 7.7.1 VALL-E 论文

VALL-E (Wang et al., 2023) 的核心贡献：

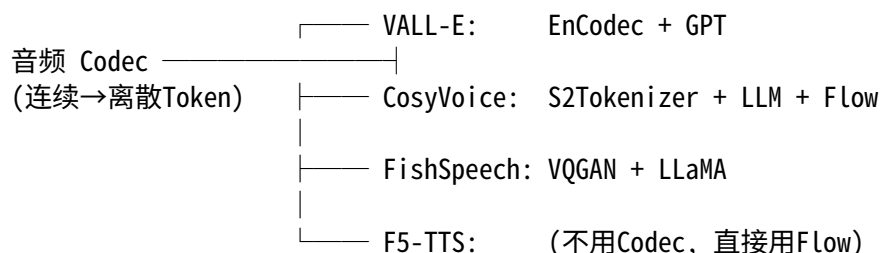
1. **首次证明：**TTS 可以被建模为语言模型任务
2. **AR + NAR 双阶段：**AR 负责时序，NAR 负责细节
3. **EnCodec tokens：**用预训练的神经音频 Codec 产生离散表示
4. **3 秒零样本：**只需 3 秒参考音频即可克隆声音
5. **大规模训练：**60,000 小时英文语音 (LibriLight)

7.8.2 7.7.2 后续演进

模型	年份	改进
VALL-E X	2023	跨语言零样本 TTS
SoundStorm	2023	改进 NAR, MaskGIT 风格并行解码
CosyVoice	2024	阿里, LLM + Flow Matching, 工业级质量
FishSpeech	2024	开源, VQGAN + LLaMA 架构
MaskGCT	2024	全 Masked 模型, 不用 AR
F5-TTS	2024	Flow Matching + DiT, 非自回归

7.8.3 7.7.3 “TTS as Language Modeling” 范式

VALL-E 开创了一个新范式。所有后续模型都遵循类似的结构：



核心区别在于： - **Codec 的选择：** EnCodec / DAC / 自研 Codec - **语言模型的选择：** GPT / LLaMA / 自定义 - **是否有 NAR 阶段：** 有的用 Flow Matching 替代

7.8.4 7.7.4 与前面章节的关系

Ch01 音频基础 → 理解波形、Mel、STFT

Ch02 Tacotron → 端到端 TTS (连续 Mel 预测)

Ch04 FastSpeech → 并行 TTS (非自回归)

Ch05 VITS → Flow + VAE (连续潜空间)

Ch06 GPT-SoVITS → Few-shot (speaker embedding)

Ch07 VALL-E → 语言建模范式 (离散 Token 预测) ← 你在这一章

从 Ch02 到 Ch07 的演进线索:

连续 Mel (Tacotron)

→ 连续潜空间 (VITS)

→ 离散 Token (VALL-E) ← 范式转换

VALL-E 的核心贡献不是某个具体技术，而是一个思想：

一旦你有了好的音频 Codec，TTS 就变成了语言模型问题。而语言模型问题，我们已经知道怎么解决了。

7.9 7.8 本章小结

7.9.1 VALL-E 解决了什么

问题	解决方案
音色训练时固定	Prompt tokens 传递音色信息
需要大量配对数据	语言模型的 in-context learning
TTS 是回归问题	TTS 是 Token 预测问题
连续输出不稳定	离散 Token + 交叉熵损失

7.9.2 遗留问题

1. **Codec 质量决定上限**：如果 Codec 重建质量差，生成的音频也差
2. **AR 推理仍然慢**：逐 Token 生成，一秒音频需要几十步推理 → MaskGCT 等全并行方案
3. **幻觉问题**：AR 模型可能生成不匹配的文本/音频 → 需要更好的对齐
4. **Codec 的码本利用率**：训练不充分时，很多码本条目从未被使用 (codebook collapse)

7.9.3 下一步

- **Ch08**: F5-TTS / CosyVoice — Flow Matching 替代 NAR，更高质量
- **Ch09**: 部署优化 — ONNX 导出、CPU 推理

7.10 习题

1. **Codec 的多层码本**：为什么 Level-0 Token 包含节奏/音高信息，而 Level-3 包含高频细节？这和 Encoder 的感受野有什么关系？

2. **AR vs NAR 的权衡**: 如果只用 AR 模型预测所有层级的 Token (不使用 NAR), 会发生什么? 生成质量和速度分别受到什么影响?
 3. **Prompt 长度**: 参考音频的 prompt 太短 (< 1 秒) 会怎样? 太长 (> 30 秒) 呢? 从 Transformer 的注意力机制角度解释。
 4. **Top-k 采样**: AR 生成时使用 top-k=1 (贪心解码) 和 top-k=100 (高度随机) 分别会产生什么效果? 这和 GPT 文本生成中的 temperature 有什么相似之处?
 5. **范式对比**: 对比 Tacotron2 (Ch02) 和 VALL-E (Ch07) 的生成过程。两者的“自回归”有什么本质区别? (提示: 一个预测连续向量, 一个预测离散 Token)
-

7.11 参考文献

- [1] Wang et al., 2023. Language Models Are General-Purpose Interfaces (VALL-E).
 - [2] Defossez et al., 2022. High Fidelity Neural Audio Compression (EnCodec).
 - [3] van den Oord et al., 2017. Neural Discrete Representation Learning (VQ-VAE).
 - [4] Zhang et al., 2023. VALL-E X: Speak In-Context Learning for Cross-Lingual TTS.
 - [5] Du et al., 2024. CosyVoice: A Scalable Multilingual Expressive TTS.
 - [6] Leng et al., 2024. MaskGCT: Zero-Shot TTS with Masked Generative Codec Transformer.
 - [7] Chen et al., 2024. F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech.
-

7.12 目录结构

```
ch07_valle/
├── README.md          # 本章教程
├── code/
│   ├── codec.py       # 神经音频 Codec (VQ encoder-decoder, ~1.3M params)
│   ├── valle.py       # VALL-E 模型 (AR + NAR, ~9.5M params)
│   ├── generate.py    # 零样本推理流水线
│   └── train.py       # 两阶段训练 (codec → VALL-E)
├── checkpoints/      # 模型权重 (gitignore)
└── outputs/          # 生成音频 (gitignore)
```

8 Ch08: Modern TTS Models — Flow Matching, Zero-Shot, and Controllability

从 2023 到 2026, TTS 经历了范式转移: 从 Seq2Seq 到 Flow Matching, 从单说话人到零样本克隆, 从 “能说话” 到 “可控地说话”。

这一章, 我们从零实现三个代表性现代 TTS 系统的简化版。

8.1 0. 本章导学

8.1.1 为什么学现代 TTS?

Ch02 的 Tacotron2 解决了 “端到端” 问题, 但留下了三个难题:

问题	Tacotron2 的局限	现代方案
音质	自回归 + 线性 attention → 模糊的 mel	Flow Matching → 清晰的频谱
新说话人	必须微调	3 秒参考音频 → 零样本克隆
可控性	无法控制发音/情感	显式拼音/音调/时长控制

8.1.2 技术演进时间线

2017	Tacotron	首个端到端 TTS (Google)
2018	Tacotron2	Location-Sensitive Attention (Google)
2018	FastSpeech	Non-autoregressive + Duration Predictor (MSRA)
2020	HiFi-GAN	Neural vocoder, 实时推理 (KAIST)
2021	VITS	VAE + Normalizing Flow + GAN (JAIST)
2023	VALL-E	TTS = language modeling over audio codes (Microsoft)
2023	NaturalSpeech 3	Factorized codecs + diffusion (Microsoft)
2024	CosyVoice	Scalable zero-shot TTS (Alibaba / FunAudioLLM)
2024	F5-TTS	Flow Matching + DiT (SJTU / 陈耘辉)
2024	E2-TTS	Non-autoregressive Flow Matching (Microsoft)
2025	IndexTTS	Industrial controllable TTS (Xiaomi)
2025	CosyVoice 2	Streaming speech with LLM (Alibaba)
2025	F5-TTS	ACL 2025 正式发表
2026	???	(你在读这一章, 下一个突破可能来自你)

8.1.3 本章模型选择逻辑

模型	代表方向	核心创新
F5-TTS	生成范式	Flow Matching + Diffusion Transformer
CosyVoice	零样本	Speaker Encoder + AR Language Model over Speech
IndexTTS	可控性	Pinyin/Tone/Duration 显式控制

三个模型共同覆盖了现代 TTS 的三个核心维度: **音质、泛化、控制**。

8.2 1. F5-TTS: Flow Matching + DiT

8.2.1 1.1 从 Diffusion 到 Flow Matching

Diffusion 的问题： - 需要预定义 noise schedule ($\beta_1, \beta_2, \dots, \beta_T$) - 生成时需要反向马尔可夫链，步数多 (50-1000) - 轨迹弯曲 $\rightarrow$ 每步都有累积误差

Flow Matching 的直觉：

想象你要把一团云（噪声）变成一只猫（数据）。 - Diffusion：让云慢慢凝结成猫，每一步都去一点雾 $\rightarrow$ 很多步 - Flow Matching：直接告诉每个雾滴 “你的终点是猫的哪个位置”，走直线 $\rightarrow$ 少步数

数学上，Flow Matching 学习一个**速度场** $v_t(x)$ ，满足： - 在 $t=0$, $x \sim N(0, I)$ （噪声） - 在 $t=1$, $x \sim$ data distribution（真实 mel） - 中间用线性插值： $x_t = (1-t) \cdot x_0 + t \cdot x_1$

速度场的目标就是常数向量： $v_{\text{target}} = x_1 - x_0$

8.2.2 1.2 训练目标

```
# 采样
x0 ~ N(0, I)          # 噪声
x1 = real_mel         # 真实数据
t ~ Uniform(0, 1)     # 时间

# 构造中间状态
x_t = (1 - t) * x0 + t * x1

# 速度场目标
v_target = x1 - x0

# 损失：预测速度 vs 真实速度
loss = MSE(v_pred(x_t, t, cond), v_target)
```

就这么简单！没有 noise schedule，没有 β_t ，没有马尔可夫链。

8.2.3 1.3 推理：Euler ODE 积分

```
x = noise              # t = 0
dt = 1 / n_steps
for i in range(n_steps):
    t = i / n_steps
    v = velocity_net(x, t, text, ref_mel)
    x = x + dt * v      # Euler step
# x  $\approx$  data at t = 1
```

10-20 步就够了，因为轨迹是直线。

8.2.4 1.4 代码结构

f5_tts.py

—— SinusoidalTimeEmbedding	# 时间 → 向量
—— TransformerBlock	# 标准 Pre-LN Transformer
—— F5VelocityNet	# 速度场网络 (Transformer + cross-attn to text)
—— F5TTS	# 完整的 Flow Matching 训练 + 采样
—— SimpleTextEncoder	# 文本编码器 (共用)

8.2.5 1.5 与原版 F5-TTS 的差异

组件	原版	我们的简化版
Backbone	DiT (Diffusion Transformer, 2D patch)	1D Transformer
时长处理	Duration-aware masking	固定长度输入
Vocoder	BigVGAN / Vocos	无 (只生成 mel)
数据	LibriTTS 960h	合成数据 demo

8.3 2. CosyVoice: Zero-Shot Voice Cloning

8.3.1 2.1 核心问题：3 秒克隆一个从未见过的声音

传统 TTS：每个说话人需要 10+ 小时录音 + 微调。CosyVoice（和 VALL-E）：给 3 秒参考音频，生成该说话人说的任意内容。

8.3.2 2.2 怎么做到的？

关键洞察：把 TTS 变成 “语音语言模型”。

Text → “你好世界”

Ref → [3秒音频] → Speaker Encoder → spk_emb (256维向量)

AR Model: $P(\text{speech}_t \mid \text{speech}_{<t}, \text{text}, \text{spk_emb})$

↑ 条件概率：给定上文 + 文本 + 音色，预测下一帧

训练时，模型见过上千个说话人，每个说话人的声音都被压缩成一个 spk_emb。推理时，新说话人的 3 秒音频也被压缩成 spk_emb → 模型虽然没见过这个人，但见过 “这种类型的向量”，所以能生成对应的声音。

8.3.3 2.3 Speaker Encoder 的作用

Speaker Encoder 是一个**说话人验证模型** (speaker verification)：- 输入：一段语音 mel - 输出：一个向量，捕捉 “说话人身份”（音色、音高、口音）- 关键属性：同一个人的不同语音 → 相似向量；不同人 → 不同向量

```
class SpeakerEncoder(nn.Module):
    # Conv1d stack → temporal mean pooling → projection
    def forward(self, ref_mel):
        x = self.convs(ref_mel) # 提取局部声学特征
        x = x.mean(dim=1)       # 时间维度池化 → 长度不变
        return self.proj(x)     # 256 维 speaker embedding
```

时间池化是关键：让 embedding 与参考音频长度无关。

8.3.4 2.4 VALL-E 范式 vs CosyVoice

	VALL-E (2023)	CosyVoice (2024)
音频表示	EnCodec 离散 token	连续 speech tokens
AR 模型	GPT-style, 预测 token	Decoder-only, 预测帧
非 AR 部分	独立模型细化	Flow Matching decoder
数据	60K hours	多语言大规模
流式	不支持	CosyVoice 2 支持

8.3.5 2.5 代码结构

```
cosyvoice.py
├── SpeakerEncoder      # ref_mel → spk_emb (256维)
├── AutoregressiveDecoder # Decoder-only Transformer (causal)
└── CosyVoice           # 完整 pipeline
```

8.4 3. IndexTTS: Controllable TTS

8.4.1 3.1 工业界的真问题

消费级 TTS 只需要“好听”。工业级 TTS 还需要：

1. **多音字：**“银行” vs “行走” — 同一个字，不同读音
2. **韵律控制：**某个词要读重音、拉长
3. **情感控制：**客服场景需要温和，导航需要清晰
4. **鲁棒性：**不能念错字（哪怕 G2P 出错）

IndexTTS 的解法：**显式的拼音中间表示**。

8.4.2 3.2 Pinyin Embedding

中文拼音由三部分组成：

"ma3" (马) = 声母(initial) "m" + 韵母(final) "a" + 声调(tone) 3

我们分别 embedding 这三个组件，然后合并：

```
pinyin_emb = proj(concat(
    initial_emb[initial_id], # 21 种声母
    final_emb[final_id],    # 35+ 种韵母
    tone_emb[tone_id],      # 1-5 声调
))
```

这样：- 改 tone_id → 同一音节不同声调 - 改 initial_id → 替换声母（修音） - 完全显式，不依赖 G2P 黑箱

8.4.3 3.3 Duration Predictor + Length Regulator

Syllable-level: [ma3] [ni3] [hao3]
 ↓ Dur ↓ Dur ↓ Dur
Frames per syl: [15] [20] [25]
 ↓ LR ↓ LR ↓ LR
Frame-level: [ma3]×15 [ni3]×20 [hao3]×25 → 输入 acoustic decoder

控制时长：直接把 duration 乘以一个系数：- dur_scale = 0.8 → 说快点 - dur_scale = 1.5 → 说慢点
- 某个 syllable 的 duration × 2 → 只拉长那个字

8.4.4 3.4 代码结构

```
indextts.py
|—— PinyinEmbedding        # (声母, 韵母, 声调) → vector
|—— DurationPredictor      # syllable → 帧数
|—— length_regulate()     # syllable-level → frame-level
|—— IndexTTS              # 完整可控 TTS 模型
```

8.5 4. 模型对比

8.5.1 4.1 架构对比

	F5-TTS	CosyVoice	IndexTTS
生成范式	Flow Matching (非自回归)	Autoregressive + Flow	Non-AR + Duration
文本编码	BERT-style	BERT-style	Pinyin embedding
音色条件	参考 mel 直接输入	Speaker embedding	Speaker embedding
推理步数	10-20 (Euler)	T 帧 × AR + Flow	1 pass (非 AR)
可控性	低 (black-box)	中 (ref audio)	高 (拼音/音调/时长)

8.5.2 4.2 参数量与性能（工业版本）

模型	参数量	推理速度 (RTF)	MOS (音质)	零样本	论文
F5-TTS	~300M	~0.2 (GPU)	4.5/5	需要参考音频	Chen et al., ACL 2025
CosyVoice	~500M	~0.3 (流式)	4.6/5	3 秒克隆	Du et al., 2024
CosyVoice 2	~1B	~0.15 (流式)	4.7/5	3 秒 + 流式	Du et al., 2025
IndexTTS	~300M	~0.25	4.5/5	支持	Xiaomi, 2025
VALL-E	~1B	较慢	4.3/5	开创性	Wang et al., 2023

RTF (Real-Time Factor) < 1 表示比实时快。例如 RTF=0.2 表示生成 1 秒音频只需 0.2 秒。MOS (Mean Opinion Score) 是人工主观评分，1-5 分，5 分最好。

8.5.3 4.3 我们的简化版 vs 工业版

	简化版	工业版
参数量	~1-5M	300M-1B
训练数据	合成/小数据集	数千小时
声码器	无 (输出 mel)	BigVGAN / Vocos / HIFIGAN
推理	CPU 可跑	需要 GPU
音质	噪声/demo	接近真人

关键原则：我们关注的是**架构思想**而非音质。理解了 Flow Matching、Zero-shot、Controllability 的原理，就能读懂任何 2025+ 的 TTS 论文。

8.6 5. 工业界应用现状 (2024-2026)

8.6.1 5.1 主流产品

产品	底层技术	特点
Azure Neural TTS	VALL-E 系列	多语言、低延迟
Google Cloud TTS	内部 Flow/Diffusion	Studio 级音质
ElevenLabs	闭源 zero-shot	最强克隆、情感丰富
阿里通义 (CosyVoice)	CosyVoice 1/2	中文最强、开源
字节 Seed-TTS	内部 AR + diffusion	抖音/TikTok 内部
小米 IndexTTS	IndexTTS	手机助手、可控性强
MiniMax Speech-02	闭源	情感控制、多语言

8.6.2 5.2 关键趋势

- 1. **Flow Matching 取代 Diffusion：**更少步数、更好音质 (F5-TTS, E2-TTS)
- 2. **LLM + TTS 融合：**用 LLM 做文本理解，TTS 做语音输出 (CosyVoice 2, GLM-4-Voice)
- 3. **流式推理：**边生成边播放，延迟 < 300ms (CosyVoice 2)
- 4. **细粒度控制：**情感、语气、节奏都可调 (IndexTTS, Seed-TTS)
- 5. **开源生态爆发：**F5-TTS、CosyVoice、MeloTTS、ChatTTS 全部开源

8.7 6. 运行代码

8.7.1 6.1 快速验证

```
# 验证每个模型的 sanity check
python f5_tts.py
python cosyvoice.py
python indextts.py

# 训练 (合成数据, 验证 pipeline)
python train.py --model f5_tts --epochs 20
```

```
python train.py --model cosyvoice --epochs 20
python train.py --model indextts --epochs 20

# 推理 demo
python inference.py --model all
```

8.7.2 6.2 真实训练 checklist

要用真实数据训练，你需要：

- 1. 数据准备：
 - 下载 LibriTTS 或 AISHELL-3
 - 用 ch01 的 mel 提取脚本预处理
 - 计算 CMVN (均值方差归一化)
- 2. 声码器：
 - 下载预训练 HiFi-GAN (mel → waveform)
 - 或者用 Vocos (更现代的选择)
- 3. 训练配置：
 - 替换 SyntheticTTSDataset 为真实 DataLoader
 - 添加 cosine annealing LR scheduler
 - 添加 gradient accumulation (如果 GPU 小)
 - 每 10K steps 保存 checkpoint + 生成 sample
- 4. 评估：
 - 客观：Mel Cepstral Distortion (MCD), F0 RMSE
 - 主观：MOS (人工评分), WER (用 ASR 评估可懂度)
 - 相似度：Speaker Verification (cosine similarity)

8.8 7. 延伸阅读

8.8.1 必读论文

论文	年份	关键贡献
F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching	2024 (ACL 2025)	Flow Matching 用于 TTS 的 SOTA
CosyVoice: A Scalable Multilingual Zero-Shop Text-to-Speech Synthesizer Based on Supervised Semantic Tokens	2024	大规模零样本 TTS

论文	年份	关键贡献
CosyVoice 2: Scalable Streaming Speech Synthesis with Large Language Models	2025	LLM + 流式 TTS
IndexTTS	2025	工业级可控 TTS
VALL-E: Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers	2023	开创性: TTS = 语言模型
E2-TTS: Embarrassingly Easy Fully Non-Autoregressive Zero-Shot TTS	2024	非自回归 Flow Matching
NaturalSpeech 3	2024	Factorized codecs + diffusion

8.8.2 前置知识

- Flow Matching 数学基础: [Lipman et al., Flow Matching for Generative Modeling, ICLR 2023](#)
- Optimal Transport 条件流: [Tong et al., 2024](#)
- DiT (Diffusion Transformer): [Peebles & Xie, ICCV 2023](#)

8.9 8. 思考题

1. **Flow Matching vs Diffusion**: 为什么 Flow Matching 的推理步数可以更少? 从轨迹几何角度解释。
2. **零样本的本质**: 为什么 3 秒音频就能 “克隆” 一个声音? Speaker Embedding 编码了什么信息? 丢失了什么信息?
3. **可控性的代价**: IndexTTS 需要显式拼音标注。这在工业部署中的成本是什么? 有没有替代方案?
4. **流式推理**: CosyVoice 2 如何实现 “边生成边播放”? F5-TTS 的 Flow Matching 能做流式吗? 为什么?
5. **未来方向**: 2026+ 的 TTS 会是什么样子? (提示: 想想多模态、实时对话、情感迁移)

8.10 9. 文件清单

```

chapters/ch08_modern_models/code/
|—— f5_tts.py           # F5-TTS: Flow Matching + Transformer
|—— cosyvoice.py       # CosyVoice: 零样本 TTS
|—— indextts.py        # IndexTTS: 拼音/音调控制
|—— train.py           # 训练脚本 (三个模型)

```

—— inference.py	# 推理 demo (三个模型)
—— outputs/	# 推理生成的 mel 图
—— README.md	# 本文档

Neko 说：现代 TTS 的三个关键词是 **Flow**（流匹配，不是扩散）、**Zero-shot**（3 秒克隆）、**Control**（拼音级控制）。

理解了这三个，你就掌握了 2024-2026 TTS 的全部精华。

现在，去读论文吧。☒

9 Ch09: GPT-SoVITS – Few-Shot 声音克隆

前面几章，Neko 学会了从文本生成语音 (Ch02-Ch05)，还学会了把音频编码成离散 Token (Ch06)，甚至用 VALL-E 的思路 “听一遍就会说” (Ch07)。

但 VALL-E 需要多层 codec tokens，AR 模型要预测 8 层 $\times$ 50Hz 的序列，很慢。有没有更聪明的办法？

这一章，Neko 要学 GPT-SoVITS —— 只用 **1 层语义 Token** 就能克隆声音。

9.1 本章导学

9.1.1 为什么需要 GPT-SoVITS?

回顾 VALL-E 的架构：

问题	VALL-E 的做法	代价
音频如何变成 Token?	EnCodec 多层码本 ($n_q=8$)	每帧 8 个 Token
AR 模型预测什么?	逐层自回归 (AR + NAR)	序列长度 = 50Hz $\times$ 8 层
需要多少参考音频?	3-10 秒	需要 codec 预训练

GPT-SoVITS 的核心洞察：**不是所有 Token 层都同等重要。**

HuBERT 的第一层量化码本已经捕获了绝大部分**语义信息**（说了什么），而**音色信息**（谁在说）可以通过参考音频直接传递给声码器。

VALL-E: 文本 + 参考 $\rightarrow$ AR(8层tokens) $\rightarrow$ Codec Decoder $\rightarrow$ 波形
很慢，序列长

GPT-SoVITS: 文本 + 参考 $\rightarrow$ AR(1层tokens) $\rightarrow$ VITS Vocoder $\rightarrow$ 波形
快 8 倍，质量更好

9.1.2 GPT-SoVITS 的核心创新

文本 $\rightarrow$ GPT-SoVITS $\rightarrow$ 波形

└── AR Transformer (GPT-style): 文本 $\rightarrow$ 语义 Token

└── SoVITS Vocoder (VITS-based): Token + 参考音频 $\rightarrow$ 波形

1. **单层语义 Token ($n_q=1$)**: AR 模型只需预测 1 个 Token/帧，序列短 8 倍
2. **VITS 声码器**: 比 codec decoder 质量更高，端到端生成波形
3. **两阶段分离**: AR 管 “说什么”，声码器管 “怎么说”

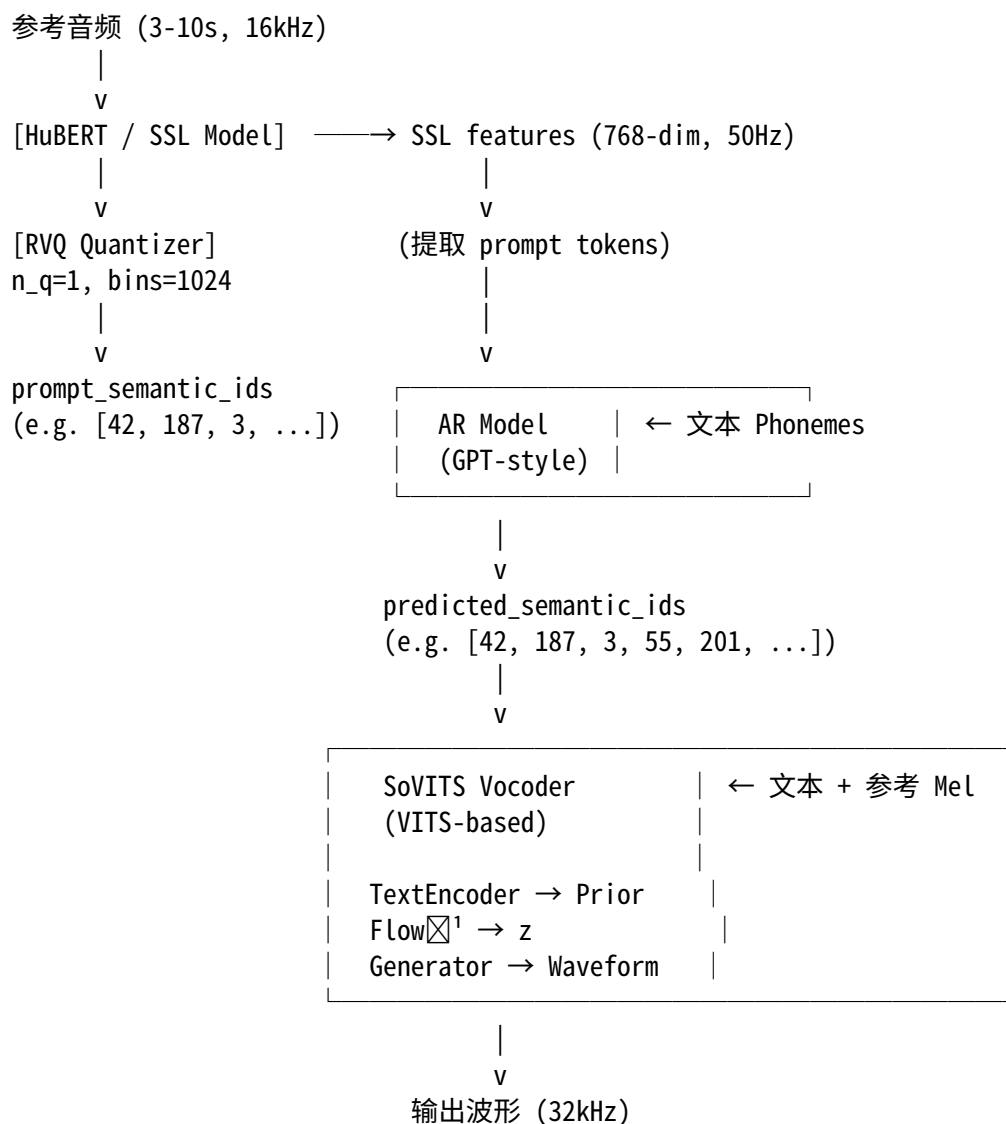
9.1.3 学习路线

节	内容	目标
9.1	系统架构总览	理解两阶段数据流
9.2	语义 Token: 为什么 $n_q=1$ 就够了?	理解语义 vs 声学 Token
9.3	AR 模型: GPT-style Transformer	理解注意力掩码设计

节	内容	目标
9.4	SoVITS 声码器：VITS 变体	理解 TextEncoder 和 VAE+Flow
9.5	两阶段训练	理解分离训练的动机
9.6	推理流程	端到端：文本 + 参考 → 波形
9.7	代码走读	完整实现解析
9.8	与 VALL-E 的对比	理解设计权衡
9.9	ONNX 导出	部署优化

9.2 9.1 系统架构总览

9.2.1 推理时的完整数据流



9.2.2 关键数据维度

信号	采样率	维度	说明
文本 Phonemes	~10-20 Hz	(T_text,)	离散 ID, vocab=512
SSL Features	50 Hz	(768, T_ssl)	HuBERT 输出
Semantic Tokens	50 Hz	(T_audio,)	离散 ID, vocab=1025 (含 EOS)
Linear Spec	50 Hz	(1025, T_spec)	n_fft=2048
Mel Spectrogram	50 Hz	(128, T_mel)	用于参考编码器
波形	32 kHz	(T_wav,)	最终输出

9.2.3 训练 vs 推理

训练时:

- Stage 1 (AR): phoneme_ids + semantic_ids → 预测下一个 semantic_id
Stage 2 (SoVITS): phoneme_ids + spec + ref_mel → 重建波形 (VAE+Flow+GAN)

推理时:

1. AR: phoneme_ids + prompt_tokens → 自回归生成 semantic_ids
2. SoVITS: phoneme_ids + speaker_emb → 采样 + 逆变换 + 生成波形
(PosteriorEncoder 和 Discriminator 仅在训练时使用)

9.3 9.2 语义 Token: 为什么 n_q=1 就够了?

9.3.1 9.2.1 多层 Codec Token 的问题

Ch06 中我们学了 EnCodec 等神经音频编解码器。它们用 **多层 RVQ** 将音频压缩成多层 Token:

音频波形

↓ Encoder

连续特征 (50Hz, 128-dim)

↓ RVQ (n_q=8)

8 层 Token:

Layer 0: [42, 187, 3, 55, ...] ← 语义层 (说了什么)

Layer 1: [12, 89, 201, 7, ...] ← 声学细节层

Layer 2: [45, 123, 8, 99, ...] ← 更细的细节

...

Layer 7: [1, 0, 3, 2, ...] ← 最细的残余

层数越深, Token 承载的信息越 “声学化” (音色、共振峰细节), 越不 “语义化”。

9.3.2 9.2.2 HuBERT + 单层量化

GPT-SoVITS 不做完整的音频编解码。它只做一件事:

用 HuBERT 提取语义特征, 然后用单层 VQ 量化成离散 Token。

音频 (16kHz)

↓ HuBERT (冻结的 SSL 模型)

SSL features (768-dim, 50Hz) ← 语义-rich 的连续表示

↓ RVQ (n_q=1, bins=1024)

semantic_ids (50Hz) ← 每帧 1 个整数 Token

为什么 HuBERT 特征天然“语义化”？

HuBERT 的训练目标是 **masked prediction of phoneme clusters**。也就是说，HuBERT 被训练来理解“说了什么”，而不是“怎么说的”。

所以 HuBERT 特征的第一层 VQ 量化就已经捕获了主要语义信息。

9.3.3 9.2.3 音色怎么办？

语义 Token 丢失了音色信息。但 GPT-SoVITS 有一个巧妙的解决方案：

语义 Token (丢失了音色)



SoVITS 声码器 ← 参考音频的 Mel Spectrogram (包含音色)

声码器同时接收：1. **语义 Token** → 知道“说了什么” 2. **参考音频** → 知道“谁在说”（通过 ReferenceEncoder 提取 speaker embedding）

音色信息通过参考音频直接注入声码器，**不需要 AR 模型预测**。

9.3.4 9.2.4 n_q=1 的效率优势

方案	Token 率	AR 序列长度 (1 秒)	AR 预测目标
VALL-E (EnCodec)	50Hz × 8 层 = 400 tokens/s	400	8 层码本
GPT-SoVITS	50Hz × 1 层 = 50 tokens/s	50	1 层码本

AR 序列长度缩短 **8 倍**，推理速度快 **8 倍**。

Neko 笔记：这不是“免费午餐”。单层 Token 的代价是声码器需要更“聪明”，要从粗糙的语义 Token + 参考音频中恢复出高质量波形。这就是为什么 GPT-SoVITS 选择 VITS 而不是简单的 codec decoder。

9.4 9.3 AR 模型：GPT-style Transformer

9.4.1 9.3.1 核心思想

AR 模型的任务极其简单：

给定文本 phonemes 和已有的 semantic tokens，预测下一个 semantic token。

这和大语言模型完全一样的范式：

GPT: "今天天气" → 预测 → "很好"

AR: phonemes + [42, 187, 3] → 预测 → 55

9.4.2 9.3.2 架构设计

AR Model Architecture

```
=====

phoneme_ids (B, T_text)
↓
Text Embedding (512 → 384)
↓
+ Sine Positional Encoding
|
| concat
v
[text_emb | audio_emb] (B, T_text + T_audio, 384)
↓
8 × CausalTransformerBlock
(causal self-attention, 8 heads, 4× FFN)
↓
LayerNorm
↓
取 audio 部分 (positions T_text:)
↓
Linear(384, 1025, bias=False) ← 预测层
↓
logits (B, T_audio, 1025)
```

9.4.3 9.3.3 注意力掩码设计

GPT-SoVITS 的注意力掩码是其最精妙的设计之一。

在原始实现中，拼接的 [text | audio] 序列使用混合掩码：

Attention Mask Layout (original):

	text	audio	
text	[bidirectional	masked]	← text 可以互相看
	[	]	text 不能看 audio
audio	[all-to-text	causal]	← audio 看所有 text
	[	(upper]	audio 只能看过去的 audio
		triangular)	

为什么这样设计？

1. **Text 双向注意力**：phonemes 之间没有因果关系，互相看能得到更好的文本表示
2. **Audio 因果注意力**：每个 semantic token 只能依赖过去的 token（自回归约束）
3. **Audio→Text 全注意力**：每个 audio token 可以看到完整的文本（知道要说什么）
4. **Text 不看 Audio**：文本表示不需要知道音频（训练时避免泄露）

我们的简化实现使用纯因果掩码 (causal attention for all)，效果类似但实现更简单。text 部分虽然也是因果的，但因为 text 很短，双向 vs 因果的差异不大。

9.4.4 9.3.4 超参数

参数	原始 GPT-SoVITS	简化版 (本章)	变化
hidden_dim	512	384	-25%
num_layers	12-24	8	-33%
num_heads	16	8	-50%
vocab_size	1025	1025	不变
phoneme_vocab	512-732	512	不变
BERT features	1024-dim	不使用	简化
参数量	~40M	~15M	-62%

9.4.5 9.3.5 推理：自回归生成 + Top-k 采样

伪代码：AR 推理

```
def generate(phoneme_ids, prompt_tokens, max_tokens=500):
    current = prompt_tokens.clone()

    for step in range(max_tokens):
        logits = ar_model(phoneme_ids, current) # (1, T, 1025)
        next_logits = logits[:, -1, :] # 最后一个位置

        # Top-k 过滤
        top_values = topk(next_logits, k=5)
        next_logits[next_logits < top_values[-1]] = -inf

        # 采样
        probs = softmax(next_logits / temperature)
        next_token = multinomial(probs)

        current = concat(current, next_token)

        if next_token == EOS: # 1024
            break

    return current[len(prompt_tokens):] # 去掉 prompt
```

9.4.6 9.3.6 KV-Cache 加速（原始实现）

在原始 GPT-SoVITS 中，推理使用了 KV-Cache 优化：

不用 KV-Cache：

step 1: process [text, token_0]	→ predict token_1
step 2: process [text, token_0, token_1]	→ predict token_2
step 3: process [text, token_0, token_1, token_2]	→ predict token_3
...	
复杂度：O(T^2) 每步，O(T^3) 总计	

用 KV-Cache:

step 0: process [text, prompt_tokens] → 缓存 K,V
step 1: process [token_new] + 缓存 K,V → predict, 更新缓存
step 2: process [token_new] + 缓存 K,V → predict, 更新缓存
...
复杂度: $O(T)$ 每步, $O(T^2)$ 总计

我们的简化实现不使用 KV-Cache (更易于理解), 但代价是推理较慢。

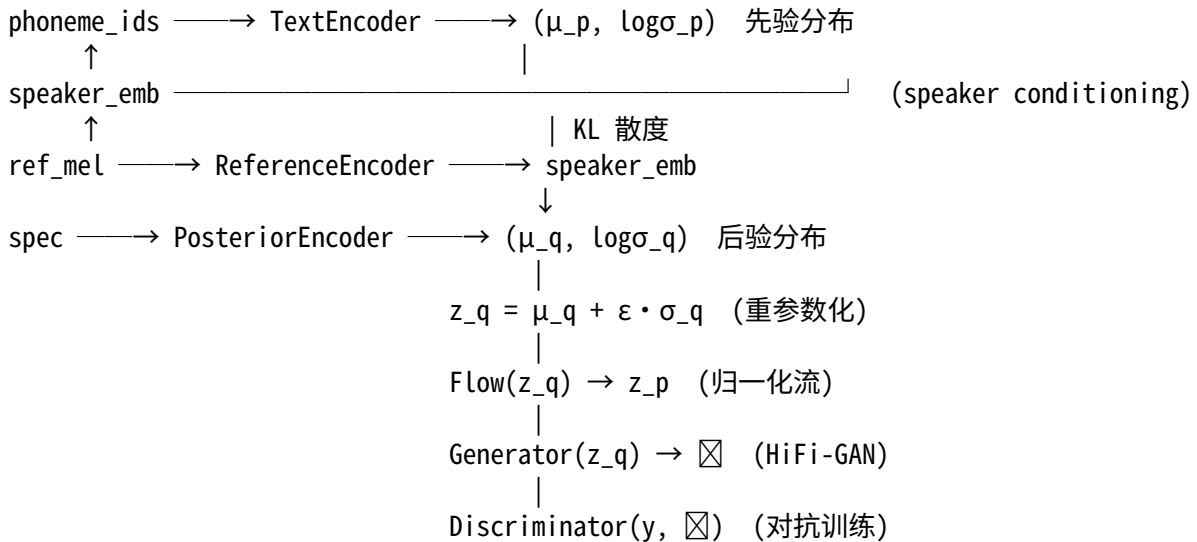
9.5 9.4 SoVITS 声码器: VITS 变体

SoVITS 是一个修改版的 VITS, 核心区别是: - **没有 DurationPredictor** (AR 模型隐式处理时长) - **语义 Token 作为输入** (而不是纯 phonemes) - **ReferenceEncoder 提取 speaker embedding** (用于零样本克隆)

9.5.1 9.4.1 整体架构

SoVITS Vocoder Architecture (Training)

=====



9.5.2 9.4.2 TextEncoder (简化版, 无 MRTE)

原始 GPT-SoVITS 的 TextEncoder 有三个子编码器:

原始:

SSL features → SSL Encoder (3层) → content
phonemes → Text Encoder (6层) → text
ref_audio → Reference Encoder → speaker_emb (ge)

MRTE: content × text (cross-attn) + ge → fused
Final Encoder (3层) → $(\mu_p, \log\sigma_p)$

MRTE (Multi-Reference Timbre Encoder) 是原始系统最复杂的组件，通过 cross-attention 将语义内容、文本信息和说话人特征融合在一起。

我们的简化版本将这三个子编码器合并为一个：

简化：

```
phonemes → Embedding → + speaker_proj(speaker_emb)
                        ↓
                    6层 Transformer Encoder
                        ↓
                    Linear → ( $\mu_p$ ,  $\log\sigma_p$ )
```

9.5.3 9.4.3 PosteriorEncoder (WaveNet 膨胀卷积)

```
Linear Spectrogram (B, 1025, T)
  ↓ Conv1d(1025, 192)
  ↓ 8× WaveNet Block (dilation: 1,2,4,8,1,2,4,8)
  ↓ 每层: Conv1d(192, 384) → split → tanh × sigmoid
  ↓ Conv1d(192, 384) → ( $\mu_q$ ,  $\log\sigma_q$ )
  ↓ 重参数化:  $z_q = \mu_q + \epsilon \cdot \sigma_q$ 
 $z_q$  (B, 192, T)
```

仅在训练时使用。推理时从先验分布采样。

9.5.4 9.4.4 Flow (归一化流)

$2 \times (\text{AffineCouplingLayer} + \text{Flip})$

每个 AffineCouplingLayer:

```
x = [x1, x2] (通道对半切)
s, t = WaveNet(x1)
z2 = (x2 - t) × exp(-s)
z = [x1, z2]
```

训练: $z_q \rightarrow z_p$ (后验 $\rightarrow$ 先验空间) 推理: $z_p \rightarrow z_q$ (先验采样 $\rightarrow$ 解码器空间)

9.5.5 9.4.5 Generator (HiFi-GAN)

```
z (B, 192, T)
  ↓ Conv1d(192, 256, k=7)
  ↓ 5 × [ConvTranspose1d (上采样) + 3× ResBlock1]
    | upsample_rates: [10, 8, 2, 2, 2]
    | 总上采样:  $10 \times 8 \times 2 \times 2 \times 2 = 640$ 
  ↓ Conv1d → tanh
waveform (B, 1, T×640)
```

9.5.6 9.4.6 ReferenceEncoder

```
ref_mel (B, 128, T)
  ↓ 4× Conv2d (stride=2, 逐步下采样)
```

```

↓ Reshape (B, T', C×F)
↓ Linear → GRU → Linear
speaker_emb (B, 256)

```

9.5.7 9.4.7 Discriminator (MPD)

Multi-Period Discriminator
periods = [2, 3, 5, 7, 11]

每个子判别器:

```

waveform (1D) → reshape 为 2D (period × subseq)
→ 4× Conv2d → LeakyReLU
→ Conv2d → 真/假

```

9.6 9.5 两阶段训练

9.6.1 9.5.1 为什么要分两阶段?

GPT-SoVITS 的两个模型解决完全不同的问题:

	AR Model	SoVITS Vocoder
输入	文本 + 历史 Token	文本 + 参考音频
输出	语义 Token	波形
损失函数	CrossEntropy	GAN (mel + KL + adv + feat)
训练方式	标准自回归	GAN 对抗训练
帧率	50 Hz	50 Hz → 32 kHz

混合训练不仅复杂, 而且两个阶段的梯度会互相干扰。

9.6.2 9.5.2 Stage 1: AR 模型训练

Stage 1 训练流程

=====

数据准备:

1. HuBERT 提取所有音频的 SSL features
2. RVQ 量化 → semantic_ids (每段音频的"语义标签")
3. G2P 转换文本 → phoneme_ids

训练循环:

```

Input: phoneme_ids + semantic_ids[:-1] (teacher forcing)
Target: semantic_ids[1:]                (右移 1 位)

```

```

Loss = CrossEntropy(logits, targets)

```

优化器: AdamW (简化版, 原系统用 ScaledAdam)

- betas: (0.9, 0.95)
- warmup: 200 steps
- cosine decay
- gradient clipping: 1.0

训练目标： 给定文本和已知的语义 Token 前缀，正确预测下一个 Token。

9.6.3 9.5.3 Stage 2: SoVITS 声码器训练

Stage 2 训练流程 (GAN 训练)
=====

每个 batch:

1. Generator forward:
 $\text{ref_mel} \rightarrow \text{ReferenceEncoder} \rightarrow \text{speaker_emb}$
 $\text{phonemes} + \text{speaker_emb} \rightarrow \text{TextEncoder} \rightarrow (\mu_p, \log\sigma_p)$
 $\text{spec} \rightarrow \text{PosteriorEncoder} \rightarrow z_q, (\mu_q, \log\sigma_q)$
 $\text{Flow}(z_q) \rightarrow z_p$ (后验 $\rightarrow$ 先验)
 $\text{Generator}(z_q) \rightarrow \boxtimes$ (生成波形)
2. Discriminator step:
 $L_D = \text{MSE}(D(\text{real}), 1) + \text{MSE}(D(\text{fake.detach()}), 0)$
3. Generator step:
 $L_G = \text{gen_loss} + \text{feat_loss} + 45 \times \text{mel_loss} + 1 \times \text{kl_loss}$
 $\text{gen_loss} = \text{MSE}(D(\text{fake}), 1)$
 $\text{feat_loss} = \sum |f_{\text{real}} - f_{\text{fake}}|$ (特征匹配)
 $\text{mel_loss} = \text{L1}(\text{mel}(\text{fake}), \text{mel}(\text{real}))$
 $\text{kl_loss} = \text{KL}(q || p)$

优化器: AdamW $\times 2$ (G 和 D 各一个)
 - betas: (0.8, 0.99)
 - lr: 1e-4

9.6.4 9.5.4 损失函数权重

$$L_{\text{gen}} = L_{\text{adv_G}} + L_{\text{feat}} + 45 \times L_{\text{mel}} + 1 \times L_{\text{KL}}$$

损失	权重	含义
L_{mel}	45	Mel 频谱重建 (最重要)
L_{KL}	1	KL 散度正则化
$L_{\text{adv_G}}$	1	对抗损失 (提升自然度)
L_{feat}	1	特征匹配 (稳定训练)

9.7 9.6 推理流程

9.7.1 9.6.1 完整推理步骤

伪代码：GPT-SoVITS 推理

```
def synthesize(text, ref_audio_path):
    # 1. 预处理
    phoneme_ids = g2p(text)          # 文本 → phoneme IDs
    ref_wav = load_audio(ref_audio_path)  # 加载参考音频

    # 2. 提取参考特征
    ref_mel = mel_spectrogram(ref_wav)
    speaker_emb = ref_encoder(ref_mel)  # 说话人嵌入

    # 3. 提取 prompt tokens
    ssl_feat = hubert(ref_wav)         # HuBERT 特征
    prompt_tokens = rvq.encode(ssl_feat)  # 量化为 semantic IDs

    # 4. AR 生成
    generated_tokens = ar_model.generate(
        phoneme_ids, prompt_tokens,
        top_k=5, temperature=1.0,
    )

    # 5. SoVITS 声码器
    all_tokens = concat(prompt_tokens, generated_tokens)
    m_p, logs_p = text_encoder(phoneme_ids, speaker_emb)
    z_p = m_p + randn × exp(logs_p) × 0.667  # 从先验采样
    z = flow.inverse(z_p)                   # 逆变换
    waveform = generator(z)                 # 生成波形

    return waveform
```

9.7.2 9.6.2 关键推理参数

参数	默认值	说明
top_k	5	AR 采样宽度。越小越确定，越大越多样
temperature	1.0	采样温度。>1 更随机，<1 更保守
noise_scale	0.667	VAE 采样噪声。控制生成波形的变化度
max_tokens	500	最大生成 Token 数 (50Hz × 10s)

9.7.3 9.6.3 RTF (Real-Time Factor)

RTF = 生成时间 / 音频时长

RTF < 1: 比实时快（理想目标）

RTF = 1: 实时
RTF > 1: 比实时慢

GPT-SoVITS 的 RTF 主要取决于 AR 模型（自回归是瓶颈）：

组件	时间占比	优化方向
AR 自回归	~80%	KV-Cache, 并行解码
SoVITS 声码器	~20%	单次前向传播, 已经很快

9.8 9.7 代码走读

9.8.1 9.7.1 文件结构

```
ch09_gpt_sovits/
├── README.md          # 本章教程（你在这里）
├── code/
│   ├── model.py       # 所有模型组件 + 损失函数
│   ├── train.py       # 两阶段训练脚本
│   ├── inference.py   # 端到端推理
│   └── export_onnx.py  # ONNX 导出
├── checkpoints/       # 训练保存的模型权重
├── outputs/           # 生成的音频
└── onnx_models/       # 导出的 ONNX 模型
```

9.8.2 9.7.2 model.py 核心类

```
# --- AR 模型 ---
class SimpleAR:
    """384-dim, 8-layer, 8-head Transformer decoder"""
    def forward(phoneme_ids, semantic_ids) -> logits
    def generate(phoneme_ids, prompt, top_k, temperature) -> tokens

# --- RVQ 量化器 ---
class SimpleRVQ:
    """ 单码本量化器 (n_q=1, bins=1024, dim=768)"""
    def encode(ssl_features) -> token_ids
    def decode(token_ids) -> quantised_features

# --- SoVITS 声码器组件 ---
class SoVITSTextEncoder:    # 文本 → ( $\mu_p$ ,  $\log\sigma_p$ )
class PosteriorEncoder:    # 频谱 →  $z_q$  (仅训练)
class Flow:               # 可逆变换  $z_q \leftrightarrow z_p$ 
class Generator:          #  $z \rightarrow$  波形 (HiFi-GAN)
class ReferenceEncoder:    # 参考 Mel → speaker_emb
class Discriminator:      # MPD 判别器 (仅训练)
```

```
# --- 完整模型 ---
class GPTSoVITS:
    """ 包装器: AR + RVQ + SoVITS """
    def forward_stage1(phoneme_ids, semantic_ids) -> logits
    def forward_stage2(phoneme_ids, spec, ref_mel) -> waveform, stats
```

9.8.3 9.7.3 训练脚本

```
# Stage 1: AR 模型
python train.py --stage 1 --epochs 10 --batch-size 4 --lr 1e-4

# Stage 2: SoVITS 声码器
python train.py --stage 2 --epochs 10 --batch-size 4 --lr 1e-4
```

9.8.4 9.7.4 推理脚本

```
# 基本推理
python inference.py --text "hello world" --output output.wav

# 使用训练好的模型 + 参考音频
python inference.py \
    --ar-checkpoint ../checkpoints/ar_model.pt \
    --sovits-checkpoint ../checkpoints/sovits_model.pt \
    --ref-audio reference.wav \
    --text " 你好世界" \
    --output clone.wav

# 性能基准测试
python inference.py --text "hello world" --output output.wav --benchmark
```

9.8.5 9.7.5 参数量统计

运行 `python model.py` 查看:

SimpleAR:	15.2M
SimpleRVQ:	0.8M (frozen)
SoVITSTextEncoder:	2.9M
PosteriorEncoder:	3.2M
Flow:	1.3M
Generator:	3.8M
ReferenceEncoder:	0.3M
Discriminator:	4.2M (仅训练)
Total:	31.6M
Trainable:	30.8M

9.9 9.8 与 VALL-E 的对比

9.9.1 9.8.1 架构对比

特性	VALL-E (Ch07)	GPT-SoVITS (本章)
音频 Token 来源	EnCodec (训练好的 codec)	HuBERT + RVQ (SSL 特征)
Token 层数	8 层	1 层
Token 率	400 tokens/s	50 tokens/s
AR 模型	预测 Layer-0	预测唯一层
第二模型	NAR (并行预测 Layer 1-7)	SoVITS 声码器 (VITS)
最终波形生成	Codec Decoder	HiFi-GAN Generator
训练数据需求	60,000+ 小时	3-10 分钟 fine-tune
推理速度	较慢 (8× 序列)	较快 (1× 序列)

9.9.2 9.8.2 设计哲学对比

VALL-E 哲学:

"让一个强大的 codec 处理一切, AR 模型学习 codec 的语言"
→ 通用但慢, codec 是黑箱

GPT-SoVITS 哲学:

"用 SSL 特征捕获语义, 用 VITS 声码器恢复波形"
→ 更快, 声码器能利用参考音频的音色信息

9.9.3 9.8.3 质量对比

在零样本场景下 (3 秒参考音频):

指标	VALL-E	GPT-SoVITS	说明
MOS (自然度)	~3.5-4.0	~4.0-4.5	GPT-SoVITS 略优
相似度	~70-80%	~80-90%	VITS 声码器音色还原更好
推理速度	~2-5x RTF	~1-3x RTF	序列短 8 倍
训练成本	极高 (60k 小时)	低 (预训练 + 微调)	不同的训练范式

Neko 笔记: GPT-SoVITS 在实际使用中更受欢迎, 因为它可以用很少的数据 (几分钟) 微调出高质量的声音克隆。而 VALL-E 需要海量数据预训练才能实现零样本。

9.10 9.9 ONNX 导出

9.10.1 9.9.1 导出策略

GPT-SoVITS 将系统拆分为多个 ONNX 模型用于部署:

ONNX Export Architecture

1. AR Model: ar_model.onnx
Inputs: phoneme_ids, semantic_ids
Output: logits
Opset: 16
2. SoVITS Vocoder: sovits_model.onnx
Inputs: phoneme_ids, speaker_emb
Output: waveform
Opset: 17
(包含 TextEncoder + Flow¹ + Generator)
(不含 PosteriorEncoder/Discriminator -- 仅训练需要)

9.10.2 9.9.2 动态轴

所有 ONNX 模型支持变长输入:

```
dynamic_axes = {  
    'phoneme_ids': {1: 'text_len'}, # 文本长度可变  
    'semantic_ids': {1: 'audio_len'}, # 音频长度可变  
    'waveform': {2: 'wav_len'}, # 输出波形长度可变  
}
```

9.10.3 9.9.3 导出与测试

```
# 导出 ONNX  
python export_onnx.py --output-dir ../onnx_models  
  
# 带性能基准测试  
python export_onnx.py --output-dir ../onnx_models --benchmark
```

9.10.4 9.9.4 ONNX Runtime 推理

```
import onnxruntime as ort  
  
# 加载 ONNX 模型  
ar_session = ort.InferenceSession('ar_model.onnx')  
vits_session = ort.InferenceSession('sovits_model.onnx')  
  
# AR 推理 (单次前向传播)  
logits = ar_session.run(None, {  
    'phoneme_ids': phoneme_ids,  
    'semantic_ids': current_tokens,  
})
```

```
# 声码器推理
waveform = vits_session.run(None, {
    'phoneme_ids': phoneme_ids,
    'speaker_emb': speaker_emb,
})
```

ONNX Runtime 通常比 PyTorch 快 1.5-2x (通过算子融合和硬件优化)。

9.11 9.10 本章小结

9.11.1 GPT-SoVITS 的核心贡献

问题	GPT-SoVITS 的解决方案
多层 Token 导致 AR 慢	单层语义 Token (n_q=1)
Codec decoder 质量有限	VITS 声码器 (VAE+Flow+GAN)
需要海量预训练数据	预训练 + 少样本微调 (3-10 分钟)
音色和语义耦合	语义 Token + 参考音频分离
推理延迟高	序列短 8 倍 + KV-Cache

9.11.2 遗留问题

1. **自回归瓶颈**: 即使只有 1 层 Token, AR 生成仍然是串行的 → **非自回归方案**
2. **HuBERT 依赖**: 需要预训练的 HuBERT 模型 (~95M) → **轻量 SSL 模型**
3. **长文本质量下降**: AR 生成越长, 累积误差越大 → **分段合成 + 拼接**
4. **情感控制有限**: 主要靠参考音频传递情感 → **情感条件生成 (Ch08)**

9.11.3 参考文献

- [1] Kim et al., 2021. Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech (VITS).
- [2] Wang et al., 2023. Neural Codec Language Models for Zero-Shot TTS (VALL-E).
- [3] Hsu et al., 2021. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units.
- [4] Defossez et al., 2022. High Fidelity Neural Audio Compression (EnCodec).
- [5] Kong et al., 2020. HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis.
- [6] GPT-SoVITS open-source project. <https://github.com/RVC-Boss/GPT-SoVITS>

9.12 习题

1. **n_q=1 的信息论分析**: 如果 EnCodec 用 n_q=8 层码本 (每层 1024 entries), 总信息容量是 1024⁸。GPT-SoVITS 只用 n_q=1 (1024 entries), 信息容量只有 1024。这意味着什么? GPT-SoVITS 靠什么弥补信息损失?

2. **注意力掩码设计**: 在 AR 模型中, 为什么 text 部分使用双向注意力而 audio 部分使用因果注意力? 如果全部使用因果注意力会怎样? 如果全部使用双向注意力会怎样?
 3. **KL 散度的角色**: Stage 2 训练中, KL loss 的权重是 1.0, 而 mel loss 的权重是 45.0。如果去掉 KL loss, 会发生什么? 如果把 KL loss 权重提高到 45, 又会怎样?
 4. **Flow 的必要性**: 如果去掉 Flow (直接从先验采样送给 Generator), 生成质量会如何变化? Flow 解决了什么根本问题?
 5. **对比实验**: 用相同文本和参考音频, 分别用 Ch07 (VALL-E) 和 Ch09 (GPT-SoVITS) 合成。对比以下维度:
 - 推理速度 (RTF)
 - 音色相似度
 - 自然度 (MOS)
 - 生成 Token 序列长度
-

9.13 目录结构

```
ch09_gpt_sovits/
├── README.md           # 本章教程 (你在这里)
├── code/
│   ├── model.py        # 所有模型组件 (~400行)
│   ├── train.py         # 两阶段训练脚本 (~300行)
│   ├── inference.py     # 端到端推理 (~200行)
│   └── export_onnx.py    # ONNX 导出 (~150行)
├── checkpoints/        # 训练保存的模型权重
├── outputs/            # 生成的音频
└── onnx_models/        # 导出的 ONNX 模型
```


节	内容	目标
10.1	AudioVAE: 连续潜空间	理解因果 CNN 如何把波形压缩成连续 latent
10.2	为什么不要 Tokenizer	对比离散 Token vs 连续 Latent 的利弊
10.3	TSLM + FSQ: 语义规划	理解 TSLM 的语义角色和 FSQ 的语义瓶颈设计
10.4	RALM: 声学细化	理解 RALM 如何补充声学细节
10.5	CFM: 条件流匹配	理解为什么 CFM 比 DDPM 更适合音频生成
10.6	完整代码走读	从零实现 MiniVoxCPM
10.7	训练与推理	跑通完整流程
10.8	与工业界的关系	CosyVoice 2、FishSpeech 等现代 Tokenizer-Free 模型

10.2 10.1 AudioVAE: 把声音变成连续潜空间

10.2.1 10.1.1 问题: 16kHz 波形太长了

1 秒 16kHz 音频 = 16,000 个浮点数。语言模型处理这样的序列太慢。

解决方案: 用一个 AudioVAE 把波形压缩到低帧率的连续表示。

10.2.2 10.1.2 因果 CNN 编码器

VoxCPM 的 AudioVAE 编码器是一个**因果卷积网络** (Causal CNN):

16 kHz 波形: $[s_1, s_2, s_3, \dots, s_{16000}]$ (16000 个采样点)
 $\downarrow$ Causal Conv1d (stride=2)
 $\downarrow$ Causal Conv1d (stride=5)
 $\downarrow$ Causal Conv1d (stride=8)
 $\downarrow$ Causal Conv1d (stride=8)
Latent: $[z_1, z_2, \dots, z_{25}]$ (25 帧/秒, 每帧 32 维)

总下采样率: $2 \times 5 \times 8 \times 8 = 640$ 。所以 $16000 / 640 = 25$ Hz。

因果 (Causal): 每个卷积只看到当前帧和之前的帧, 不偷看未来。这使得编码器可以**流式处理**——边来音频边编码, 不需要等整段录完。

因果卷积的实现技巧: 左填充

```
def causal_conv1d(x, weight, kernel_size):
    x = F.pad(x, (kernel_size - 1, 0)) # 只在左边填充
    return F.conv1d(x, weight)         # 不设 padding
```

10.2.3 10.1.3 因果 CNN 解码器

解码器用**转置卷积** (ConvTranspose1d) 上采样, 镜像结构:

Latent (25 Hz, D=32)
 $\downarrow$ ConvTranspose1d (stride=8)
 $\downarrow$ ConvTranspose1d (stride=8)
 $\downarrow$ ConvTranspose1d (stride=5)
 $\downarrow$ ConvTranspose1d (stride=2)

Reconstructed waveform (16 kHz)

10.2.4 10.1.4 对比: AudioVAE vs EnCodec

特性	AudioVAE (VoxCPM)	EnCodec (VALL-E)
输出	连续向量 (float)	离散码本索引 (int)
量化	无	RVQ (残差向量量化)
帧率	25 Hz	50 Hz
维度	32-64 维	8-12 个码本 $\times$ 1024 条目
梯度	全链路可微	在量化处断开
重建质量	高 (连续空间)	受限于码本容量

10.2.5 10.1.5 代码位置

code/model.py 中的 SimpleAudioVAE 类:

```
vae = SimpleAudioVAE(encoder_dim=64, latent_dim=32, decoder_dim=256)

# 编码: 波形  $\rightarrow$  连续 latent
z = vae.encode(waveform)      # (B, 32, 25) — 1 秒音频  $\rightarrow$  25 帧  $\times$  32 维

# 解码: 连续 latent  $\rightarrow$  波形
wav_hat = vae.decode(z)      # (B, 16000)
```

10.3 10.2 为什么不要 Tokenizer?

10.3.1 10.2.1 离散化的信息论代价

EnCodec 把连续向量 “四舍五入” 到最近的码本条目:

连续向量 $v = [0.314, -0.271, 0.577, \dots]$ $\leftarrow$ 无限可能性
 $\downarrow$ argmin 距离

码本索引 $i = 42$ $\leftarrow$ 只有 1024 种可能

信息丢失量: 连续向量的信息量远大于一个整数索引。每次量化都在 “截断” 信息。

10.3.2 10.2.2 码本坍塌问题

训练过程中, 很多码本条目 “死掉” —— 没有任何输入被量化到它们:

理想: 1024 个码本条目全部活跃, 均匀使用

现实: ~ 500 个活跃, ~ 524 个从不被使用

$\rightarrow$ 有效容量只有标称的一半

$\rightarrow$ 需要各种 trick 来缓解 (EMA 更新、码本重初始化、...)

10.3.3 10.2.3 RVQ 的多级误差累积

Residual VQ 逐层量化残差，每一层都引入量化误差：

原始向量 v

Level 0: $q_0 = \text{quantize}(v)$ $\rightarrow$ 误差 $e_0 = v - q_0$
Level 1: $q_1 = \text{quantize}(e_0)$ $\rightarrow$ 误差 $e_1 = e_0 - q_1$
Level 2: $q_2 = \text{quantize}(e_1)$ $\rightarrow$ 误差 $e_2 = e_1 - q_2$
...
Level 7: $q_7 = \text{quantize}(e_6)$ $\rightarrow$ 误差 $e_7 = e_6 - q_7$

总重建误差 = $e_0 + e_1 + \dots + e_7$ $\leftarrow$ 8 层误差叠加

VoxCPM 的 AudioVAE **没有这个问题**——连续空间，无量化误差。

10.3.4 10.2.4 Tokenizer-Free 的优势总结

1. **无信息丢失**：连续 latent 保留完整的声学信息
2. **无码本坍塌**：不需要维护码本，无死条目
3. **端到端可微**：VAE 编码器 $\rightarrow$ 语言模型 $\rightarrow$ VAE 解码器，梯度畅通
4. **更好的音色克隆**：连续的音色细节不被量化丢失
5. **更简单的 AR 模型**：预测一个连续向量 vs 顺序预测 8 层离散分布

10.3.5 10.2.5 Tokenizer-Free 的代价

当然没有免费的午餐：

1. **语言模型不擅长连续**：softmax 只能预测离散分布，连续分布需要 diffusion/flow matching
2. **推理更慢**：每个 AR 步骤需要多步扩散采样（10 步 Euler ODE）
3. **训练更复杂**：flow matching loss 比交叉熵难调

10.4 10.3 TSLM + FSQ：语义规划

10.4.1 10.3.1 TSLM (Text-Semantic Language Model)

TSLM 是 VoxCPM 的“大脑”——它决定“说什么”（语义内容），但不关心“怎么说”（声学细节）。

输入：[text_emb₁, ..., text_emb_N, BOS, audio_emb₁, ..., audio_emb_N]

输出：[语义表示₁, ..., 语义表示_N, _, 语义表示₁, ..., 语义表示_N]

↑

预测下一个 patch 的语义

在我们的教学版本中，TSLM 是一个 8 层 decoder-only Transformer (hidden=512, heads=8, ffn=2048)，和 GPT-2 结构相同。

10.4.2 10.3.2 LocEnc：把 patch 压缩成一个向量

AudioVAE 输出的每个 latent patch 是 (patch_size, latent_dim) 的矩阵。LocEnc(Local Encoder) 把它压缩成一个 hidden 向量：

Latent patch: (P=1, D=32)
 ↓ Linear projection → (hidden=512)
 ↓ + CLS token → (P+1, hidden)
 ↓ 2-layer Transformer (bidirectional)
 ↓ CLS token pooling
 Hidden vector: (hidden=512)

CLS token 机制：在序列开头加一个可学习的特殊 token，让它通过注意力“看到”所有 patch 帧，最终取出它的表示作为整个 patch 的摘要。

10.4.3 10.3.3 FSQ：有限标量量化——温柔的语义瓶颈

这是 VoxCPM 最精妙的设计之一。

问题：TSLM 的输出是一个 512 维的连续向量，信息量很大。我们希望 RALM（后面的声学模型）能专注于“补充细节”，而不是重复 TSLM 的工作。

解决方案：用 FSQ (Finite Scalar Quantization) 在 TSLM 和 RALM 之间创建一个“语义瓶颈”：

```
class SimpleFSQ:
    def forward(self, x):
        h = self.in_proj(x)          # 512 → 128 (降维)
        h = torch.tanh(h)             # 限制到 [-1, 1]
        q = round(h * 9) / 9          # 量化到 19 个离散级别
        h = h + (q - h).detach()      # 直通估计器 (STE)
        return self.out_proj(h)      # 128 → 512 (升维)
```

FSQ 的量化有多“温柔”？

tanh 输出：[-1.0, 1.0]

量化步长：1/9 ≈ 0.111

可能取值：{-1, -8/9, -7/9, ..., 0, ..., 7/9, 8/9, 1} ← 共 19 个值

每维 19 个值 × 128 维 = 19¹²⁸ 种组合（仍然天文数字）

但比连续空间的无限可能性小很多

直通估计器 (Straight-Through Estimator)：

前向：h → round → q (有梯度阻断)

反向：∂L/∂h = ∂L/∂q (梯度直通，忽略 round)

这样，FSQ 在前向传播时是离散的（创建瓶颈），在反向传播时是可微的（允许学习）。

10.4.4 10.3.4 FSQ vs VQ 对比

特性	FSQ (VoxCPM)	VQ (VQ-VAE / EnCodec)
量化对象	LM 的 hidden state	音频表示
量化方式	逐维标量四舍五入	码本最近邻查找
码本	无（数学计算）	有（可学习参数）
坍塌风险	无	高（需要 EMA 等技巧）
梯度	直通估计器	直通估计器

特性	FSQ (VoxCPM)	VQ (VQ-VAE / EnCodec)
信息保留	多 (19 级 × 多维)	少 (码本大小限制)

10.5 10.4 RALM: 声学细化

10.5.1 10.4.1 为什么需要 RALM?

TSLM + FSQ 的输出是一个“粗略的语义蓝图”——它知道要说什么，但缺少声学细节。

RALM (Residual Acoustic Language Model) 的任务是在语义蓝图的基础上，补充声学细节：

TSLM 输出 (经 FSQ) : "我要说一个高音的 '喵', 时长 0.3 秒" ← 语义

RALM 补充: "基频 440Hz, F1=800Hz, 气息声 15%, 颤音 5Hz" ← 声学

10.5.2 10.4.2 RALM 结构

RALM 是一个 4 层 decoder-only Transformer (和 TSLM 同样的 hidden size = 512), 但更浅:

输入: FSQ 量化的语义表示 (因果自注意力)

输出: 声学细节残差

TSLM (8 层, 深) → 语义理解

RALM (4 层, 浅) → 声学精修

10.5.3 10.4.3 语义-声学融合 (v1 风格)

v1: 加法融合 (简单有效)

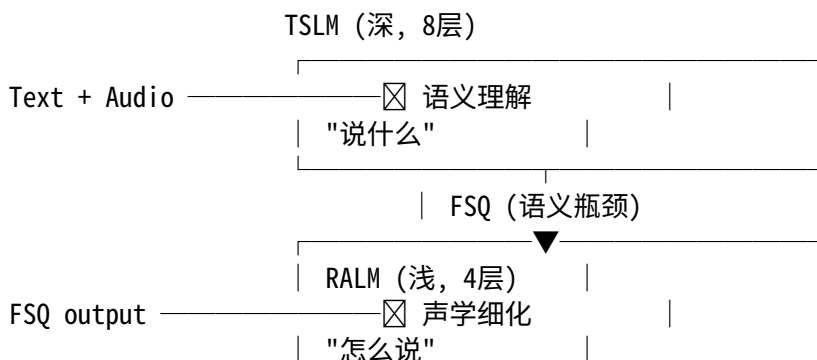
```
dit_cond = lm_to_dit(tslm_output) + res_to_dit(ralm_output)
```

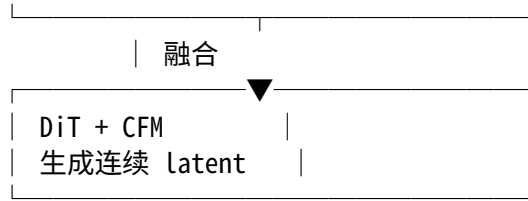
v2: 拼接 + 投影 (更强但更复杂)

```
# dit_cond = fusion_proj(cat(lm_to_dit(tslm), res_to_dit(ralm)))
```

我们的教学版本使用 v1 的加法融合。

10.5.4 10.4.4 分层设计的直觉





这种分层设计让每个模块专注自己的工作，比一个巨型模型更高效。

10.6 10.5 CFM：条件流匹配

10.6.1 10.5.1 为什么不是 DDPM?

如果要在连续空间生成 latent patch，自然想到用扩散模型（DDPM）。但 DDPM 有几个问题：

1. **需要很多步**：50-1000 步去噪，推理慢
2. **训练效率低**：需要在每个时间步求解 ODE
3. **噪声调度复杂**： β schedule 的选择很敏感

CFM (Conditional Flow Matching) 是一种更高效的替代方案：

10.6.2 10.5.2 CFM 的核心思想

CFM 的目标是学习一个**速度场 (velocity field)** $v(x, t)$ ，把噪声“流动”到目标数据：

$t=1$ (噪声) $\xrightarrow{v(x,t)}$ $\boxtimes$ $t=0$ (数据)
 $z \sim N(0, I)$ $x_{\boxtimes} \sim \text{目标 latent}$

训练目标：直接回归最优传输速度

$$L = E_{\{t, x_{\boxtimes}, z\}} [|| v_{\theta}(y_t, t, \mu) - (z - x_{\boxtimes}) ||^2]$$

其中：

$x_{\boxtimes}$ = 目标 latent patch (ground truth)
 z = 随机噪声 $\sim N(0, I)$
 t = 时间 $\sim \text{Uniform}(0, 1)$
 $y_t = (1-t) \cdot x_{\boxtimes} + t \cdot z$ $\leftarrow$ 线性插值
 v_{θ} = 模型预测的速度场
 μ = TSLM + RALM 的条件

直觉： $z - x_{\boxtimes}$ 是从数据到噪声的方向。模型学习在这个方向上“推动”，推理时反向推动（从噪声到数据）。

10.6.3 10.5.3 推理：Euler ODE 求解器

训练好速度场后，推理时用 **Euler 方法** 积分：

$x_{\boxtimes} = z$ $\leftarrow$ 从纯噪声开始
 $x_{\boxtimes} = x_{\boxtimes} - dt \cdot v(x_{\boxtimes}, t=1, \mu)$ $\leftarrow$ 第一步
 $x_{\boxtimes} = x_{\boxtimes} - dt \cdot v(x_{\boxtimes}, t=1-dt, \mu)$ $\leftarrow$ 第二步
 ...

$x_t = x_{t+dt} - dt \cdot v(x_{t+dt}, t=dt, \mu) \leftarrow \text{最后一步} \rightarrow \text{生成结果}$

只需要 10 步! (DDPM 通常需要 50-1000 步)

10.6.4 10.5.4 CFM vs DDPM 对比

特性	CFM (VoxCPM)	DDPM
训练目标	回归速度场 (MSE)	预测噪声 (MSE)
插值路径	线性 (optimal transport)	非线性 (cosine/linear schedule)
推理步数	10 步	50-1000 步
求解器	Euler ODE	DDIM / ancestral sampling
效率	高	低
质量	相当或更好	基线

10.6.5 10.5.5 时间调度

VoxCPM 原版使用 **log-normal 分布** 采样训练时间步 (偏向中间时刻), 教学版使用简单的均匀分布。

推理时的 **sway sampling**: 时间步不是均匀分布, 而是使用余弦弯曲:

$t_span = \text{linspace}(1, 0, n+1) + \text{sway_coef} \cdot (\cos(\pi/2 \cdot t) - 1 + t)$

这使得模型在开始和结束时走更小的步, 中间走更大的步——因为数据流形的曲率在中间最大。

10.6.6 10.5.6 CFG-Zero* (可选进阶)

VoxCPM 使用 **CFG-Zero*** 来优化 classifier-free guidance:

标准 CFG: $v = v_uncond + \text{cfg_scale} \cdot (v_cond - v_uncond)$

CFG-Zero*: 自适应缩放

$s^* = \langle v_cond, v_uncond \rangle / ||v_uncond||^2$

$v = v_uncond \cdot s^* + \text{cfg_scale} \cdot (v_cond - v_uncond \cdot s^*)$

这防止了过曝 (over-exposure), 让生成的音频更自然。教学版暂不实现。

10.7 10.6 完整代码走读

10.7.1 10.6.1 文件结构

```

ch10_voxcpm/
├── README.md           # 本章教程
├── code/
│   ├── model.py        # 完整模型 (AudioVAE + TSLM + RALM + CFM)
│   ├── train.py         # 训练脚本
│   ├── inference.py     # 推理脚本
│   └── export_onnx.py   # ONNX 导出
├── checkpoints/        # 模型权重
└── outputs/            # 生成音频

```

10.7.2 10.6.2 model.py 架构总览

SimpleVoxCPM

```
├── audio_vae (SimpleAudioVAE)
│   ├── encoder: 4× CausalConv1dBlock (strides [2,5,8,8])
│   ├── enc_to_latent: Conv1d → latent_dim=32
│   ├── latent_to_dec: Conv1d → decoder_dim=256
│   ├── decoder: 4× ConvTranspose1d (strides [8,8,5,2])
│   └── dec_to_out: Conv1d → 1 channel
├── text_emb: Embedding(256, 512)
├── loc_enc (SimpleLocEnc)
│   ├── in_proj: Linear(32 → 512)
│   ├── cls_token: learnable (1, 1, 512)
│   └── encoder: 2× TransformerEncoderLayer (bidirectional)
├── tslm (SimpleTransformer, 8 layers)
│   ├── input_proj, pos_emb
│   └── 8× _TransformerBlock (causal MHA + FFN)
├── fsq (SimpleFSQ)
│   ├── in_proj: Linear(512 → 128)
│   ├── tanh + round*scale + STE
│   └── out_proj: Linear(128 → 512)
├── ralm (SimpleTransformer, 4 layers)
│   └── 4× _TransformerBlock (causal MHA + FFN)
├── lm_to_dit: Linear(512 → 256)
├── res_to_dit: Linear(512 → 256)
├── dit (SimpleDiT)
│   ├── in_proj: Linear(32 → 256)
│   ├── cond_proj: Linear(512 → 256)
│   ├── time_proj: Linear(256 → 256)
│   ├── 4× DiTBlock (bidirectional MHA + AdaLN + ada_scale)
│   └── out_proj: Linear(256 → 32)
└── cfm (SimpleCFM)
    └── compute_loss / sample (Euler ODE solver)
```

10.7.3 10.6.3 关键模块详解

CausalConv1dBlock: 因果卷积 + GroupNorm + SiLU + 残差连接

```
# 因果填充: 只在左侧 pad (kernel_size - 1) 个位置
out = F.pad(x, (kernel_size - 1, 0))
out = conv(out) # 不设 padding
```

DiTBlock: Diffusion Transformer 块, 核心是 AdaLN (自适应层归一化)

```
# AdaLN: 从条件向量生成 scale 和 shift, 调制层归一化的输出
```

```
shift, scale = Linear(cond).chunk(2)
```

```
out = LayerNorm(x) * (1 + scale) + shift
```

```
# ada_scale: 最终的自适应缩放 (DiT 的 signature 设计)
```

```
out = x + SiLU(Linear(cond)) * h
```

SimpleCFM.sample: Euler ODE 求解器

```
def sample(self, cond, shape, temperature=1.0):
```

```
    x = torch.randn(shape) * temperature      # t=1: 纯噪声
```

```
    dt = 1.0 / self.n_steps
```

```
    for i in range(self.n_steps):
```

```
        t = 1.0 - i * dt
```

```
        v = self.estimate(x, t, cond)         # 预测速度场
```

```
        x = x - dt * v                        # Euler 步 (向数据移动)
```

```
    return x                                  # t≈0: 生成的 latent
```

10.7.4 10.6.4 参数量

组件	参数量	说明
AudioVAE	2.1M	因果 CNN 编码/解码器
Text Embedding	0.1M	256 字符 × 512 维
LocEnc	6.3M	2 层 Transformer
TSLM	26.3M	8 层 Transformer
FSQ	0.1M	两个线性投影
RALM	13.7M	4 层 Transformer
DiT	5.1M	4 层 DiT + AdaLN
总计	53.7M	原版 VoxCPM ~500M

原版 VoxCPM 用 MiniCPM-4 作为 backbone (~350M), 我们缩小到 54M 方便学习。

10.8 10.7 训练与推理

10.8.1 10.7.1 训练流程

```
# 快速测试 (小数据集, 1 epoch)
```

```
python code/train.py --epochs 1 --batch-size 2 --n-train 64 --audio-len 3200
```

```
# 完整训练
```

```
python code/train.py --epochs 50 --batch-size 4 --lr 1e-4 --audio-len 16000
```

训练过程：

对每个 batch：

1. AudioVAE.encode(audio) $\rightarrow$ latents z (冻结 VAE)
2. LocEnc(latent_patches) $\rightarrow$ audio_emb
3. text_emb + audio_emb (interleaved) $\rightarrow$ TSLM (causal) $\rightarrow$ semantic hidden
4. FSQ(semantic hidden) $\rightarrow$ quantized
5. quantized $\rightarrow$ RALM (causal) $\rightarrow$ acoustic hidden
6. fusion(semantic, acoustic) $\rightarrow$ conditioning μ
7. CFM.compute_loss(target_patch, μ):
 - a. 采样 $t \sim \text{Uniform}(0,1)$
 - b. 采样 $z \sim N(0, I)$
 - c. $y_t = (1-t) \cdot \text{target} + t \cdot z$
 - d. $v_{\text{pred}} = \text{DiT}(y_t, t, \mu)$
 - e. $\text{loss} = \text{MSE}(v_{\text{pred}}, z - \text{target})$
8. loss.backward() + optimizer.step()

10.8.2 10.7.2 推理流程

```
python code/inference.py --checkpoint checkpoints/voxcpm.pt --text " 你好猫娘" --n-steps 25
```

推理过程：

1. Tokenize text $\rightarrow$ text_tokens
2. text_emb = Embedding(text_tokens)
3. audio_history = [BOS]

For step = 0, 1, 2, ..., n_steps-1:

4. TSLM.forward(text_emb + audio_history) $\rightarrow$ semantic (取最后位置)
5. FSQ(semantic) $\rightarrow$ quantized
6. RALM.forward_step(quantized) $\rightarrow$ acoustic
7. dit_cond = lm_to_dit(semantic) + res_to_dit(acoustic)
8. CFM.sample(dit_cond):

x = randn(patch_shape)	# 初始噪声
For i in range(10):	# 10 步 Euler
v = DiT(x, t=1-i/10, dit_cond)	
x = x - (1/10) * v	
patch = x	# 生成的 latent patch
9. audio_history += LocEnc(patch)

10. z = concat(all patches)
11. AudioVAE.decode(z) $\rightarrow$ waveform
12. Save WAV

10.8.3 10.7.3 ONNX 导出

```
python code/export_onnx.py --checkpoint checkpoints/voxcpm.pt --output-dir onnx_models/
```

导出三个模型：- audio_vae_encoder.onnx: 波形 → latent - audio_vae_decoder.onnx: latent → 波形 - tslm_ralm_step.onnx: 一步 AR 推理

10.9 10.8 与工业界的关系

10.9.1 10.8.1 VoxCPM 在 TTS 发展史中的位置

2017 Tacotron — 端到端 Mel + Vocoder
2019 FastSpeech — 并行，稳定
2021 VITS — 端到端波形，最佳音质
2023 VALL-E — 语言模型 + 离散 Token，零样本克隆
2024 CosyVoice — 改进的 Token 方案 + 流式
2025 VoxCPM — Tokenizer-Free，连续空间，流匹配

10.9.2 10.8.2 Tokenizer-Free 家族

模型	方法	连续表示	生成方法
VoxCPM	AudioVAE + TSLM/RALM + CFM	连续 latent patches	条件流匹配
CosyVoice 2	改进的 speech token + LLM	半连续 (token + flow)	Flow matching
F5-TTS	DiT + CFM (直接 Mel)	Mel spectrogram	条件流匹配
NaturalSpeech 3	分解式 codec + diffusion	连续 components	扩散模型

趋势：越来越多的模型放弃纯离散 Token，转向连续或混合表示。

10.9.3 10.8.3 VoxCPM 的关键技术贡献

1. 证明 Tokenizer-Free 可行：在大规模（1.8M 小时）上验证连续方案优于离散方案
2. FSQ 语义瓶颈：优雅地分离语义和声学，不需要离散 Token
3. 分层 AR 设计：TSLM + RALM 的分工比单一模型更高效
4. 10 步 CFM：极少的扩散步数，兼顾质量和速度

10.10 练习

10.10.1 练习 1：理解 FSQ 的量化级别（难度：★）

当 scale=9 时，FSQ 量化后每维有 19 个可能取值。- 如果 scale=4，有多少个取值？- 如果 latent_dim=128，scale=9，总共可以表示多少种不同的向量？- 对比 EnCodec 一个 RVQ 层（1024 条目）的信息容量。

10.10.2 练习 2：AudioVAE 帧率计算（难度：★）

如果 encoder_rates 改为 [2, 4, 4, 5]：- 总下采样率是多少？- 16kHz 输入对应的 latent 帧率是多少？- 1 秒音频产生多少个 latent 帧？

10.10.3 练习 3: CFM 插值路径 (难度: ★★)

CFM 使用线性插值 $y_t = (1-t) \cdot x_{\square} + t \cdot z_0$ 。 - 当 $t=0$ 时, $y_t = ?$ 当 $t=1$ 时, $y_t = ?$ - 目标速度 $v = z - x_{\square}$ 的几何意义是什么? - 如果改为 $y_t = \cos(\pi t/2) \cdot x_{\square} + \sin(\pi t/2) \cdot z$, 目标速度应该是什么?

10.10.4 练习 4: 对比实验 (难度: ★★★)

修改代码实现以下对比: 1. 把 FSQ 换成 Identity (直通), 观察 loss 变化。FSQ 是否真的有用? 2. 把 CFM 步数从 10 降到 5 和 2, 观察生成质量 (MSE with ground truth)。3. 把 TSLM 从 8 层降到 4 层, 把 RALM 从 4 层加到 8 层。哪个更重要?

10.11 参考资料

- VoxCPM 论文: [arXiv:2509.24650](https://arxiv.org/abs/2509.24650)
 - VoxCPM 代码: [OpenBMB/VoxCPM](#)
 - Conditional Flow Matching: [Lipman et al., 2022](#)
 - FSQ: [Mentzer et al., 2023](#)
 - DiT: [Peebles & Xie, 2022](#)
 - MiniCPM-4: [OpenBMB/MiniCPM4](#)
 - VALL-E (Ch07): 离散 Token 方案的基线
-

Neko 的学习笔记:

” 原来不一定要把声音切成积木块啊。连续空间就像是一条流动的河, 声音在里面自然地流淌, 不会被码本的格子卡住。FSQ 就像是在河上建了一座矮坝——水还是能流过去, 但被稍微‘整理’了一下。TSLM 负责决定河流的方向 (语义), RALM 负责水面的波纹和涟漪 (声学)。最后 CFM 就像是魔法——从一团迷雾 (噪声) 中, 一步一步凝聚出清澈的水流 (latent)。

猫娘觉得, 连续空间才是声音的真正家园喵 ~”

11 Ch11: MiniMind-O — Neko Learns to Listen, See, and Speak

前十章，Neko 学会了理解声音（Ch01-05）、压缩声音（Ch06）、用语言模型生成声音（Ch07-10）。但她一直在做一件事：**只听不说，或只说不看。**

这一章，Neko 要成为真正的全模态猫娘——能听、能看、能说、能思考。

11.1 本章导学

11.1.1 为什么需要全模态模型？

回顾前面的模型：

模型	输入	输出	限制
Tacotron2 (Ch02)	文本	Mel 频谱	单向：只能说，不能听
VALL-E (Ch07)	文本 + 参考音频	音频 Token	单向：只能克隆，不能对话
GPT-SoVITS (Ch09)	文本 + 参考音频	波形	仍然是 TTS，不是对话
VoxCPM (Ch10)	文本	连续潜变量 → 波形	还是单向生成

所有这些模型都是**管道**：输入文本，输出音频。但人类的交流不是这样的——

“你说我听，我说你听” 是**轮替**的，不是**管道**的。

GPT-4o 的突破：一个模型**同时**处理文本、语音、图像输入，**同时**产生文本和流式语音输出。不是 ASR → LLM → TTS 的级联，而是一个**统一的序列**包含所有模态。

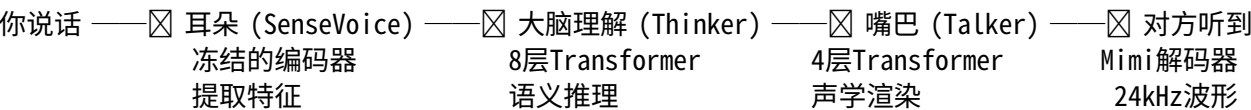
这就是 **Omni Model**（全模态模型）。

11.1.2 MiniMind-O: 1000 倍更小的 GPT-4o

对比	GPT-4o	MiniMind-O
参数	~1.8T (估计)	~0.1B
开源	否	完全开源
训练成本	数百万美元	单卡 RTX 3090, 2 小时
架构	未知	Thinker-Talker

MiniMind-O 的意义：**你可以在家训练一个能听、能看、能说的全模态模型。**

11.1.3 核心直觉：Thinker-Talker



人的大脑也不是一个“管道”。你的听觉皮层处理声音，视觉皮层处理图像，前额叶做推理，运动皮层控制说话——它们是**分工合作**的。

MiniMind-O 的 Thinker-Talker 就是这个思路：- **Thinker**（思考者）：理解文本、语音、图像，产生语义表示 - **Talker**（说话者）：把语义表示变成音频编码，产生流式语音

11.1.4 学习路线

节	内容	目标
11.1	从 TTS 到 Omni	理解为什么级联 ASR+LLM+TTS 不够好
11.2	Thinker-Talker 架构	理解语义路径与声学路径的分离
11.3	音频输入：冻结编码器	理解 SenseVoice 如何把声音变成特征
11.4	桥接层	理解为什么中间层比最终层更适合做条件
11.5	音频输出：Mimi Codec	理解 8 层码本如何表示声音
11.6	多 Token 预测 (MTP)	理解如何并行预测所有码本
11.7	序列格式	理解文本和 8 路音频如何共存于同一序列
11.8	流式生成	理解模型如何在生成未完成时就开始播放
11.9	声音克隆	理解参考音频如何控制输出音色
11.10	训练流水线	理解增量式能力引入策略
11.11	VAD 与打断	理解实时交互的工程实现
11.12	从零实现 SimpleOmni	完整代码走读
11.13	实验与对比	训练、评估、与工业模型对比

- 11.2 11.1 从 TTS 到 Omni
- 11.3 11.2 Thinker-Talker 架构
- 11.4 11.3 音频输入：冻结编码器
- 11.5 11.4 桥接层：为什么不用最后一层？
- 11.6 11.5 音频输出：Mimi Codec
- 11.7 11.6 多 Token 预测 (MTP)
- 11.8 11.7 序列格式：9 条流的故事
- 11.9 11.8 流式生成
- 11.10 11.9 声音克隆
- 11.11 11.10 训练流水线
- 11.12 11.11 VAD 与打断 (Barge-In)
- 11.13 11.12 从零实现 SimpleOmni
- 11.14 11.13 实验与对比

11.15 代码

ch11_minimind_o/
|—— README.md ← 你在这里

```
└── code/
    ├── model.py      ← SimpleOmni 教学实现
    ├── train.py      ← 训练脚本 (T2A + A2A)
    ├── inference.py   ← 推理演示 (文本/语音→语音)
    └── export_onnx.py ← ONNX 导出
```

11.16 参考资料

1. Gong, J. (2026). “MiniMind-O Technical Report: An Open Small-Scale Speech-Native Omni Model.” [arXiv:2605.03937](#)
2. [MiniMind-O GitHub](#) (Apache 2.0)
3. [MiniMind LLM](#) — 语言模型基础
4. [Mimi Neural Codec](#) — Kyutai 音频编解码器
5. Qwen2.5-Omni Technical Report. [arXiv:2503.20215](#)

11.17 前置知识

- **Ch06 (Neural Codec)**: Mimi 就是一种神经编解码器
- **Ch07 (VALL-E)**: 把语音当语言建模的核心思想
- **Ch08 (Modern Models)**: 工业级模型的架构概览